CS111 - Scientific Computing Basics

Spring 2025

UCSB

Contents

1 Fundamentals of Linear Algebra **4**

1.1 Vectors and Matrices 4

1.2 Matrix Arithmetic 8

1.2.1 Addition and Subtraction 8

1.2.2 Dot Product 10

1.2.3 Matrix Inverse 14

2 Linear Systems of Equations **15**

2.1 Linearity 15

2.1.1 Linear Systems 16

2.1.2 Rank 18

2.2 LU Factorization 21

2.2.1 LU Factoriazation and Gaussian Elimination 22

2.2.2 Pivoting and LUP Decomposition 24

2.2.3 Solving Linear Systems using LU Factorization 26

2.3 Floating Point Numbers and Errors 28

3 Eigenvalues and Matrix Properties **39**
3.1 Eigenvalues and Eigenvectors . . . 39
3.2 Symmetric Positive Definite (SPD) Matrices . . . 43
3.2.1 Cholesky Decomposition . . . 45
3.3 Orthogonal Matrices . . . 49
3.3.1 QR Decomposition . . . 51
3.3.2 Grahm Schmidt . . . 52

4 Iterative Methods for Linear Systems **58**
4.1 Iterative Methods for Linear Systems . . . 58
4.2 Jacobi Iteration . . . 60
4.2.1 Jacobi Iteration . . . 60
4.2.2 Convergence of Jacobi Iteration . . . 61
4.2.3 Stopping Criteria . . . 62
4.3 Conjugate Gradient Method . . . 63
4.3.1 From Linear Systems to Optimization . . . 63
4.3.2 **A**-Conjugate Directions . . . 64
4.3.3 The CG Algorithm and Convergence . . . 65
4.3.4 Preconditioning . . . 67
4.4 Stability and Condition Number . . . 68
4.4.1 Why Not Invert Directly? . . . 68
4.4.2 Numerical Stability . . . 68
4.4.3 Condition Number . . . 69
4.4.4 Conditioning vs. Stability . . . 70
4.4.5 Stability of Matrix Factorizations . . . 71

5 Advanced Matrix Decompositions **73**
5.1 Singular Value Decomposition and the Four Fundamental Subspaces . . . 74
5.1.1 The Four Fundamental Subspaces . . . 74
5.1.2 Rank Truncation and the Eckart-Young-Mirsky Theorem . . . 75
5.2 Matrix Norms . . . 77
5.3 Overdetermined Systems and Least Squares . . . 80
5.3.1 Overdetermined Systems . . . 81
5.3.2 Normal Equations . . . 81
5.3.3 QR-Based Least Squares . . . 82

6 Applications of Numerical Linear Algebra **84**
6.1 Principal Component Analysis via SVD 84
6.1.1 Principal Components as Variance-Maximizing Directions 84
6.1.2 The SVD-PCA Connection 85
6.1.3 Dimensionality Reduction 86
6.1.4 Statistical Interpretation 86
6.2 Spectral Graph Theory 88
6.3 The PageRank Algorithm 94

1 Fundamentals of Linear Algebra

1.1 Vectors and Matrices

Def $\mathbb{R}$**.** The set of all real numbers. Real numbers form the backbone of most numerical computations and serve as the primary field in many applications across science and engineering.

Def $\mathbb{Z}$**.** The set of all integers. Integers are important for problems involving countable or discrete quantities and are a subset of the real numbers.

Def $\mathbb{C}$**.** The set of all complex numbers. Although our primary focus is on real scalars, complex numbers extend the idea of numbers and are essential in advanced topics such as signal processing, control theory, and quantum mechanics.

Def **Scalars.** Elements of number spaces (e.g., $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Z}$) are called **scalars**. They represent individual quantities and serve as the fundamental building blocks from which vectors and matrices are constructed.

Scalars can be viewed as the atoms of numerical computation. In advanced studies, such as functional analysis or numerical linear algebra, understanding the properties of scalars is crucial, particularly when extending these concepts to infinite-dimensional spaces or complex systems.

Def **Vectors.** A **vector** is a one-dimensional array of scalars. Vectors can represent points, directions, or more abstract quantities within a vector space, and they are fundamental in representing data and linear relationships.

Note Vectors may be represented either as row vectors or column vectors. Although both forms contain the same information, the distinction becomes important in the context of linear transformations, dual spaces, and inner product spaces.

Example

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1+2i \\ 7i \\ 3 \end{pmatrix}, \ldots$$

Just as scalars belong to number systems like $\mathbb{R}$ or $\mathbb{C}$, vectors inhabit **vector spaces**. While these notes treat vector spaces in an informal manner, later chapters will provide a rigorous formulation of vector spaces and their properties.

Def $\mathbb{R}^n$**.** The set of all real vectors of size n. When we write $x \in \mathbb{R}^n$, it indicates that x is a vector with n real entries. Vectors in $\mathbb{R}^n$ are typically expressed in one of

the following forms:

$$\boldsymbol{x} \in \mathbb{R}^{n\times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad \text{(column vector)}$$

$$\boldsymbol{x} \in \mathbb{R}^{1\times n} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}, \quad \text{(row vector)}$$

For these notes, the column vector form is adopted as the default representation.

Note In our discussion, we focus on finite-dimensional vector spaces. In more advanced topics, infinite-dimensional vector spaces—central to functional analysis—play a significant role in areas such as differential equations and quantum mechanics.

Def **Matrices.** A **matrix** is a two-dimensional (or higher-dimensional) array of scalars. A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is typically written as

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Matrices are used to represent linear transformations, systems of linear equations, and more complex data structures.

Example Examples of two-dimensional matrices include:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{pmatrix}, \quad \text{etc.}$$

Note Matrices can also be interpreted as vectors whose entries are themselves vectors. For instance,

$$\mathbf{A} = \begin{pmatrix} \text{---} & \boldsymbol{r}_1 & \text{---} \\ & \vdots & \\ \text{---} & \boldsymbol{r}_n & \text{---} \end{pmatrix} = \begin{pmatrix} | & & | \\ \boldsymbol{c}_1 & \cdots & \boldsymbol{c}_n \\ | & & | \end{pmatrix},$$

where:

1. $r_i = \mathbf{A}[i]$ denotes the *row vector* corresponding to the ith row of $\mathbf{A}$.
2. $c_i = \mathbf{A}[:, i]$ denotes the *column vector* corresponding to the ith column of $\mathbf{A}$.

This viewpoint is particularly useful when exploring advanced topics such as block matrices, tensor decompositions, and matrix factorization methods.

Def **Matrix Transpose.** The **transpose** of $\mathbf{A} \in \mathbb{R}^{m\times n}$, denoted $\mathbf{A}^T$, is the $n \times m$ matrix obtained by interchanging rows and columns:

$$(\mathbf{A}^T)_{ij} = a_{ji}, \quad 1 \le i \le n,\ 1 \le j \le m.$$

For a column vector $\boldsymbol{x} \in \mathbb{R}^n$, the transpose $\boldsymbol{x}^T$ is its corresponding row vector and vice versa.

Def **Symmetric and Skew-Symmetric Matrices.** A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is:

- **symmetric** if $\mathbf{A}^T = \mathbf{A}$, equivalently $a_{ij} = a_{ji}$ for all i, j.
- **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$, equivalently $a_{ij} = -a_{ji}$ for all i, j.

Note Every square matrix $\mathbf{A}$ decomposes uniquely into a symmetric part and a skew-symmetric part:

$$\mathbf{A} = \underbrace{\tfrac{1}{2}(\mathbf{A} + \mathbf{A}^T)}_{\text{symmetric}} + \underbrace{\tfrac{1}{2}(\mathbf{A} - \mathbf{A}^T)}_{\text{skew-symmetric}}.$$

Symmetric matrices arise constantly in scientific computing: covariance matrices in statistics, graph Laplacians in network problems, and Hessians of quadratic objectives in optimization are all symmetric.

Thm 1.1 *Theorem 1.1* **(Properties of the Transpose).** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{B} \in \mathbb{R}^{n\times p}$, and $\alpha \in \mathbb{R}$. Then:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$ (for conformable $\mathbf{A}, \mathbf{B}$)
3. $(\alpha\mathbf{A})^T = \alpha\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$ **(Reversal Law)**

Proof *Proof.* Properties (1)–(3) follow directly from the definition $(\mathbf{A}^T)_{ij} = a_{ji}$ applied entry-wise. For example, for (2):

$$(\mathbf{A} + \mathbf{B})^T_{ij} = (\mathbf{A} + \mathbf{B})_{ji} = a_{ji} + b_{ji} = (\mathbf{A}^T)_{ij} + (\mathbf{B}^T)_{ij} = (\mathbf{A}^T + \mathbf{B}^T)_{ij}.$$

For (4), let $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m\times p}$. By definition of transpose and matrix product:

$$(\mathbf{C}^T)_{ij} = c_{ji} = \sum_{k=1}^{n} a_{jk}\, b_{ki}.$$

Expanding $\mathbf{B}^T\mathbf{A}^T$ using $(\mathbf{B}^T)_{ik} = b_{ki}$ and $(\mathbf{A}^T)_{kj} = a_{jk}$:

$$(\mathbf{B}^T\mathbf{A}^T)_{ij} = \sum_{k=1}^{n} (\mathbf{B}^T)_{ik}\,(\mathbf{A}^T)_{kj} = \sum_{k=1}^{n} b_{ki}\,a_{jk}.$$

Both sums are identical, so $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$. □

Cor 1.2 *Corollary 1.2* **(Reversal Law for Multiple Factors).** By induction on the Reversal Law:

$$(\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_k)^T = \mathbf{A}_k^T\cdots\mathbf{A}_2^T\mathbf{A}_1^T.$$

Ex 1.1 Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$.

1. State the dimensions of $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{AA}^T$, and $\mathbf{A}^T\mathbf{A}$.
2. Compute $\mathbf{A}^T$ explicitly. Then compute $\mathbf{AA}^T$ and verify it is symmetric.
3. In NumPy (0-based indexing), write the expression to retrieve element a_{12} (first row, second column of the mathematical matrix). Then write a one-liner for $\mathbf{AA}^T$.
4. Is $\mathbf{AA}^T = \mathbf{A}^T\mathbf{A}$ in general? What can you say about each product individually?

Ex 1.2 Let $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$ be a nonzero column vector.

1. The **outer product** is $\boldsymbol{x}\boldsymbol{x}^T \in \mathbb{R}^{n\times n}$. Write out entry (i, j) explicitly.
2. Use the Reversal Law to show $\boldsymbol{x}\boldsymbol{x}^T$ is always symmetric.
3. What is the rank of $\boldsymbol{x}\boldsymbol{x}^T$? Justify by arguing about the column space. (*Hint: every column of* $\boldsymbol{x}\boldsymbol{x}^T$ *is a scalar multiple of* $\boldsymbol{x}$.)
4. The **inner product** is $\boldsymbol{x}^T\boldsymbol{x} \in \mathbb{R}$. Express it as a sum. Why is $\boldsymbol{x}^T\boldsymbol{x} \geq 0$ with equality iff $\boldsymbol{x} = \mathbf{0}$? How does this relate to the Euclidean length of $\boldsymbol{x}$?

1.2 Matrix Arithmetic

We are all familiar with scalar arithmetic, including operations like addition, subtraction, and multiplication, as well as fundamental properties such as associativity and distributivity. In the following sections, we extend these familiar concepts to vectors and matrices, exploring how these operations are defined and applied in higher dimensions.

1.2.1 Addition and Subtraction

Matrix addition is performed element-wise. Note that matrices can only be added (or subtracted) if they have the same dimensions.

Def **Matrix Sum.** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$ be matrices. Their sum is defined as

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} (a+b)_{11} & \cdots & (a+b)_{1n} \\ \vdots & \ddots & \vdots \\ (a+b)_{n1} & \cdots & (a+b)_{nn} \end{pmatrix},$$

where each entry satisfies $(a+b)_{ij} = a_{ij} + b_{ij}$ for $1 \le i \le m$ and $1 \le j \le n$.

Note For two matrices $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{p\times q}$, the sum $\mathbf{A}+\mathbf{B}$ is defined if and only if $m = p$ and $n = q$.

Commutativity and Associativity of Matrix Addition

Let

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} \quad \text{and} \quad \mathbf{B}+\mathbf{A} = \begin{pmatrix} e+a & f+b \\ g+c & h+d \end{pmatrix}.$$

Since addition of real numbers is commutative, we have $\mathbf{A}+\mathbf{B} = \mathbf{B}+\mathbf{A}$. Furthermore, for any third matrix $\mathbf{C} = \begin{pmatrix} i & j \\ k & l \end{pmatrix}$,

$$(\mathbf{A}+\mathbf{B})+\mathbf{C} = \begin{pmatrix} (a+e)+i & (b+f)+j \\ (c+g)+k & (d+h)+l \end{pmatrix},$$

and

$$\mathbf{A}+(\mathbf{B}+\mathbf{C}) = \begin{pmatrix} a+(e+i) & b+(f+j) \\ c+(g+k) & d+(h+l) \end{pmatrix}.$$

By associativity of addition for real numbers, these two results are identical. Hence, matrix addition is both commutative and associative.

Thm 1.3

Theorem 1.3 **($\mathbb{R}^{m\times n}$ as a Vector Space).** The set $\mathbb{R}^{m\times n}$ equipped with matrix addition and scalar multiplication forms a real vector space of dimension mn.

Proof *Proof.* Each axiom follows from the corresponding property of $\mathbb{R}$ applied entry-wise. For **commutativity**:

$$(\mathbf{A}+\mathbf{B})_{ij} = a_{ij} + b_{ij} = b_{ij} + a_{ij} = (\mathbf{B}+\mathbf{A})_{ij}.$$

The **additive identity** is $\mathbf{0}$ since $(\mathbf{A}+\mathbf{0})_{ij} = a_{ij} + 0 = a_{ij}$. The **additive inverse** of $\mathbf{A}$ is $-\mathbf{A}$ (negate every entry), satisfying $(\mathbf{A}+(-\mathbf{A}))_{ij} = 0$.
For the **dimension**: the mn matrices $\mathbf{E}^{(ij)}$ with a 1 in position (i,j) and zeros elsewhere are linearly independent and every $\mathbf{A} \in \mathbb{R}^{m\times n}$ decomposes as $\mathbf{A} = \sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}\,\mathbf{E}^{(ij)}$, so they form a basis of size mn. $\square$

Def **Scalar Multiplication.** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $c \in \mathbb{R}$. Then scalar multiplication is defined as

$$c\mathbf{A} = c\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} ca_{11} & \cdots & ca_{1n} \\ \vdots & \ddots & \vdots \\ ca_{n1} & \cdots & ca_{nn} \end{pmatrix},$$

where each entry satisfies $(ca)_{ij} = c \cdot a_{ij}$.

Distributivity of Scalar Multiplication over Matrix Addition

Let $\alpha \in \mathbb{R}$ and

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\alpha(\mathbf{A}+\mathbf{B}) = \alpha\begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} = \begin{pmatrix} \alpha(a+e) & \alpha(b+f) \\ \alpha(c+g) & \alpha(d+h) \end{pmatrix}.$$

Since scalar multiplication distributes over addition for real numbers, this equals

$$\begin{pmatrix} \alpha a+\alpha e & \alpha b+\alpha f \\ \alpha c+\alpha g & \alpha d+\alpha h \end{pmatrix} = \alpha\mathbf{A}+\alpha\mathbf{B}.$$

1.2.2 Dot Product

Def **Dot Product.** For vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^n$, the dot product (or inner product) is defined as

$$\boldsymbol{a}\cdot\boldsymbol{b} = \boldsymbol{a}^t\boldsymbol{b} = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix}\begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} = \sum_{i=1}^{n} a_i b_i. \tag{1}$$

Def **Matrix Product.** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$. Their product $\mathbf{AB}$ is an $m \times p$ matrix defined by

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj},$$

or equivalently, if we consider the rows of $\mathbf{A}$ as vectors $\boldsymbol{a}_i$ and the columns of $\mathbf{B}$ as vectors $\boldsymbol{b}_j$, then

$$\mathbf{AB} = \begin{pmatrix} \boldsymbol{a}_1\cdot\boldsymbol{b}_1 & \cdots & \boldsymbol{a}_1\cdot\boldsymbol{b}_p \\ \vdots & \ddots & \vdots \\ \boldsymbol{a}_m\cdot\boldsymbol{b}_1 & \cdots & \boldsymbol{a}_m\cdot\boldsymbol{b}_p \end{pmatrix}.$$

Note Matrix multiplication is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. Moreover, it is associative and distributive over addition, but in general it is not commutative; that is, $\mathbf{AB} \neq \mathbf{BA}$ for most matrices.

Lem 1.4 *Lemma 1.4* **(Computational Cost of Matrix Multiplication).** Computing $\mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$ requires exactly mnp multiplications and $m(n-1)p$ additions using the standard algorithm, giving $O(mnp)$ floating-point operations (flops). For two square $n \times n$ matrices this reduces to $O(n^3)$ flops.

Note The $O(n^3)$ cost of naive matrix multiplication is a central bottleneck in scientific computing. Strassen's algorithm reduces this to $O(n^{\log_2 7}) \approx O(n^{2.807})$, but is rarely used in practice due to numerical stability concerns and cache-unfriendly memory access. Modern BLAS/LAPACK implementations approach peak hardware throughput via blocking strategies, but the asymptotic cost remains $O(n^3)$. For many applications, avoiding explicit matrix-matrix products entirely (e.g., exploiting sparsity or factored structure) is more impactful than algorithmic improvements.

Compatibility for Matrix Multiplication

Let $\mathbf{A}$ be a 3×4 matrix and $\mathbf{B}$ be a 4×2 matrix. Since the number of columns in $\mathbf{A}$ (4) equals the number of rows in $\mathbf{B}$ (4), the product $\mathbf{AB}$ is defined and will result in a 3×2 matrix. Conversely, if $\mathbf{B}$ were instead a 5×2 matrix, the product would be undefined.

Distributive Property of Matrix Multiplication

Let

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}.$$

Then,

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A} \begin{pmatrix} b_{11} + c_{11} & b_{12} + c_{12} \\ b_{21} + c_{21} & b_{22} + c_{22} \end{pmatrix},$$

and by computing the entries using the definition of matrix multiplication, one can verify that

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}.$$

Non-Commutativity of Matrix Multiplication

Consider the matrices

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then,

$$\mathbf{AB} = \begin{pmatrix} 1 \cdot 0 + 0 \cdot 1 & 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 0 + 2 \cdot 1 & 0 \cdot 1 + 2 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix},$$

and

$$\mathbf{BA} = \begin{pmatrix} 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 0 + 1 \cdot 2 \\ 1 \cdot 1 + 0 \cdot 0 & 1 \cdot 0 + 0 \cdot 2 \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Since $\mathbf{AB} \neq \mathbf{BA}$ and clearly $\mathbf{A} \neq \mathbf{B}$, this example shows that matrix multiplication is not commutative.

Ex 1.3 Let $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$.

1. Compute $\mathbf{AB}$ and $\mathbf{BA}$ by hand. Confirm $\mathbf{AB} \neq \mathbf{BA}$.
2. Find a nonzero matrix $\mathbf{C} \neq \mathbf{I}$ that commutes with $\mathbf{A}$ (i.e., $\mathbf{AC} = \mathbf{CA}$). *Hint: diagonal matrices always commute with diagonal matrices. What if you try* $\mathbf{C} = \alpha\mathbf{A} + \beta\mathbf{I}$*?*
3. Verify associativity: compute $(\mathbf{AB})\boldsymbol{c}$ and $\mathbf{A}(\mathbf{B}\boldsymbol{c})$ for $\boldsymbol{c} = (1, 0)^T$ and confirm they are equal.
4. For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\boldsymbol{c} \in \mathbb{R}^n$, compare the flop count of $(\mathbf{AB})\boldsymbol{c}$ versus $\mathbf{A}(\mathbf{B}\boldsymbol{c})$. Which order is cheaper and by what factor?

Many properties of scalar arithmetic extend naturally to both vectors and matrices. In fact, matrices can be viewed as elements of a higher-dimensional vector space, and matrix multiplication can be interpreted as the composition of linear transformations. While the notations for vectors and matrices are similar, their roles in linear algebra differ:

- Vectors typically represent points or directions in space.
- Matrices often represent linear maps or transformations between vector spaces.

This conceptual distinction becomes crucial when discussing topics such as vector spaces, eigenvalues, and linear transformations.

Ex 1.4 **(Associativity and Computational Cost)** Suppose $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$. We wish to compute $\mathbf{AB}\boldsymbol{x}$.

1. *Approach 1:* Compute $\mathbf{M} = \mathbf{AB}$ first, then $\mathbf{M}\boldsymbol{x}$. What is the total flop count?
2. *Approach 2:* Compute $\mathbf{A}(\mathbf{B}\boldsymbol{x})$ via two sequential matrix-vector products. What is the total flop count?
3. For large n, which approach is more efficient and by what asymptotic factor?
4. Write NumPy code for both approaches using `@`. Does NumPy automatically choose the optimal evaluation order for a chain like `A @ B @ x`?

1.2.3 Matrix Inverse

Def **Matrix Inverse.** For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the inverse of $\mathbf{A}$, denoted $\mathbf{A}^{-1}$, is defined as the unique matrix satisfying

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. A matrix that has an inverse is called *invertible* or *nonsingular*.

Note A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$, which also implies that $\mathbf{A}$ has full rank.

Uniqueness of the Inverse

Suppose $\mathbf{A}$ is an invertible matrix and that $\mathbf{B}$ and $\mathbf{C}$ are both inverses of $\mathbf{A}$; that is,

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I} \quad \text{and} \quad \mathbf{AC} = \mathbf{CA} = \mathbf{I}.$$

Then,

$$\mathbf{B} = \mathbf{BI} = \mathbf{B}(\mathbf{AC}) = (\mathbf{BA})\mathbf{C} = \mathbf{IC} = \mathbf{C}.$$

Thus, the inverse of $\mathbf{A}$ is unique.

Thm 1.5 *Theorem 1.5* **(Properties of the Matrix Inverse).** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ be invertible and $\alpha \in \mathbb{R} \setminus \{0\}$. Then:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ **(Reversal Law)**
3. $(\alpha\mathbf{A})^{-1} = \frac{1}{\alpha}\mathbf{A}^{-1}$
4. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Proof *Proof.* **(1):** $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ and $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ show $\mathbf{A}$ is both a left and right inverse of $\mathbf{A}^{-1}$, so $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$.
(2): Check the right-inverse condition:

$$(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} = \mathbf{A}\mathbf{I}\mathbf{A}^{-1} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

Left-inverse: $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{B}^{-1}(\mathbf{A}^{-1}\mathbf{A})\mathbf{B} = \mathbf{B}^{-1}\mathbf{I}\mathbf{B} = \mathbf{I}$. By uniqueness of the inverse, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.
(3): $(\alpha\mathbf{A})\left(\frac{1}{\alpha}\mathbf{A}^{-1}\right) = \frac{\alpha}{\alpha}\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.
(4): Transpose both sides of $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ using the Reversal Law: $(\mathbf{A}^{-1})^T\mathbf{A}^T = \mathbf{I}$, so $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$. $\square$

Note

Property (2) is critical in scientific computing. To solve $(\mathbf{AB})\boldsymbol{x} = \boldsymbol{c}$, we should *not* compute $(\mathbf{AB})^{-1}$ explicitly (cost: $O(n^3)$ with a large constant and numerical error amplification). Instead, use the reversal law: first solve $\mathbf{A}\boldsymbol{y} = \boldsymbol{c}$ for $\boldsymbol{y}$ via LU factorization, then solve $\mathbf{B}\boldsymbol{x} = \boldsymbol{y}$. The function `np.linalg.solve` does exactly this.

Ex 1.5

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$, compute $\mathbf{A}^{-1}$ by hand using the 2×2 formula $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Verify by checking $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.
2. Given $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$, use the Reversal Law to compute $(\mathbf{AB})^{-1}$ without re-inverting a 2×2 system from scratch.
3. Explain why `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Address both numerical accuracy and computational cost.
4. True or False: if $\mathbf{A}^2 = \mathbf{I}$, then $\mathbf{A}^{-1} = \mathbf{A}$. Prove your answer and give a concrete 2×2 example beyond $\mathbf{I}$ itself.

Ex 1.6 **(Identifying Invertibility)** For each matrix below, determine without computing the inverse whether it is invertible. Give a one-sentence justification.

1. $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$
2. $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
3. $\begin{pmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & -1 \end{pmatrix}$
4. $\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}$

2 Linear Systems of Equations

2.1 Linearity

Linearity is a fundamental concept in numerical linear algebra that underpins many of the algorithms and methods discussed in this course. At its core, linearity describes mathematical structures that behave in a predictable way under addition and scalar multiplication.

A mapping or operation L is considered linear if it satisfies two key properties:

- **Additivity**: $L(\mathbf{x}+\mathbf{y}) = L(\mathbf{x}) + L(\mathbf{y})$ for all inputs $\mathbf{x}$ and $\mathbf{y}$
- **Homogeneity**: $L(\alpha\mathbf{x}) = \alpha L(\mathbf{x})$ for all inputs $\mathbf{x}$ and all scalars α

These properties can be combined into the principle of superposition: $L(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha L(\mathbf{x}) + \beta L(\mathbf{y})$ for all inputs $\mathbf{x}$ and $\mathbf{y}$, and all scalars α and β.

Linear mappings preserve vector operations, which allows us to decompose complex problems into simpler ones and then combine the solutions. This property is what makes linear algebra so powerful for a wide range of applications, from solving systems of equations to approximating functions, analyzing data, and modeling physical phenomena.

In numerical linear algebra, we often represent linear mappings as matrices. The multiplication of a matrix $\mathbf{A}$ by a vector $\mathbf{x}$ is a linear operation, making matrices the perfect tool for expressing and analyzing linear transformations. The computational challenges that arise in numerical linear algebra—such as solving linear systems, finding eigenvalues, or computing matrix decompositions—all fundamentally involve manipulating these linear structures efficiently and accurately.

2.1.1 Linear Systems

Def **Linear Systems .** A **system of linear equations** is a collection of equations in which each equation is a linear combination of the unknown variables. Such a system can be written in the form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + &\quad \cdots \quad + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + &\quad \cdots \quad + a_{2n}x_n = b_2, \\ &\quad \vdots \\ a_{m1}x_1 + a_{m2}x_2 + &\quad \cdots \quad + a_{mn}x_n = b_m, \end{aligned}$$

where each a_{ij} (with $1 \le i \le m$ and $1 \le j \le n$) is a known constant and $x_1, \ldots, x_n$ are the unknowns. In matrix-vector form, this system is succinctly expressed as

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b},$$

where

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}.$$

The mapping $T : \mathbb{R}^n \to \mathbb{R}^m$ defined by $T(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x}$ is a linear transformation. That is, for any vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and scalar $c \in \mathbb{R}$, we have:

$$T(\boldsymbol{x} + \boldsymbol{y}) = \mathbf{A}(\boldsymbol{x} + \boldsymbol{y}) = \mathbf{A}\boldsymbol{x} + \mathbf{A}\boldsymbol{y} = T(\boldsymbol{x}) + T(\boldsymbol{y}),$$

$$T(c\boldsymbol{x}) = \mathbf{A}(c\boldsymbol{x}) = c\,\mathbf{A}\boldsymbol{x} = c\,T(\boldsymbol{x}).$$

These properties, known as additivity and homogeneity, are the defining features of linear maps.

Thm 2.1 *Theorem 2.1* **(Matrix-Vector Multiplication is Linear).** The map $T : \mathbb{R}^n \to \mathbb{R}^m$ defined by $T(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x}$ is a linear transformation. That is, for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$:

$$T(\boldsymbol{x} + \boldsymbol{y}) = T(\boldsymbol{x}) + T(\boldsymbol{y}), \qquad T(\alpha\boldsymbol{x}) = \alpha T(\boldsymbol{x}).$$

Equivalently, $T(\alpha\boldsymbol{x} + \beta\boldsymbol{y}) = \alpha T(\boldsymbol{x}) + \beta T(\boldsymbol{y})$ for all $\alpha, \beta \in \mathbb{R}$.

Proof *Proof.* By distributivity of matrix multiplication: $\mathbf{A}(\boldsymbol{x} + \boldsymbol{y}) = \mathbf{A}\boldsymbol{x} + \mathbf{A}\boldsymbol{y} = T(\boldsymbol{x}) + T(\boldsymbol{y})$. By scalar compatibility: $\mathbf{A}(\alpha\boldsymbol{x}) = \alpha(\mathbf{A}\boldsymbol{x}) = \alpha T(\boldsymbol{x})$. □

Def **Existence and Uniqueness .** For a square system (i.e., when $m = n$), the linear system

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$$

has a unique solution if and only if the coefficient matrix $\mathbf{A}$ is invertible, which is equivalent to $\det(\mathbf{A}) \neq 0$. If $\mathbf{A}$ is singular (i.e., noninvertible, $\det(\mathbf{A}) = 0$), then the system either has no solution or has infinitely many solutions. In the latter case, the set of solutions forms an affine subspace of $\mathbb{R}^n$.

Thm 2.2 *Theorem 2.2* **(Equivalences for Invertibility).** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$. The following are equivalent:

1. $\mathbf{A}$ is invertible.
2. $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a unique solution for every $\boldsymbol{b} \in \mathbb{R}^n$.
3. $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ has only the trivial solution $\boldsymbol{x} = \mathbf{0}$.
4. $\operatorname{rank}(\mathbf{A}) = n$.
5. The columns of $\mathbf{A}$ are linearly independent.

Proof *Proof.* (1) $\Rightarrow$ (2): Set $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$; then $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. If $\mathbf{A}\boldsymbol{x}_1 = \mathbf{A}\boldsymbol{x}_2$, then $\mathbf{A}(\boldsymbol{x}_1 - \boldsymbol{x}_2) = \mathbf{0}$, so $\boldsymbol{x}_1 - \boldsymbol{x}_2 = \mathbf{A}^{-1}\mathbf{0} = \mathbf{0}$.
(2) $\Rightarrow$ (3): Applying (2) with $\boldsymbol{b} = \mathbf{0}$ forces the unique solution to be $\boldsymbol{x} = \mathbf{0}$.
(3) $\Rightarrow$ (4): Null space of $\mathbf{A}$ is $\{\mathbf{0}\}$, so $\operatorname{nullity}(\mathbf{A}) = 0$. By the Rank-Nullity Theorem, $\operatorname{rank}(\mathbf{A}) = n$.
(4) $\Rightarrow$ (1): A square matrix with rank n has n linearly independent columns that span $\mathbb{R}^n$, which is the condition for invertibility. □

Def **Linear Combination.** A **linear combination** of vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ is any vector $\boldsymbol{y}$ that can be written as

$$\boldsymbol{y} = \alpha_1 \boldsymbol{x}_1 + \alpha_2 \boldsymbol{x}_2 + \cdots + \alpha_n \boldsymbol{x}_n,$$

where $\alpha_1, \ldots, \alpha_n \in \mathbb{R}$.

Note View each vector as an ingredient; a linear combination mixes these ingredients in various proportions to produce new vectors.

Def **Linear Dependence.** A set of vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ is **linearly dependent** if there exist scalars $\alpha_1, \ldots, \alpha_n$, not all zero, such that

$$\alpha_1 \boldsymbol{x}_1 + \alpha_2 \boldsymbol{x}_2 + \cdots + \alpha_n \boldsymbol{x}_n = \mathbf{0}.$$

If the only solution is $\alpha_1 = \cdots = \alpha_n = 0$, the vectors are **linearly independent**.

Note Linear dependence indicates redundancy; one or more vectors in the set can be expressed as a combination of the others.

Def **Basis.** A **basis** for a vector space $\mathcal{V}$ is a set of vectors that is both linearly independent and spanning. Every vector in $\mathcal{V}$ can be uniquely expressed as a linear combination of the basis vectors. The number of basis vectors is the **dimension** of $\mathcal{V}$.

2.1.2 Rank

Def **Rank.** The **rank** of a matrix $\mathbf{A}$ is defined as the maximum number of linearly independent rows (or equivalently, columns) of $\mathbf{A}$. Equivalently, it is the dimension of the row space (or column space) of $\mathbf{A}$. For an $m \times n$ matrix, the rank is always less than or equal to $\min(m, n)$.

Note Intuitively, the rank measures the amount of "information" contained in the matrix. It indicates the maximum number of independent directions or components that the matrix can represent. For instance, if a matrix has a rank of r, then there are r rows (or columns) that contribute unique information, while the remaining rows (or columns) can be expressed as linear combinations of these.

Note For an $m \times n$ matrix, since there are m rows and n columns, the rank cannot exceed either m or n. In particular, if $\mathbf{A}$ is square (i.e., $m = n$) and full rank (i.e., rank $= n$), then $\mathbf{A}$ is invertible.

Def **Full Rank.** A matrix is said to be **full rank** if its rank is as large as possible, that is, if $\text{rank}(\mathbf{A}) = \min(m, n)$. In the case of a square matrix, full rank implies invertibility.

Example **Example: Determining the Rank of a Matrix**

Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{pmatrix}.$$

Observe that the second row is exactly twice the first row, suggesting a dependency. To determine the rank, we perform elementary row operations to obtain the row-echelon form:

Step 1: Leave the first row unchanged:

$$R_1 = (1, 2, 3).$$

Step 2: Replace the second row by subtracting $2R_1$:

$$R_2 \to R_2 - 2R_1 : \quad (2, 4, 6) - 2(1, 2, 3) = (0, 0, 0).$$

Step 3: Replace the third row by subtracting R_1:

$$R_3 \to R_3 - R_1 : \quad (1, 1, 1) - (1, 2, 3) = (0, -1, -2).$$

The matrix becomes:

$$\mathbf{A}' = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{pmatrix}.$$

Swap R_2 and R_3 to bring the nonzero row up:

$$\mathbf{A}'' = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{pmatrix}.$$

Finally, multiply the second row by -1:

$$\mathbf{A}''' = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{pmatrix}.$$

Since there are two nonzero rows (pivot rows), the rank of $\mathbf{A}$ is 2.

Def **Rank-Nullity Theorem.** For any $m \times n$ matrix $\mathbf{A}$,

$$\operatorname{rank}(\mathbf{A}) + \dim(\mathbf{A}) = n,$$

where the nullity of $\mathbf{A}$ is the dimension of its null space. This relationship is fundamental in understanding the solution structure of the homogeneous system $\mathbf{A}\boldsymbol{x} = \mathbf{0}$.

Thm 2.3

Theorem 2.3 **(Rank-Nullity Theorem).** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$,

$$\operatorname{rank}(\mathbf{A}) + \operatorname{nullity}(\mathbf{A}) = n,$$

where $\operatorname{nullity}(\mathbf{A}) = \dim(\ker(\mathbf{A}))$ and $\ker(\mathbf{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$.

Note For a square $n \times n$ system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the Rank-Nullity Theorem gives a complete picture:

- $\operatorname{rank}(\mathbf{A}) = n$: the null space is $\{\mathbf{0}\}$, so $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a unique solution.
- $\operatorname{rank}(\mathbf{A}) = r < n$: if a solution exists, there are infinitely many — the solution set is an affine subspace of dimension $n - r$.

In NumPy, `np.linalg.matrix_rank(A)` computes the rank numerically (via SVD).

Note In many applications such as data compression, signal processing, and machine learning, the rank of a matrix is used to identify redundancy in data. Low-rank approximations can help reduce dimensionality while preserving essential information.

Def **Linear Span.** The **linear span** of a set of vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$, denoted $\operatorname{span}\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$, is the set of all linear combinations of these vectors:

$$\operatorname{span}\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\} = \{\alpha_1\boldsymbol{x}_1 + \cdots + \alpha_n\boldsymbol{x}_n : \alpha_i \in \mathbb{R}\}.$$

This is the smallest subspace of the vector space that contains the given vectors.

Example **Basis for span$\{(1,2),(3,4)\}$ in $\mathbb{R}^2$**

Consider the vectors $\boldsymbol{v}_1 = (1,2)$ and $\boldsymbol{v}_2 = (3,4)$. To check their linear independence, assume there exist scalars α and β such that

$$\alpha(1,2) + \beta(3,4) = (0,0).$$

This yields the system:

$$\alpha + 3\beta = 0, \quad 2\alpha + 4\beta = 0.$$

Multiplying the first equation by 2 gives $2\alpha + 6\beta = 0$. Subtracting the second equation, we have

$$(2\alpha + 6\beta) - (2\alpha + 4\beta) = 2\beta = 0 \quad \Rightarrow \quad \beta = 0.$$

Substituting back, $\alpha = 0$. Since the only solution is $\alpha = \beta = 0$, the vectors are linearly independent. Hence, $\{(1,2),(3,4)\}$ forms a basis for $\operatorname{span}\{(1,2),(3,4)\}$, which in this case is all of $\mathbb{R}^2$.

Ex 2.1 Consider the system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & -1 \\ 2 & 5 & 0 \\ -1 & 1 & 6 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 4 \\ 3 \end{pmatrix}.$$

1. Determine the rank of $\mathbf{A}$ by row-reducing the augmented matrix $[\mathbf{A} \mid \boldsymbol{b}]$.
2. How many solutions does this system have? Justify using the Rank-Nullity Theorem.
3. Find the solution(s).
4. In NumPy, solve using `np.linalg.solve(A, b)` and check the residual `np.linalg.norm(b - A @ x)`. What residual do you expect for a consistent system?

Ex 2.2 **(Linear Independence and the Null Space)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$ (a wide matrix).

1. Argue that the columns of $\mathbf{A}$ must be linearly dependent. (*Hint: use Rank-Nullity with rank*$(\mathbf{A}) \leq m < n$.)
2. What does this imply about the existence and uniqueness of solutions to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$?
3. Give a 2×3 example where $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has infinitely many solutions, and find the general solution as a particular solution plus the null space.
4. Can a wide system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ ever have no solution? Explain.

Ex 2.3 **(Superposition Principle)** Suppose $\boldsymbol{x}_1$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}_1$ and $\boldsymbol{x}_2$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}_2$.

1. Show that $\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2$ solves $\mathbf{A}\boldsymbol{x} = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2$.
2. Use this principle to solve $\mathbf{A}\boldsymbol{x} = 3\boldsymbol{b}_1 - 2\boldsymbol{b}_2$ without any new computation.
3. Why is this principle computationally valuable when solving $\mathbf{AX} = \mathbf{B}$ for a matrix right-hand side $\mathbf{B}$? How does `np.linalg.solve(A, B)` exploit this?

2.2 LU Factorization

The LU factorization expresses a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ as the product

$$\mathbf{A} = \mathbf{L}\,\mathbf{U},$$

where $\mathbf{L}$ is a lower triangular matrix with ones on the diagonal (a unit lower triangular matrix) and $\mathbf{U}$ is an upper triangular matrix. This factorization simplifies solving linear systems $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ by allowing the solution to be obtained via forward substitution (for $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$) followed by backward substitution (for $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$).

2.2.1 LU Factoriazation and Gaussian Elimination

Def **LU Factorization.** An **LU Factorization** of a matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is a decomposition of the form

$$\mathbf{A} = \mathbf{L}\,\mathbf{U},$$

with

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n1} & \ell_{n2} & \cdots & 1 \end{pmatrix} \quad \text{and} \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}.$$

Def **Gaussian elimination.** The procedure for LU factorization closely follows Gaussian elimination. For example, given

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix},$$

one begins by eliminating the entries below a_{11}. For $i = 2, \ldots, n$, set

$$\ell_{i1} = \frac{a_{i1}}{a_{11}},$$

and update row i by replacing it with

$$\text{row}_i \leftarrow \text{row}_i - \ell_{i1}\,\text{row}_1.$$

This results in zeros below a_{11} and new entries

$$a_{ij}^{(1)} = a_{ij} - \ell_{i1}\,a_{1j}, \quad j = 2, \ldots, n.$$

Repeating this process on the $(n-1)\times(n-1)$ submatrix produces the full LU decomposition.

Thm 2.4 *Theorem 2.4* **(LU Factorization Existence and Uniqueness).** A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ has a unique LU factorization (without pivoting) if and only if all leading principal submatrices $\mathbf{A}_k \in \mathbb{R}^{k\times k}$ (the top-left $k \times k$ blocks) are nonsingular for $k = 1, \ldots, n-1$.

Proof *Proof.* At step k of Gaussian elimination, the pivot is the (k,k) entry of the current reduced matrix, which equals $\det(\mathbf{A}_k)/\det(\mathbf{A}_{k-1})$ (an inductive result). This pivot is nonzero iff $\det(\mathbf{A}_k) \neq 0$. If all $n-1$ pivots are nonzero, elimination completes and produces a unique $\mathbf{L}, \mathbf{U}$ since each multiplier $\ell_{ik} = a_{ik}/a_{kk}$ and each updated entry are determined uniquely. $\square$

Thm 2.5 *Theorem 2.5* **(LU Factorization Cost).** Gaussian elimination to compute the LU factorization of $\mathbf{A} \in \mathbb{R}^{n\times n}$ costs

$$\frac{2}{3}n^3 + O(n^2) \text{ flops.}$$

Proof *Proof.* At elimination step k ($k = 1, \ldots, n-1$), we compute $n-k$ multipliers ($n-k$ divisions) and update an $(n-k)\times(n-k)$ submatrix ($2(n-k)^2$ flops for multiply-add pairs). The dominant cost is:

$$\sum_{k=1}^{n-1} 2(n-k)^2 = 2\sum_{j=1}^{n-1} j^2 = 2\cdot\frac{(n-1)n(2n-1)}{6} = \frac{2n^3}{3} + O(n^2).$$

□

Note The $\frac{2}{3}n^3$ constant is exactly twice the $\frac{1}{3}n^3$ cost for a triangular solve. So the factorization dominates: it costs $O(n^3)$ once, but each solve thereafter is only $O(n^2)$.

Example To illustrate, consider the 3×3 matrix

$$\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ -2 & -3 & 1 \end{pmatrix}.$$

We wish to decompose $\mathbf{A}$ as $\mathbf{A} = \mathbf{L}\,\mathbf{U}$ with

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}.$$

Begin by taking the first row of $\mathbf{A}$ as the first row of $\mathbf{U}$:

$$u_{11} = 2, \quad u_{12} = 3, \quad u_{13} = 1.$$

The multipliers for the first column are then

$$\ell_{21} = \frac{4}{2} = 2, \quad \ell_{31} = \frac{-2}{2} = -1.$$

After eliminating the first column, update the remaining rows. For row 2:

$$u_{22} = 7 - 2\cdot 3 = 1, \quad u_{23} = 3 - 2\cdot 1 = 1.$$

For row 3, the updated entries become

$$\tilde{a}_{32} = -3 - (-1)\cdot 3 = 0, \quad \tilde{a}_{33} = 1 - (-1)\cdot 1 = 2.$$

Thus, $\ell_{32} = 0$ and $u_{33} = 2$. Consequently, we have

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}.$$

A direct multiplication verifies that $\mathbf{L}\,\mathbf{U} = \mathbf{A}$.

Ex 2.4 Compute the LU factorization (without pivoting) of

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{pmatrix}.$$

1. Carry out the elimination steps explicitly: compute multipliers ℓ_{21}, ℓ_{31} at step 1, update the submatrix, then compute ℓ_{32} at step 2.
2. Write $\mathbf{L}$ and $\mathbf{U}$ explicitly. Verify $\mathbf{LU} = \mathbf{A}$.
3. Use the factorization to solve $\mathbf{A}\boldsymbol{x} = (5, 15, 4)^T$ via forward then backward substitution.
4. In Python, verify with `from scipy.linalg import lu` and `lu(A)`. Why does SciPy always return a permutation $\mathbf{P}$ even if no row swaps are needed?

2.2.2 Pivoting and LUP Decomposition

Def **Pivoting and LUP Decomposition.** When the matrix $\mathbf{A}$ has zeros or small entries in prospective pivot positions, it is necessary to perform row interchanges to maintain numerical stability. These interchanges are recorded by a permutation matrix $\mathbf{P}$, leading to the factorization

$$\mathbf{PA} = \mathbf{L}\,\mathbf{U},$$

which is known as the *LUP Decomposition*. Here, $\mathbf{L}$ is a unit lower triangular matrix and $\mathbf{U}$ is an upper triangular matrix.

Partial pivoting selects the largest available pivot (in absolute value) by interchanging (permuting) rows. This minimizes the risk of division by a small number and reduces the growth of rounding errors. Although complete pivoting, which interchanges both rows and columns, can further improve stability, it is generally more computationally expensive.

The permutation matrix $\mathbf{P}$ is orthogonal in the sense that $\mathbf{P}^T\mathbf{P} = \mathbf{I}$. In fact, for any permutation matrix $\mathbf{P}$, one has $\mathbf{P}^{-1} = \mathbf{P}^T$, reflecting that $\mathbf{P}$ merely rearranges the rows of the identity matrix.

Example

For example, consider the matrix

$$\mathbf{A} = \begin{pmatrix} 0 & 2 & 1 \\ 2 & 1 & 0 \\ 1 & 0 & 2 \end{pmatrix}.$$

Since the (1,1) entry is zero, we must permute the rows to secure a nonzero (and preferably large) pivot in the (1,1) position. Observing the first column, the largest absolute value is 2 (in row 2). Therefore, we swap row 1 with row 2 using the permutation matrix

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Then,

$$\mathbf{PA} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix}.$$

Next, we perform the LU factorization on $\mathbf{PA}$. Write

$$\mathbf{PA} = \mathbf{L}\,\mathbf{U},$$

with

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{pmatrix} \quad \text{and} \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}.$$

Assign the first row of $\mathbf{U}$ from $\mathbf{PA}$:

$$u_{11} = 2, \quad u_{12} = 1, \quad u_{13} = 0.$$

Then, the multipliers for the first column are

$$\ell_{21} = \frac{0}{2} = 0, \quad \ell_{31} = \frac{1}{2} = 0.5.$$

Subtracting 0.5 times the first row from the third row updates it to

$$\begin{pmatrix} 1 & 0 & 2 \end{pmatrix} - 0.5 \begin{pmatrix} 2 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -0.5 & 2 \end{pmatrix}.$$

For the 2×2 submatrix (rows 2 and 3), set

$$u_{22} = 2, \quad u_{23} = 1,$$

and compute the multiplier

$$\ell_{32} = \frac{-0.5}{2} = -0.25.$$

Finally, update the (3,3) entry:

$$u_{33} = 2 - (-0.25)(1) = 2.25.$$

Thus, we obtain

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.5 & -0.25 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2.25 \end{pmatrix}.$$

A multiplication of $\mathbf{L}$ and $\mathbf{U}$ confirms that $\mathbf{L}\,\mathbf{U} = \mathbf{PA}$.

2.2.3 Solving Linear Systems using LU Factorization

Once a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ has been factorized as $\mathbf{A} = \mathbf{LU}$, solving the linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ reduces to two simpler steps: forward substitution and backward substitution.

Def **LU Solve.** Given the LU factorization, we solve the system $\mathbf{A}\boldsymbol{x} = \mathbf{LU}\boldsymbol{x} = \boldsymbol{b}$ by first solving

$$\mathbf{L}\boldsymbol{y} = \boldsymbol{b},$$

and then solving

$$\mathbf{U}\boldsymbol{x} = \boldsymbol{y}.$$

Since $\mathbf{L}$ is a unit lower triangular matrix, we use forward substitution. Specifically, for $i = 1, 2, \ldots, n$,

$$y_i = b_i - \sum_{j=1}^{i-1} \ell_{ij}\, y_j.$$

After obtaining $\boldsymbol{y}$, we solve the upper triangular system using backward substitution. For $i = n, n-1, \ldots, 1$,

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^{n} u_{ij}\, x_j \right).$$

Note The forward substitution process requires roughly

$$\sum_{i=1}^{n} (i-1) = \frac{n(n-1)}{2} = O(n^2)$$

operations, and backward substitution has a similar computational cost. Thus, once the LU factorization is available, solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ takes $O(n^2)$ operations.

Note

It is important to note that while solving the system after the factorization is efficient, computing the LU factorization itself generally requires $O(n^3)$ operations. Therefore, if multiple right-hand sides $\boldsymbol{b}$ are to be solved with the same matrix $\mathbf{A}$, it is advantageous to factorize $\mathbf{A}$ once and reuse the factors $\mathbf{L}$ and $\mathbf{U}$ for each subsequent system (very common in iterative methods), reducing the per-system cost to $O(n^2)$.

Thm 2.6 *Theorem 2.6* **(Total Cost of LU-Based Solve).** Solving k linear systems $\mathbf{A}\boldsymbol{x}_i = \boldsymbol{b}_i$ $(i = 1, \ldots, k)$ with the same $\mathbf{A} \in \mathbb{R}^{n\times n}$ via LU factorization costs

$$\frac{2}{3}n^3 + 2kn^2 + O(n) \text{ flops},$$

versus $k \cdot \left(\frac{2}{3}n^3 + O(n^2)\right)$ if the matrix is re-factorized for each solve.

Ex 2.5

1. Derive the exact flop count for forward substitution (computing $\boldsymbol{y}$ from $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$): count the multiplications and additions at each step $i = 1, \ldots, n$. Show the total is $n^2 - n$ arithmetic operations.

2. Suppose $\mathbf{A}$ is 1000×1000 and you need to solve for 500 right-hand sides. How many flops does the LU-then-solve approach save compared to calling `np.linalg.solve` 500 times? (Assume each call re-factorizes.)

3. Give a concrete 2×2 matrix where Gaussian elimination without pivoting fails (division by zero) but LUP succeeds. Write out $\mathbf{P}$, $\mathbf{L}$, $\mathbf{U}$.

4. In Python, use `scipy.linalg.lu_factor(A)` once and `scipy.linalg.lu_solve` for each right-hand side. Write the code for 3 right-hand sides and verify each residual is near machine precision.

Ex 2.6 **(Stability of Gaussian Elimination)** Partial pivoting selects the largest pivot at each step.

1. The **growth factor** ρ of LU with partial pivoting is $\rho = \max_{i,j,k} |u_{ij}^{(k)}| / \max_{i,j} |a_{ij}|$, where $u_{ij}^{(k)}$ are the entries at elimination step k. For partial pivoting, $\rho \leq 2^{n-1}$. What is this for $n = 50$?

2. In practice ρ is almost always $O(n)$ or smaller. Does this make LU with partial pivoting *backward stable* in theory or just in practice?

3. For a matrix where partial pivoting performs poorly (the theoretical worst case), complete pivoting—interchanging both rows and columns—is guaranteed to have $\rho = O(n^{2/3})$ but is rarely used. What is the computational reason?

2.3 Floating Point Numbers and Errors

Computing with real numbers on finite machines introduces inherent approximation errors. Understanding these approximations and their consequences is fundamental to scientific computing. In this section, we exmplore floating point representation, the types of errors that arise in numerical computations, and techniques for analyzing and managing these errors.

Def **Floating Point Representation.** Floating point numbers are represented in computers using a format similar to scientific notation. A normalized floating point number has the form

$$x = (-1)^s \times (1.f) \times 2^e,$$

where:

- s is the sign bit (0 for positive, 1 for negative)
- f is the fractional part (or significand/mantissa)
- e is the exponent
- The leading 1 before the decimal point is implicit in most formats (hidden bit)

The IEEE 754 standard defines several formats, including:

- **Single precision (32 bits)**: 1 sign bit, 8 exponent bits, 23 fraction bits
- **Double precision (64 bits)**: 1 sign bit, 11 exponent bits, 52 fraction bits

The exponent field uses a biased representation, with bias $= 2^{(k-1)} - 1$ where k is the number of exponent bits. This allows representation of both very small and very large numbers.

Example **16-bit Floating Point Representation**: et's examine a simplified 16-bit floating point format with 1 sign bit, 5 exponent bits, and 10 fraction bits. The bias for the exponent would be $2^{(5-1)} - 1 = 15$. To represent the number -3.25 in this format:

1. Convert to sign-magnitude form: sign $= 1$ (negative)
2. Convert to binary: $3.25_{10} = 11.01_2$
3. Normalize: $1.101_2 \times 2^1$
4. Determine the biased exponent: $1 + 15 = 16 = 10000_2$
5. Encode the significand: 1.101_2 gives fraction bits 1010000000

6. Final representation: 1 10000 1010000000

To count the total number of representable values in this format:

- Special values: ± 0, $\pm\infty$, and NaN configurations
- Normal numbers: $2 \times (2^5 - 2) \times 2^{10} = 2 \times 30 \times 1024 = 61,440$
- Subnormal numbers: $2 \times 2^{10} = 2048$

The total is approximately $63,488+$ NaN configurations.

Def **Machine Epsilon ($\varepsilon_{\text{mach}}$).** Machine epsilon is a fundamental constant in floating point arithmetic, defined as the difference between 1 and the next representable floating point number. Equivalently, it is the smallest ε such that $1 + \varepsilon > 1$ in floating point arithmetic.
For a floating point system with p bits in the significand (including the hidden bit), machine epsilon is:

$$\varepsilon_{\text{mach}} = 2^{-p+1}$$

Machine epsilon determines the relative precision of the floating point system. For any normalized floating point number x, the gap to the next representable number is approximately $\varepsilon_{\text{mach}} \cdot |x|$.

Example **Calculating Machine Epsilon**: For IEEE double precision (64 bits):

- 52 fraction bits + 1 hidden bit = 53 bits of precision
- $\varepsilon_{\text{mach}} = 2^{-53+1} = 2^{-52} \approx 2.22 \times 10^{-16}$

Try verifying this experimentally!

Def **Unit Roundoff (u).** The unit roundoff u is half of machine epsilon:

$$u = \frac{\varepsilon_{\text{mach}}}{2} = 2^{-p}$$

Unit roundoff represents the maximum relative error introduced when rounding a real number to its nearest floating point representation. For any real number x within the representable range:

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u$$

where $\text{fl}(x)$ denotes the floating point representation of x.

Note The numerical difference between machine epsilon and unit roundoff is subtle but important. Machine epsilon is the smallest number that, when added to 1, yields a different floating point number. Unit roundoff is the maximum relative

error in representing any number. In IEEE arithmetic with round-to-nearest mode, $u = \varepsilon_{\text{mach}}/2$.

Thm 2.7 *Theorem 2.7* **(Fundamental Theorem of Floating Point Arithmetic).** Let $\text{fl}(x)$ denote the nearest floating point number to a real x (within the normalized range). Then

$$\text{fl}(x) = x(1+\delta), \quad |\delta| \leq u = 2^{-p},$$

where p is the number of significand bits. Furthermore, for any floating point a, b and operation $\circ \in \{+, -, \times, \div\}$, IEEE 754 guarantees:

$$\text{fl}(a \circ b) = (a \circ b)(1+\delta), \quad |\delta| \leq u.$$

Proof *Proof.* The real line is partitioned into intervals of the form $[x, x(1+2^{1-p}))$, each containing exactly one floating point number (the left endpoint). Rounding to nearest halves this error. For the second statement, IEEE 754 mandates that each basic operation is computed as if in infinite precision and then rounded, ensuring the same $|\delta| \leq u$ bound. □

Note The IEEE 754 guarantee that each individual operation incurs at most unit roundoff is the foundation of all backward error analyses. After n sequential operations, errors can compound, but under mild assumptions (no catastrophic cancellation) the accumulated error grows as $O(nu)$, which is acceptable for double precision when $n \ll 10^{16}$.

Def **Absolute and Relative Error.** When a real number x is approximated by a computed value $\hat{x}$, the **absolute error** is defined as

$$\delta = \hat{x} - x$$

The **relative error** is given by

$$\varepsilon = \frac{\hat{x} - x}{x} = \frac{\delta}{x}$$

so that the computed value can be expressed as

$$\hat{x} = x + \delta \quad \text{or} \quad \hat{x} = x(1+\varepsilon)$$

Example **Absolute vs. Relative Error**: Consider approximating $\pi \approx 3.14159265359...$ with different values:

Approximation	Absolute Error	Relative Error
3.14	0.00159...	0.000507...
3.1	0.04159...	0.01324...
3	0.14159...	0.04507...

Now consider approximating 10^{-6} by 0.99×10^{-6}:

- Absolute error: $\delta = 0.99 \times 10^{-6} - 10^{-6} = -0.01 \times 10^{-6} = -10^{-8}$
- Relative error: $\varepsilon = \frac{-10^{-8}}{10^{-6}} = -0.01$ or -1%

This example illustrates why relative error is often more meaningful than absolute error: a small absolute error might represent a large relative error for small numbers, and vice versa for large numbers.

Def **Floating Point Operations and Error Propagation.** In IEEE floating point arithmetic, operations like addition, subtraction, multiplication, and division are performed as if the exact result were calculated and then rounded to the nearest floating point number. For any binary operation $\circledast$ between floating point numbers x and y:

$$\mathrm{fl}(x \circledast y) = (x \circledast y)(1 + \delta), \quad |\delta| \leq u$$

where u is the unit roundoff. This model helps us analyze how errors propagate through computations.

Example **Error Propagation in Addition**: Consider adding two floating point numbers a and b. The computed sum has the form:

$$\mathrm{fl}(a + b) = (a + b)(1 + \delta), \quad |\delta| \leq u$$

If $a \approx -b$ (nearly equal in magnitude but opposite in sign), cancellation occurs. For example, with 3-digit decimal arithmetic:

$$\begin{aligned} a &= 1.00 \\ b &= -0.995 \end{aligned}$$

The exact sum is 0.005, but in 3-digit arithmetic:

$$\begin{aligned} \mathrm{fl}(a + b) &= \mathrm{fl}(1.00 + (-0.995)) \\ &= \mathrm{fl}(0.005) \\ &= 0.00500 \end{aligned}$$

If a and b both have small relative errors ($\hat{a} = a(1 + \varepsilon_a)$ and $\hat{b} = b(1 + \varepsilon_b)$ where $|\varepsilon_a|, |\varepsilon_b| \leq u$), the relative error in the computed sum can be much larger:

$$\frac{\mathrm{fl}(\hat{a} + \hat{b}) - (a + b)}{a + b} \approx \frac{a\varepsilon_a + b\varepsilon_b}{a + b}$$

When $a \approx -b$, the denominator becomes very small, magnifying the error.

Def

Catastrophic Cancellation. Catastrophic cancellation occurs when subtracting two nearly equal numbers. Although the absolute error in each number may be small, their difference can lead to a significant loss of significant digits. If $\hat{x} = x(1+\varepsilon_x)$ and $\hat{y} = y(1+\varepsilon_y)$ where $x \approx y$, then:

$$\hat{x} - \hat{y} = (x - y) + x\varepsilon_x - y\varepsilon_y$$

The relative error becomes:

$$\frac{(\hat{x} - \hat{y}) - (x - y)}{x - y} = \frac{x\varepsilon_x - y\varepsilon_y}{x - y}$$

When $x \approx y$, the denominator is small, potentially making the relative error very large.

Thm 2.8 *Theorem 2.8* **(Cancellation Error Bound).** Let $\hat{x} = x(1+\varepsilon_x)$ and $\hat{y} = y(1+\varepsilon_y)$ with $|\varepsilon_x|, |\varepsilon_y| \leq u$ and $x > y > 0$. Then the relative error in the computed difference satisfies

$$\left|\frac{(\hat{x} - \hat{y}) - (x - y)}{x - y}\right| \leq u \cdot \frac{x + y}{x - y}.$$

The amplification factor $(x + y)/(x - y)$ is unbounded as $x \to y^+$, explaining why catastrophic cancellation can cause total loss of significance.

Proof *Proof.* The error in the difference is $\hat{x} - \hat{y} - (x - y) = x\varepsilon_x - y\varepsilon_y$. Taking absolute value and dividing by $|x - y|$:

$$\frac{|x\varepsilon_x - y\varepsilon_y|}{x - y} \leq \frac{|x||\varepsilon_x| + |y||\varepsilon_y|}{x - y} \leq u \cdot \frac{x + y}{x - y}.$$

□

Example **Catastrophic Cancellation Example**:
Consider computing $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^8$ using double precision. Direct computation:

$$\begin{aligned}
\sqrt{10^8 + 1} &\approx 10000.00000005 \\
\sqrt{10^8} &= 10000 \\
f(10^8) &\approx 10000.00000005 - 10000 = 5 \times 10^{-8}
\end{aligned}$$

Due to rounding in floating point, we might only get:

$$f(10^8) \approx 0$$

However, algebraic manipulation gives an equivalent form:

$$f(x) = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

Using this formula:

$$f(10^8) = \frac{1}{10000.00000005 + 10000} \approx \frac{1}{20000} = 5 \times 10^{-5}$$

The second approach avoids catastrophic cancellation and gives a more accurate result. This illustrates that mathematical reformulation can often mitigate numerical issues.

Note IEEE 754 defines special bit patterns to handle results that fall outside the range of normalized numbers.

Def **Signed Zero.** Both $+0$ and -0 are represented and compare as equal, but behave differently in certain operations (e.g., $1/(+0) = +\infty$ vs. $1/(-0) = -\infty$).

Def **Subnormal Numbers.** Numbers smaller than the smallest normalized value $2^{e_{\min}}$ are represented with a leading zero bit instead of one, allowing gradual underflow at the cost of reduced precision. The smallest positive double-precision subnormal is $2^{-1074} \approx 4.94 \times 10^{-324}$.

Def **Infinities.** $+\infty$ and $-\infty$ arise from overflow or division by a nonzero number by zero (e.g., `1.0/0.0`). They propagate correctly through arithmetic: $x + \infty = \infty$ for any finite x.

Def **NaN (Not a Number).** Produced by undefined operations such as $0/0$, $\infty - \infty$, or $\sqrt{-1}$. Any arithmetic involving a NaN returns a NaN, and NaN compares unequal to everything including itself (`nan != nan` is `True` in Python).

Ex 2.7

1. In Python, evaluate `float('inf') + 1`, `float('inf') - float('inf')`, and `0.0 * float('inf')`. Classify each result as a normal float, $\pm\infty$, or NaN. Explain using the IEEE 754 rules above.
2. Explain why `float('nan') == float('nan')` returns `False` in Python. What does this imply for checking if a computation produced a NaN?
3. What is the smallest positive subnormal number in IEEE single precision (1 sign, 8 exponent, 23 fraction bits)? How does its relative rounding error compare to that of a normalized number?

Example **Subnormal Numbers Range and Precision**: For IEEE double precision:

- Smallest positive normalized number: $2^{-1022} \approx 2.22 \times 10^{-308}$
- Smallest positive subnormal number: $2^{-1022} \times 2^{-52} = 2^{-1074} \approx 4.94 \times 10^{-324}$

Subnormal numbers fill the gap between 0 and the smallest normalized number, but with reduced precision. While a normalized double precision number has 53 bits of precision, a subnormal number might have significantly fewer, depending on its magnitude.

Example For example, the subnormal number 2^{-1073} has only 1 bit of precision, making its relative representation error much larger than for normalized numbers.

Note Floating-point arithmetic is not real arithmetic. The following properties distinguish it from the exact operations on $\mathbb{R}$ and must be internalized when designing numerical algorithms.

Def **Non-Associativity.** In floating-point arithmetic, $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$ in general. Each addition introduces a rounding error, and changing the order changes which intermediate values are rounded.

Def **Non-Distributivity.** $a \otimes (b \oplus c) \neq (a \otimes b) \oplus (a \otimes c)$ in general, because $b \oplus c$ is rounded before multiplication, while $a \otimes b$ and $a \otimes c$ are rounded separately before addition.

Def **Absorption.** When $|a| \gg |b|$, the sum $\mathrm{fl}(a + b)$ may equal a exactly, because b falls below the unit roundoff relative to a. The smaller number is *absorbed* and contributes nothing.

Def **Inexact Cancellation.** Subtracting two nearly equal floating-point numbers amplifies relative errors catastrophically. This is the phenomenon of catastrophic cancellation discussed above, and is a consequence of the non-associativity and rounding model.

Ex 2.8

1. Using 4-significant-digit decimal arithmetic, compute $(1000.+0.001)-1000.$ and $1000.+(0.001-1000.)$. Which order is accurate? Which demonstrates absorption?
2. In Python, verify non-distributivity: compute `a*(b+c)` vs `a*b + a*c` for `a = 1e15`, `b = 1.0`, `c = -1.0`. Explain the discrepancy.
3. True or False: if `a == b` in floating point, then `a - b == 0.0` always. What about the converse?

Example **Non-associativity Example**: Consider adding three numbers in different orders using 4-digit decimal arithmetic:

$$\begin{aligned}(1.000 \times 10^{10} + 1.000) + (-1.000 \times 10^{10}) &= 1.000 \times 10^{10} + (-1.000 \times 10^{10}) \\ &= 0.000\end{aligned}$$

But in the other order:

$$\begin{aligned}1.000 \times 10^{10} + (1.000 + (-1.000 \times 10^{10})) &= 1.000 \times 10^{10} + (-9.999 \times 10^{9}) \\ &= 1.000 \times 10^{9}\end{aligned}$$

The large difference in results demonstrates why the order of operations matters in floating point arithmetic, especially when combining values of significantly different magnitudes.

Note Error analysis is the systematic study of how errors enter and propagate through a numerical computation. Four complementary strategies are used in practice.

Def **Forward Error Analysis.** Tracks how errors in the input data and rounding errors at each arithmetic operation propagate *forward* through the algorithm to bound the total error in the output. Straightforward but can produce pessimistic overestimates.

Def **Backward Error Analysis.** Rather than tracking errors forward, asks: *what is the smallest perturbation to the input that would make the computed output exact?* If this backward error is small (say, of order $u\|\mathbf{A}\|$), the algorithm solved a nearby problem exactly — which is the best one can hope for in floating-point arithmetic.

Def **Interval Arithmetic.** Each number is replaced by a guaranteed enclosing interval $[a, b]$, and every operation maps intervals to intervals in a way that always contains the true result. Produces certified bounds but can suffer from interval explosion for long computations.

Def **Condition Number Analysis.** Measures the intrinsic sensitivity of a *problem* to small perturbations in its input, independently of the algorithm. A problem with condition number κ can lose up to $\log_{10} \kappa$ digits of accuracy even with a perfect algorithm. An algorithm is *backward stable* if its backward error is $O(u)$.

Ex 2.9

1. For solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, explain the difference between the forward error $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|/\|\boldsymbol{x}\|$ and the backward error $\|\boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}\|/(\|\mathbf{A}\|\|\hat{\boldsymbol{x}}\|)$. Which is easier to compute after the fact, and why?

2. An algorithm has backward error 10^{-15} applied to a problem with condition number $\kappa(\mathbf{A}) = 10^8$. What is the worst-case forward error? Is this acceptable for double-precision computation?

3. Why is backward error analysis generally preferred over forward error analysis for evaluating numerical algorithms? Give one scenario where forward analysis might still be useful.

Example Let $\hat{\boldsymbol{x}}$ denote the computed solution to the linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Backward error analysis seeks to determine the perturbation $\mathbf{E}$ to $\mathbf{A}$ such that $\hat{\boldsymbol{x}}$ is the exact solution to the perturbed system:

$$(\mathbf{A} + \mathbf{E})\hat{\boldsymbol{x}} = \boldsymbol{b}.$$

To proceed, define the residual:

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}.$$

By rearranging we get:

$$\mathbf{A}\hat{\boldsymbol{x}} + \boldsymbol{r} = \boldsymbol{b},$$

which can be rewritten as:

$$(\mathbf{A} + \mathbf{E})\hat{\boldsymbol{x}} = \boldsymbol{b},$$

where the perturbation matrix $\mathbf{E}$ is given explicitly by:

$$\mathbf{E} = -\frac{\boldsymbol{r}\hat{\boldsymbol{x}}^\top}{\|\hat{\boldsymbol{x}}\|^2}.$$

The *relative backward error* is defined as:

$$\frac{\|\mathbf{E}\|}{\|\mathbf{A}\|}.$$

In practice, this quantity is often much smaller than the *relative forward error*:

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|}.$$

This explains why algorithms that exhibit large forward errors may still be practically useful: they produce solutions that are exact for a nearby problem.

Floating point arithmetic forms the foundation of numerical computing, with its inherent limitations shaping the design and analysis of algorithms. By understanding floating point representation, error propagation, and special cases, we can develop numerically stable algorithms that produce reliable results even in the presence of unavoidable computational errors. Remember that mathematical reformulations, careful algorithm design, and appropriate error analysis are often more effective than simply increasing precision.

Ex 2.10

1. How many significant decimal digits does an IEEE double-precision number carry? (*Hint:* $u = 2^{-52} \approx 2.22 \times 10^{-16}$; $-\log_{10}(u) \approx ?$)

2. In Python, evaluate `1.0 + 1e-16` and `1.0 + 2e-16`. Explain both results using $\varepsilon_{\text{mach}}$.

3. Write a Python function `compute_eps()` that finds machine epsilon empirically (without `sys.float_info.epsilon` or `np.finfo`). *Hint: start from 1.0 and halve repeatedly until* $1 + \varepsilon = 1$.

4. Confirm that $(1.0 + u) - 1.0 = u$ but $(1.0 + u/2) - 1.0 = 0$ in double precision. Explain in terms of the spacing between adjacent floats near 1.

Ex 2.11 **(Cancellation and Reformulation)** For each expression, identify whether catastrophic cancellation can occur and, if so, derive an algebraically equivalent form that avoids it.

1. $\dfrac{1 - \cos x}{x^2}$ for small x (use the identity $1 - \cos x = 2\sin^2(x/2)$).

2. $\sqrt{x+1} - \sqrt{x}$ for large x (rationalize the numerator).

3. The smaller root of $x^2 - 10^8 x + 1 = 0$ via the quadratic formula. What goes wrong and how do you fix it?

4. $e^x - 1$ for small x. Why does NumPy provide `np.expm1(x)`? Verify numerically that `np.expm1(1e-15)` is more accurate than `np.exp(1e-15) - 1`.

Ex 2.12 **(Backward Error Analysis)** Let $\hat{\boldsymbol{x}}$ be the computed solution to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ obtained by LU with partial pivoting.

1. Define the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$. Using the formula $\mathbf{E} = -\boldsymbol{r}\hat{\boldsymbol{x}}^T/\|\hat{\boldsymbol{x}}\|^2$, show that $(\mathbf{A} + \mathbf{E})\hat{\boldsymbol{x}} = \boldsymbol{b}$, confirming $\hat{\boldsymbol{x}}$ is the exact solution to a nearby system.

2. Show that the relative backward error satisfies $\|\mathbf{E}\|_2/\|\mathbf{A}\|_2 = \|\boldsymbol{r}\|_2/(\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2)$. (*Hint: use* $\|\boldsymbol{r}\boldsymbol{v}^T\|_2 = \|\boldsymbol{r}\|_2\|\boldsymbol{v}\|_2$ *for rank-1 matrices.*)

3. For a well-conditioned system, a small backward error implies a small forward error. Write the bound:

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} \leq \kappa(\mathbf{A}) \cdot \frac{\|\mathbf{E}\|_2}{\|\mathbf{A}\|_2},$$

   where $\kappa(\mathbf{A}) = \|\mathbf{A}\|_2\|\mathbf{A}^{-1}\|_2$ is the condition number. If $\kappa(\mathbf{A}) = 10^{10}$ and the backward error is 10^{-16}, what is the worst-case forward error?

4. In Python, construct an ill-conditioned matrix using `np.diag([1, 1e-10])`, solve with `np.linalg.solve`, and compare `np.linalg.cond(A)` with the observed forward error.

3 Eigenvalues and Matrix Properties

3.1 Eigenvalues and Eigenvectors

Def **Eigenvalues and Eigenvectors.** For a matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, a scalar $\lambda \in \mathbb{C}$ is called an *eigenvalue* of $\mathbf{A}$ if there exists a nonzero vector $\boldsymbol{v} \in \mathbb{C}^n$ such that

$$\mathbf{A}\boldsymbol{v} = \lambda \boldsymbol{v}.$$

The nonzero vector $\boldsymbol{v}$ is called an *eigenvector* of $\mathbf{A}$ corresponding to the eigenvalue λ. The set of all eigenvalues of $\mathbf{A}$ is called the *spectrum* of $\mathbf{A}$, denoted by $\sigma(\mathbf{A})$.

Intuitively, an eigenvector is a vector whose direction remains unchanged when transformed by the matrix $\mathbf{A}$—it may be stretched, compressed, or reversed, but it stays on the same line. The eigenvalue tells us the factor by which the eigenvector is scaled. Eigenvalues and eigenvectors reveal the fundamental character of a linear transformation, showing the directions in which the transformation behaves most simply.

Def **Characteristic Polynomial.** For a matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the *characteristic polynomial* is defined as

$$p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I}),$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. The equation

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0$$

is called the *characteristic equation* of $\mathbf{A}$. The roots of the characteristic polynomial are precisely the eigenvalues of $\mathbf{A}$.

Example Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

To find its eigenvalues, we compute the characteristic polynomial:

$$\begin{aligned}
p_{\mathbf{A}}(\lambda) &= \det(\mathbf{A} - \lambda \mathbf{I}) && (2)\\
&= \det \begin{pmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{pmatrix} && (3)\\
&= (3-\lambda)(3-\lambda) - 1 \cdot 1 && (4)\\
&= (3-\lambda)^2 - 1 && (5)\\
&= 9 - 6\lambda + \lambda^2 - 1 && (6)\\
&= \lambda^2 - 6\lambda + 8 && (7)\\
&= (\lambda - 4)(\lambda - 2) && (8)
\end{aligned}$$

Setting $p_{\mathbf{A}}(\lambda) = 0$, we get the eigenvalues $\lambda_1 = 4$ and $\lambda_2 = 2$.
For $\lambda_1 = 4$, we find the corresponding eigenvector by solving $(\mathbf{A} - 4\mathbf{I})\boldsymbol{v} = \mathbf{0}$:

$$\begin{pmatrix} 3-4 & 1 \\ 1 & 3-4 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{9}$$

$$\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{10}$$

This gives us $-v_1+v_2 = 0$, so $v_1 = v_2$. Any nonzero vector of the form $\boldsymbol{v}_1 = (t, t)^T$ for $t \neq 0$ is an eigenvector. A normalized eigenvector would be $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$.
Similarly, for $\lambda_2 = 2$, we solve $(\mathbf{A} - 2\mathbf{I})\boldsymbol{v} = \mathbf{0}$:

$$\begin{pmatrix} 3-2 & 1 \\ 1 & 3-2 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{11}$$

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{12}$$

This gives us $v_1 + v_2 = 0$, so $v_1 = -v_2$. Any nonzero vector of the form $\boldsymbol{v}_2 = (t, -t)^T$ for $t \neq 0$ is an eigenvector. A normalized eigenvector would be $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}(1, -1)^T$.

Prop 3.1 *Proposition 3.1* **(Eigenvalues of Triangular Matrices).** If $\mathbf{A}$ is upper or lower triangular, its eigenvalues are precisely the diagonal entries $a_{11}, a_{22}, \ldots, a_{nn}$.

Proof *Proof.* The characteristic polynomial $\det(\mathbf{A} - \lambda\mathbf{I})$ for a triangular matrix is the product of its diagonal entries $(a_{ii} - \lambda)$, giving $p_{\mathbf{A}}(\lambda) = \prod_{i=1}^{n}(a_{ii} - \lambda)$. The roots are $\lambda_i = a_{ii}$. □

Prop 3.2 *Proposition 3.2* **(Similar Matrices Share Eigenvalues).** If $\mathbf{B} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}$ for some invertible $\mathbf{V}$, then $\mathbf{A}$ and $\mathbf{B}$ have the same characteristic polynomial and hence the same eigenvalues.

Proof *Proof.* $\det(\mathbf{B}-\lambda\mathbf{I}) = \det(\mathbf{V}^{-1}\mathbf{A}\mathbf{V}-\lambda\mathbf{I}) = \det(\mathbf{V}^{-1}(\mathbf{A}-\lambda\mathbf{I})\mathbf{V}) = \det(\mathbf{V}^{-1})\det(\mathbf{A}-\lambda\mathbf{I})\det(\mathbf{V}) = \det(\mathbf{A} - \lambda\mathbf{I})$. □

Thm 3.3 *Theorem 3.3* **(Key Eigenvalue Properties).** For $\mathbf{A} \in \mathbb{R}^{n\times n}$ with eigenvalues $\lambda_1, \ldots, \lambda_n$ (counting algebraic multiplicity):

1. $\operatorname{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i$
2. $\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i$
3. $\mathbf{A}$ is singular if and only if $0 \in \sigma(\mathbf{A})$.

4. If $\mathbf{A}$ is invertible, the eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_1, \ldots, 1/\lambda_n$.

Proof *Proof.* (1) and (2): Both follow from comparing the characteristic polynomial

$$p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}) = (-1)^n(\lambda - \lambda_1)\cdots(\lambda - \lambda_n)$$

with the direct expansion of the determinant. Setting $\lambda = 0$ gives $\det(\mathbf{A}) = \prod_i \lambda_i$. The coefficient of λ^{n-1} gives $-\mathrm{tr}(\mathbf{A}) = -\sum_i \lambda_i$.
(3): Direct from (2): $\det(\mathbf{A}) = 0 \iff \prod_i \lambda_i = 0 \iff$ some $\lambda_i = 0$.
(4): If $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ with $\lambda \neq 0$, multiply both sides by $\frac{1}{\lambda}\mathbf{A}^{-1}$ to get $\mathbf{A}^{-1}\boldsymbol{v} = \frac{1}{\lambda}\boldsymbol{v}$. □

Lem 3.4 *Lemma 3.4* **(Eigenvectors for Distinct Eigenvalues are Linearly Independent).** Let $\lambda_1, \ldots, \lambda_k$ be distinct eigenvalues of $\mathbf{A}$ with corresponding eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$. Then $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ is linearly independent.

Proof *Proof.* By induction on k. The case $k = 1$ holds since eigenvectors are nonzero. Assume $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}\}$ is independent. Suppose $\sum_{i=1}^{k} c_i\boldsymbol{v}_i = \mathbf{0}$. Apply $(\mathbf{A} - \lambda_k\mathbf{I})$ to both sides:

$$\sum_{i=1}^{k-1} c_i(\lambda_i - \lambda_k)\boldsymbol{v}_i = \mathbf{0}.$$

By the inductive hypothesis, $c_i(\lambda_i - \lambda_k) = 0$ for each $i < k$. Since $\lambda_i \neq \lambda_k$, each $c_i = 0$. Substituting back gives $c_k\boldsymbol{v}_k = \mathbf{0}$, so $c_k = 0$. □

Thm 3.5 *Theorem 3.5* **(Spectral Theorem for Real Symmetric Matrices).** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric. Then:

1. All eigenvalues of $\mathbf{A}$ are real.
2. Eigenvectors corresponding to distinct eigenvalues are orthogonal.
3. $\mathbf{A}$ is orthogonally diagonalizable: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ where $\mathbf{Q}$ is orthogonal and $\mathbf{D} = \mathrm{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{R}^{n\times n}$.

Proof *Proof.* (1): Let $\lambda \in \mathbb{C}$ be an eigenvalue with eigenvector $\boldsymbol{v} \in \mathbb{C}^n$, $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$. Then $\bar{\boldsymbol{v}}^T\mathbf{A}\boldsymbol{v} = \lambda\,\bar{\boldsymbol{v}}^T\boldsymbol{v}$. But also $\bar{\boldsymbol{v}}^T\mathbf{A}\boldsymbol{v} = (\mathbf{A}\bar{\boldsymbol{v}})^T\boldsymbol{v} = \overline{(\mathbf{A}\boldsymbol{v})}^T\boldsymbol{v} = \bar{\lambda}\,\bar{\boldsymbol{v}}^T\boldsymbol{v}$. Since $\bar{\boldsymbol{v}}^T\boldsymbol{v} = \|\boldsymbol{v}\|_2^2 > 0$, we conclude $\lambda = \bar{\lambda}$, so $\lambda \in \mathbb{R}$.
(2): Let $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ and $\mathbf{A}\boldsymbol{w} = \mu\boldsymbol{w}$ with $\lambda \neq \mu$. Then:

$$\lambda\,\boldsymbol{v}^T\boldsymbol{w} = (\mathbf{A}\boldsymbol{v})^T\boldsymbol{w} = \boldsymbol{v}^T\mathbf{A}^T\boldsymbol{w} = \boldsymbol{v}^T\mathbf{A}\boldsymbol{w} = \mu\,\boldsymbol{v}^T\boldsymbol{w}.$$

Since $\lambda \neq \mu$, we get $\boldsymbol{v}^T\boldsymbol{w} = 0$.
(3): Existence of a full orthonormal eigenbasis follows from (1), (2), and the fact that the characteristic polynomial splits over $\mathbb{R}$ for symmetric matrices (proven via induction on dimension in a full course). □

Def

Diagonalization. A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is *diagonalizable* if there exists an invertible matrix $\mathbf{V}$ and a diagonal matrix $\mathbf{\Lambda}$ such that

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

where the diagonal entries of $\mathbf{\Lambda}$ are the eigenvalues of $\mathbf{A}$ and the columns of $\mathbf{V}$ are the corresponding eigenvectors.

Note A matrix $\mathbf{A}$ is diagonalizable if and only if there exist n linearly independent eigenvectors, which occurs if and only if the geometric multiplicity of each eigenvalue equals its algebraic multiplicity.

The eigenvalue problem is one of the most fundamental in numerical linear algebra, with applications ranging from structural engineering (vibration analysis) to quantum mechanics (energy states), machine learning (principal component analysis), and dynamical systems (stability analysis). The efficiency and accuracy of eigenvalue algorithms have direct implications for these applications, making them a central focus in scientific computing.

Ex 3.1 For each matrix, find all eigenvalues and a corresponding eigenvector for each, showing all steps.

1. $\mathbf{A} = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$ (verify using trace and determinant)
2. $\mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (rotation by 90°; are eigenvalues real?)
3. $\mathbf{C} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}$ (use property (5) of Key Properties)

Ex 3.2 **(Spectral Theorem in Practice)** Let $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$.

1. Find the eigenvalues and verify: $\mathrm{tr}(\mathbf{A}) = \lambda_1 + \lambda_2$ and $\det(\mathbf{A}) = \lambda_1\lambda_2$.
2. Find an orthonormal eigenbasis $\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$ and write $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ explicitly.
3. Use the decomposition to compute $\mathbf{A}^{10}$ efficiently. (*Hint:* $\mathbf{A}^k = \mathbf{Q}\mathbf{D}^k\mathbf{Q}^T$, *and* $\mathbf{D}^k$ *is easy.*)
4. In NumPy, use `np.linalg.eig(A)` and `np.linalg.eigh(A)` (the latter exploits symmetry). Compare the results and explain why `eigh` is preferred for symmetric matrices.

Ex 3.3 **(Diagonalizability and the Geometric Multiplicity)** The matrix $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$ has a repeated eigenvalue $\lambda = 2$.

1. Find the eigenspace $\ker(\mathbf{A} - 2\mathbf{I})$. What is its dimension (geometric multiplicity)?

2. Is $\mathbf{A}$ diagonalizable? Why or why not?

3. Compute $\mathbf{A}^k$ for general k using the Jordan form $\mathbf{A} = 2\mathbf{I} + \mathbf{N}$ where $\mathbf{N} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ is nilpotent. (*Hint:* $\mathbf{N}^2 = \mathbf{0}$, *so* $(2\mathbf{I}+\mathbf{N})^k = \sum_{j=0}^{1} \binom{k}{j} 2^{k-j}\mathbf{N}^j$.)

4. What does `np.linalg.eig(A)` return for this matrix? Are the eigenvectors linearly independent?

3.2 Symmetric Positive Definite (SPD) Matrices

Def **Symmetric Matrix.** A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is called *symmetric* if

$$\mathbf{A} = \mathbf{A}^T,$$

meaning that the entry in the ith row and jth column is the same as the entry in the jth row and ith column for all $1 \leq i, j \leq n$.

Def **Positive Definite Matrix.** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is *positive definite* (PD) if

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0 \quad \text{for every nonzero } \boldsymbol{x} \in \mathbb{R}^n.$$

In computational terms, when we transform any nonzero vector $\boldsymbol{x}$ using the quadratic form $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$, the result is always a positive number.

Thm 3.6 *Theorem 3.6* **(SPD Matrices Induce a Metric).** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be an SPD matrix. The function

$$d_{\mathbf{A}}(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(\boldsymbol{x} - \boldsymbol{y})^T\mathbf{A}(\boldsymbol{x} - \boldsymbol{y})}$$

defines a metric on $\mathbb{R}^n$, satisfying non-negativity, identity of indiscernibles, symmetry, and the triangle inequality.

Proof *Proof.* Non-negativity and identity of indiscernibles follow directly from the positive-definite property: $(\boldsymbol{x} - \boldsymbol{y})^T\mathbf{A}(\boldsymbol{x} - \boldsymbol{y}) \geq 0$ with equality iff $\boldsymbol{x} - \boldsymbol{y} = \mathbf{0}$. Symmetry holds since $(\boldsymbol{x} - \boldsymbol{y})^T\mathbf{A}(\boldsymbol{x} - \boldsymbol{y}) = (\boldsymbol{y} - \boldsymbol{x})^T\mathbf{A}(\boldsymbol{y} - \boldsymbol{x})$. The triangle inequality follows from the Cauchy-Schwarz inequality applied to the inner product $\langle \boldsymbol{u}, \boldsymbol{v}\rangle_{\mathbf{A}} = \boldsymbol{u}^T\mathbf{A}\boldsymbol{v}$. □

Note When $\mathbf{A} = \mathbf{I}$, $d_{\mathbf{A}}$ reduces to the standard Euclidean distance. The general $\mathbf{A}$-metric stretches space according to the eigenvectors and eigenvalues of $\mathbf{A}$: directions with large eigenvalues are "expensive" to move in.

Ex 3.4

1. For $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, compute $d_{\mathbf{A}}\big((1,0)^T, (0,0)^T\big)$ and $d_{\mathbf{A}}\big((0,1)^T, (0,0)^T\big)$. Compare to the Euclidean distances. In what direction is the $\mathbf{A}$-metric "larger"?

2. Explain geometrically: the set $\{\boldsymbol{x} : d_{\mathbf{A}}(\boldsymbol{x}, \mathbf{0}) = 1\}$ is an ellipse. What are its axes and their lengths in terms of the eigenvalues of $\mathbf{A}$?

3. Where does this metric appear in machine learning? (*Hint: Mahalanobis distance.*)

Def **SPD Interpratation.** An SPD matrix transforms vector spaces by rescaling their components in a way that preserves a consistent notion of size. In standard Euclidean space, the norm $\|\boldsymbol{x}\|$ returns a positive length for any nonzero vector $\boldsymbol{x}$. Similarly, an SPD matrix $\mathbf{A}$ creates a new norm

$$\|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T \mathbf{A} \boldsymbol{x}},$$

which remains strictly positive for all nonzero $\boldsymbol{x}$. This property ensures that the transformation never collapses vectors to zero length or produces negative squared lengths, which is crucial for numerical stability in computational algorithms.

Note Another way to visualize SPD matrices is through their geometric interpretation. When you use an SPD matrix $\mathbf{A}$ to compute distances via $\boldsymbol{x}^T \mathbf{A} \boldsymbol{x}$, the set of all vectors with constant "size" forms an ellipsoid (a stretched or squashed sphere). This is important in scientific computing applications like principal component analysis (PCA), where we often need to understand how data varies in different directions. The axes of this ellipsoid correspond to the eigenvectors of $\mathbf{A}$, and their lengths are determined by the eigenvalues.

Def **A-Inner Product.** For an SPD matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the function

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathbf{A}} = \boldsymbol{x}^T \mathbf{A} \boldsymbol{y}$$

defines a valid inner product on $\mathbb{R}^n$, inducing the norm $\|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T \mathbf{A} \boldsymbol{x}}$. This is the natural inner product for problems where $\mathbf{A}$ encodes the geometry (e.g., solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ via conjugate gradients).

Thm 3.7 *Theorem 3.7* **(SPD Characterization via Eigenvalues).** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is positive definite if and only if all its eigenvalues are strictly positive.

Proof *Proof.* By the Spectral Theorem, $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ where $\mathbf{Q}$ is orthogonal and $\mathbf{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. For any nonzero $\boldsymbol{x}$, let $\boldsymbol{y} = \mathbf{Q}^T \boldsymbol{x}$. Since $\mathbf{Q}$ is invertible, $\boldsymbol{y} \neq \mathbf{0}$. Then:

$$\boldsymbol{x}^T \mathbf{A} \boldsymbol{x} = \boldsymbol{y}^T \mathbf{D} \boldsymbol{y} = \sum_{i=1}^{n} \lambda_i y_i^2.$$

($\Rightarrow$): If all $\lambda_i > 0$ and $\boldsymbol{y} \neq \mathbf{0}$, then at least one $y_i \neq 0$, so the sum is > 0.
($\Leftarrow$): If some $\lambda_k \leq 0$, choose $\boldsymbol{x} = \mathbf{Q}\mathbf{e}_k$ (the kth column of $\mathbf{Q}$), giving $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \lambda_k \leq 0$. $\square$

Cor 3.8 *Corollary 3.8* **(SPD Matrices are Invertible).** Every SPD matrix is invertible, since all eigenvalues are positive and thus $\det(\mathbf{A}) = \prod_i \lambda_i > 0$.

From a computational perspective, the condition $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for all nonzero $\boldsymbol{x}$ ensures that the matrix $\mathbf{A}$ never maps a vector to a zero result through the quadratic form, nor does it produce negative values. This is critical in numerical algorithms because it guarantees the stability and uniqueness of solutions. For example, when solving linear systems like $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ where $\mathbf{A}$ is SPD, efficient methods like Cholesky decomposition can be used, and the solution is guaranteed to exist and be unique.

Note SPD matrices are fundamental in scientific computing applications. For example:

- In machine learning, covariance matrices are SPD and used to understand data distributions
- In numerical optimization, the Hessian matrix of a convex function is SPD near the minimum
- In finite element methods, the stiffness matrix is typically SPD
- In numerical linear algebra, SPD matrices allow for specialized algorithms like Cholesky factorization, which is twice as efficient as LU decomposition
- In image processing, SPD matrices can represent local structure tensors that capture directional information

Their computational properties make algorithms more stable, efficient, and reliable.

3.2.1 Cholesky Decomposition

Def **Cholesky Decomposition.** For a symmetric positive definite (SPD) matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the Cholesky Decomposition expresses $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T,$$

where $\mathbf{L}$ is a unique lower triangular matrix with strictly positive diagonal entries.

SPD matrices, which satisfy $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for all nonzero $\boldsymbol{x}$, are prevalent in optimization, statistics, and finite element methods. The Cholesky factorization is both computationally efficient and numerically stable since it avoids the need for pivoting.

Def **Algorithm for Cholesky Decomposition.** For an SPD matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ can be computed entry by entry as follows: For each entry l_{ij} of $\mathbf{L}$ where $1 \le i, j \le n$:

$$l_{ij} = \begin{cases} 0, & \text{if } i < j \\ \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}, & \text{if } i = j \\ \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), & \text{if } i > j \end{cases} \tag{13}$$

This algorithm proceeds row by row, using previously computed elements of $\mathbf{L}$ to determine subsequent entries.

Derivation of the Cholesky Algorithm (Matrix Form)
To derive the Cholesky algorithm, we start with the relationship $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ where $\mathbf{A}$ is an SPD matrix and $\mathbf{L}$ is lower triangular. Let's express both matrices explicitly:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad \mathbf{L} = \begin{pmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{pmatrix}$$

The transpose of $\mathbf{L}$ is:

$$\mathbf{L}^T = \begin{pmatrix} l_{11} & l_{21} & \cdots & l_{n1} \\ 0 & l_{22} & \cdots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & l_{nn} \end{pmatrix}$$

Now, the matrix product $\mathbf{L}\mathbf{L}^T$ gives us:

$$\mathbf{L}\mathbf{L}^T = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} & \cdots & l_{11}l_{n1} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & \cdots & l_{21}l_{n1} + l_{22}l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1}l_{11} & l_{n1}l_{21} + l_{n2}l_{22} & \cdots & \sum_{k=1}^{n} l_{nk}^2 \end{pmatrix}$$

Equating this with $\mathbf{A}$, we can derive the formulas for different elements of $\mathbf{L}$:
Case 1: Diagonal elements (position (i, i))
From the diagonal entries of $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, we have:

$$a_{ii} = \sum_{k=1}^{i} l_{ik}^2 = l_{i1}^2 + l_{i2}^2 + \cdots + l_{ii}^2$$

Solving for l_{ii}:

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}$$

Case 2: Below diagonal (position (i, j) where $i > j$)
From the below-diagonal entries of $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, we have:

$$a_{ij} = \sum_{k=1}^{j} l_{ik} l_{jk} = l_{i1} l_{j1} + l_{i2} l_{j2} + \cdots + l_{ij} l_{jj}$$

Since $l_{jk} = 0$ for $k > j$ (as $\mathbf{L}$ is lower triangular), the sum stops at $k = j$. Solving for l_{ij}:

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$$

Case 3: Above diagonal (position (i, j) where $i < j$)
Since $\mathbf{A}$ is symmetric, $a_{ij} = a_{ji}$. The elements of $\mathbf{L}$ above the diagonal are defined to be zero: $l_{ij} = 0$ for $i < j$.
This gives us the complete algorithm for computing the Cholesky decomposition:

$$l_{ij} = \begin{cases} 0, & \text{if } i < j \\ \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}, & \text{if } i = j \\ \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), & \text{if } i > j \end{cases}$$

The algorithm proceeds by computing $\mathbf{L}$ one column at a time, from left to right, and within each column from top to bottom.

Def **Computational Complexity (Cholesky).** The Cholesky decomposition algorithm requires:

- Approximately $\frac{1}{3}n^3$ floating-point operations (flops) for an $n \times n$ matrix
- This is roughly half the cost of LU factorization ($\frac{2}{3}n^3$ flops), which is the standard method for non-symmetric matrices
- Storage requirement of $\frac{n(n+1)}{2}$ elements (only the lower triangular part is stored)

The efficiency comes from exploiting both the symmetry of $\mathbf{A}$ and its positive definiteness, which eliminates the need for pivoting strategies.

Example Let's compute the Cholesky decomposition of:

$$\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Step 1: Compute the first column of $\mathbf{L}$.

$$l_{11} = \sqrt{a_{11}} = \sqrt{4} = 2 \tag{14}$$

$$l_{21} = \frac{a_{21}}{l_{11}} = \frac{2}{2} = 1 \tag{15}$$

Step 2: Compute the second column of $\mathbf{L}$.

$$l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{3 - 1^2} = \sqrt{2} \approx 1.414 \tag{16}$$

Therefore:

$$\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix}$$

Verification:

$$\mathbf{L}\mathbf{L}^T = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & \sqrt{2} \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix} = \mathbf{A}$$

Note The uniqueness of the Cholesky Decomposition stems from the positivity of the leading principal minors of an SPD matrix, ensuring that each pivot in the elimination process is nonzero. For an SPD matrix $\mathbf{A}$, the leading principal minor of order k, denoted as $\det(\mathbf{A}_k)$, is the determinant of the submatrix formed by the first k rows and columns. These determinants are all positive, which ensures that the diagonal entries of $\mathbf{L}$ are well-defined and unique.

Thm 3.9 *Theorem 3.9* **(Cholesky Existence and Uniqueness).** A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ admits a Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ (with $\mathbf{L}$ lower triangular and strictly positive diagonal) if and only if $\mathbf{A}$ is SPD. The factorization is unique.

Proof *Proof.* ($\Rightarrow$): Suppose $\mathbf{A} = \mathbf{L}\mathbf{L}^T$. Then $\mathbf{A}^T = (\mathbf{L}\mathbf{L}^T)^T = \mathbf{L}\mathbf{L}^T = \mathbf{A}$, so $\mathbf{A}$ is symmetric. For any nonzero $\boldsymbol{x}$:

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \boldsymbol{x}^T\mathbf{L}\mathbf{L}^T\boldsymbol{x} = \|\mathbf{L}^T\boldsymbol{x}\|_2^2 \geq 0.$$

Equality holds iff $\mathbf{L}^T\boldsymbol{x} = \mathbf{0}$. Since the diagonal of $\mathbf{L}$ is strictly positive, $\mathbf{L}$ is invertible, so $\boldsymbol{x} = \mathbf{0}$. Hence $\mathbf{A}$ is SPD.
($\Leftarrow$): If $\mathbf{A}$ is SPD, all leading principal minors are positive (each submatrix is also SPD), so the Cholesky algorithm never encounters a zero or negative under a square root.
Uniqueness: If $\mathbf{L}_1\mathbf{L}_1^T = \mathbf{L}_2\mathbf{L}_2^T$, set $\mathbf{M} = \mathbf{L}_1^{-1}\mathbf{L}_2$. Then $\mathbf{M}\mathbf{M}^T = \mathbf{I}$, so $\mathbf{M}$ is lower triangular and orthogonal. The only lower triangular orthogonal matrix with positive diagonal is $\mathbf{I}$, so $\mathbf{L}_1 = \mathbf{L}_2$. □

Thm 3.10 *Theorem 3.10* **(Cholesky Factorization Cost).** The Cholesky factorization of an SPD matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ costs $\frac{1}{3}n^3 + O(n^2)$ flops — exactly half the cost of LU factorization — and requires storing only $\frac{n(n+1)}{2}$ entries.

Proof *Proof.* At step j of the algorithm, we compute l_{jj} (1 square root) and $n-j$ below-diagonal entries l_{ij} for $i > j$, each requiring j multiplications and 1 division. The dominant cost is:

$$\sum_{j=1}^{n}(n-j)\cdot j \approx \int_0^n (n-t)\,t\,dt = \frac{n^3}{6} + \frac{n^3}{6} = \frac{n^3}{3}.$$

□

Ex 3.5

1. Which of the following matrices are SPD? For each, either prove it or find a nonzero $\boldsymbol{x}$ with $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} \leq 0$.

$$\mathbf{A}_1 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

2. Compute the Cholesky factorization of $\mathbf{A}_1$ and $\mathbf{A}_3$ by hand. Verify $\mathbf{L}\mathbf{L}^T = \mathbf{A}$.

3. Use the Cholesky factorization of $\mathbf{A}_1$ to solve $\mathbf{A}_1\boldsymbol{x} = (3, 4)^T$ via forward then backward substitution.

4. In NumPy, `np.linalg.cholesky(A)` raises `LinAlgError` if $\mathbf{A}$ is not SPD. Write code to attempt Cholesky on each matrix above and handle the exception. What does a `LinAlgError` tell you about the matrix?

Ex 3.6 **(Condition Number of SPD Matrices)** For an SPD matrix with eigenvalues $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$, the 2-norm condition number is $\kappa_2(\mathbf{A}) = \lambda_n/\lambda_1$.

1. Prove this formula. (*Hint:* $\|\mathbf{A}\|_2 = \max_i |\lambda_i|$ *for symmetric* $\mathbf{A}$*; and* $\mathbf{A}^{-1}$ *has eigenvalues* $1/\lambda_i$*.*)

2. For $\mathbf{A} = \begin{pmatrix} 10 & 0 \\ 0 & 0.01 \end{pmatrix}$, what is $\kappa_2(\mathbf{A})$? What does this imply for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$?

3. The Cholesky factor $\mathbf{L}$ satisfies $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$. Why is this numerically significant? (*Think: solving* $\mathbf{L}\boldsymbol{y} = \boldsymbol{c}$ *and* $\mathbf{L}^T\boldsymbol{x} = \boldsymbol{y}$ *each introduces what error?*)

4. In NumPy, compute `np.linalg.cond(A)` for an SPD matrix and verify it matches $\lambda_{\max}/\lambda_{\min}$ computed from `np.linalg.eigvalsh(A)`.

3.3 Orthogonal Matrices

Def **Orthogonal Vectors.** Two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ are called *orthogonal* if their inner product is zero:

$$\boldsymbol{x}^T\boldsymbol{y} = 0$$

Geometrically, orthogonal vectors are perpendicular to each other. A set of vectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ is called *mutually orthogonal* if every pair of distinct vectors in the set is orthogonal:

$$\boldsymbol{v}_i^T\boldsymbol{v}_j = 0 \quad \text{for all } i \neq j$$

Def **Orthonormal Basis.** A set of vectors $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$ in $\mathbb{R}^n$ is called an *orthonormal basis* if:

1. The vectors are *mutually orthogonal*: $\boldsymbol{q}_i^T \boldsymbol{q}_j = 0$ for all $i \neq j$
2. Each vector has unit length: $|\boldsymbol{q}_i|_2 = 1$ for all $i = 1, 2, \ldots, n$
3. The vectors span $\mathbb{R}^n$ (which is automatic if we have n linearly independent vectors in $\mathbb{R}^n$)

Equivalently, a set of vectors $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$ forms an orthonormal basis if:

$$\boldsymbol{q}_i^T \boldsymbol{q}_j = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

If we arrange these vectors as columns of a matrix $\mathbf{Q} = [\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n]$, then $\mathbf{Q}$ is an orthogonal matrix, satisfying $\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}$.

Note Any vector $\boldsymbol{v} \in \mathbb{R}^n$ can be expressed uniquely as a linear combination of vectors from an orthonormal basis $\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n$:

$$\boldsymbol{v} = (\boldsymbol{q}_1^T \boldsymbol{v})\boldsymbol{q}_1 + (\boldsymbol{q}_2^T \boldsymbol{v})\boldsymbol{q}_2 + \ldots + (\boldsymbol{q}_n^T \boldsymbol{v})\boldsymbol{q}_n = \sum_{i=1}^{n} (\boldsymbol{q}_i^T \boldsymbol{v})\boldsymbol{q}_i$$

The coefficients $\boldsymbol{q}_i^T \boldsymbol{v}$ are the projections of $\boldsymbol{v}$ onto the basis vectors. This simple formula for the coefficients is a key computational advantage of orthonormal bases - *we don't need to solve a system of equations to find the coefficients.*

Def **Orthogonal Matrix.** A square matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is called *orthogonal* if its columns form an orthonormal set in $\mathbb{R}^n$, which means: $\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}_n$ Equivalently, $\mathbf{Q}^T = \mathbf{Q}^{-1}$, meaning the transpose of an orthogonal matrix is its inverse.

Orthogonal matrices represent rotations, reflections, or combinations of these operations in $\mathbb{R}^n$. They preserve vector lengths and angles between vectors, making them particularly valuable in scientific computing where maintaining geometric properties is important. When multiplying a vector by an orthogonal matrix, the result is a vector with the same length but potentially different direction.

Thm 3.11 *Theorem 3.11* **(Orthogonal Matrices Preserve Norms and Inner Products).** Let $\mathbf{Q} \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. For all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$:

1. $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\boldsymbol{y}$ (inner-product-preserving)
2. $\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$ (isometry)
3. $\kappa_2(\mathbf{Q}) = 1$ (optimally conditioned)

Proof *Proof.* (1): $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\mathbf{Q}^T\mathbf{Q}\boldsymbol{y} = \boldsymbol{x}^T\mathbf{I}\boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y}$.
(2): Set $\boldsymbol{x} = \boldsymbol{y}$ in (1): $\|\mathbf{Q}\boldsymbol{x}\|_2^2 = \boldsymbol{x}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2$.
(3): By (2), $\|\mathbf{Q}\|_2 = \max_{\|\boldsymbol{x}\|=1} \|\mathbf{Q}\boldsymbol{x}\|_2 = 1$. Since $\mathbf{Q}^{-1} = \mathbf{Q}^T$ is also orthogonal, $\|\mathbf{Q}^{-1}\|_2 = 1$. Thus $\kappa_2(\mathbf{Q}) = \|\mathbf{Q}\|_2\|\mathbf{Q}^{-1}\|_2 = 1$. □

Note The condition number of 1 is the best possible. Multiplying by an orthogonal matrix introduces no amplification of errors, which is why orthogonal transformations (Householder reflections, Givens rotations) are preferred over Gaussian elimination when stability is paramount.

Prop 3.12 *Proposition 3.12* **(Rows of an Orthogonal Matrix are Orthonormal).** If $\mathbf{Q} \in \mathbb{R}^{n\times n}$ satisfies $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$, then $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$ as well, so the rows of $\mathbf{Q}$ also form an orthonormal basis for $\mathbb{R}^n$.

Proof *Proof.* $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$ implies $\mathbf{Q}$ is invertible with $\mathbf{Q}^{-1} = \mathbf{Q}^T$. Left-multiplying by $\mathbf{Q}$ and right-multiplying by $\mathbf{Q}^T$ gives $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$. □

Prop 3.13 *Proposition 3.13* **(Determinant of an Orthogonal Matrix).** For any orthogonal $\mathbf{Q}$, $\det(\mathbf{Q}) = \pm 1$. When $\det(\mathbf{Q}) = +1$, $\mathbf{Q}$ is a rotation; when $\det(\mathbf{Q}) = -1$, it includes a reflection.

Proof *Proof.* From $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$: $\det(\mathbf{Q}^T)\det(\mathbf{Q}) = 1$, and $\det(\mathbf{Q}^T) = \det(\mathbf{Q})$, so $\det(\mathbf{Q})^2 = 1$. □

Prop 3.14 *Proposition 3.14* **(Product of Orthogonal Matrices is Orthogonal).** If $\mathbf{Q}_1, \mathbf{Q}_2 \in \mathbb{R}^{n\times n}$ are orthogonal, then $\mathbf{Q}_1\mathbf{Q}_2$ is also orthogonal.

Proof *Proof.* $(\mathbf{Q}_1\mathbf{Q}_2)^T(\mathbf{Q}_1\mathbf{Q}_2) = \mathbf{Q}_2^T(\mathbf{Q}_1^T\mathbf{Q}_1)\mathbf{Q}_2 = \mathbf{Q}_2^T\mathbf{I}\mathbf{Q}_2 = \mathbf{I}$. □

3.3.1 QR Decomposition

Def **QR Decomposition.** For a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$, the QR Decomposition factors $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{Q}\mathbf{R},$$

where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is an orthogonal matrix (i.e., $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) and $\mathbf{R} \in \mathbb{R}^{m\times n}$ is an upper triangular matrix with zeros below the main diagonal.

The QR factorization is particularly valuable in scientific computing because it transforms problems involving arbitrary matrices into problems with triangular matrices, which are much easier to solve. In addition, the orthogonality of $\mathbf{Q}$ ensures numerical stability in many applications, as orthogonal transformations preserve Euclidean norms and don't amplify errors.

Prop 3.15

Proposition 3.15 **(QR and the Column Space).** If $\mathbf{A} \in \mathbb{R}^{m\times n}$ $(m \geq n)$ has full column rank, the first n columns of $\mathbf{Q}$ (from the full QR factorization $\mathbf{A} = \mathbf{QR}$) form an orthonormal basis for $\mathrm{col}(\mathbf{A})$.

Prop 3.16 *Proposition 3.16* **(Diagonal Entries of R).** In the QR factorization of a full-column-rank matrix $\mathbf{A}$, the diagonal entries of $\mathbf{R}$ satisfy $|r_{ii}| = \|\mathbf{A}[:, i] - \mathrm{proj}\|_2 > 0$ for each i. In particular, $\mathbf{R}$ is invertible.

Lem 3.17 *Lemma 3.17* **(QR Factorization Cost).** The QR factorization of $\mathbf{A} \in \mathbb{R}^{m\times n}$ (via Householder reflections) costs approximately $2mn^2 - \frac{2}{3}n^3$ flops.

Ex 3.7

1. For $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m = n$, simplify the QR cost formula. Compare it to LU factorization cost ($\frac{2}{3}n^3$). Which is cheaper and by what factor?
2. Why does a zero diagonal entry in $\mathbf{R}$ imply $\mathbf{A}$ is rank-deficient? (*Hint:* $\mathbf{A} = \mathbf{QR}$ *and* $\mathbf{Q}$ *is invertible.*)
3. In NumPy, `np.linalg.qr(A)` returns the thin QR by default (`mode='reduced'`). What are the shapes of $\mathbf{Q}$ and $\mathbf{R}$ for a 5×3 matrix? When would you want the full QR (`mode='complete'`)?

3.3.2 Grahm Schmidt

Def **Orthogonal Complement.** Let $S \subseteq \mathbb{R}^n$ be a subspace. Its *orthogonal complement* is

$$S^\perp = \{\boldsymbol{w} \in \mathbb{R}^n : \boldsymbol{w}^T\boldsymbol{v} = 0 \text{ for all } \boldsymbol{v} \in S\}.$$

$S^\perp$ is itself a subspace of $\mathbb{R}^n$.

Prop 3.18 *Proposition 3.18* **(Dimension of the Orthogonal Complement).** If $S \subseteq \mathbb{R}^n$ has dimension k, then $\dim(S^\perp) = n - k$ and $S \cap S^\perp = \{\mathbf{0}\}$.

Prop 3.19 *Proposition 3.19* **(Orthogonal Decomposition).** Every vector $\boldsymbol{x} \in \mathbb{R}^n$ decomposes *uniquely* as $\boldsymbol{x} = \boldsymbol{x}_S + \boldsymbol{x}_{S^\perp}$ where $\boldsymbol{x}_S \in S$ and $\boldsymbol{x}_{S^\perp} \in S^\perp$. The component $\boldsymbol{x}_S$ is the orthogonal projection of $\boldsymbol{x}$ onto S.

Prop 3.20 *Proposition 3.20* **(Null Space as an Orthogonal Complement).** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ with columns spanning $S = \mathrm{col}(\mathbf{A})$, the orthogonal complement satisfies $S^\perp = \ker(\mathbf{A}^T)$, i.e., $S^\perp = \{\boldsymbol{w} : \mathbf{A}^T\boldsymbol{w} = \mathbf{0}\}$.

Ex 3.8

1. Let $S = \mathrm{span}\{(1, 1, 0)^T, (0, 1, 1)^T\}$ in $\mathbb{R}^3$. Find a basis for $S^\perp$ by solving $\mathbf{A}^T\boldsymbol{w} = \mathbf{0}$ where $\mathbf{A}$ has the spanning vectors as columns.

2. Verify that your answer to (1) satisfies $\dim(S) + \dim(S^\perp) = 3$.

3. In NumPy, use the QR factorization of $\mathbf{A}$ to extract an orthonormal basis for $\text{col}(\mathbf{A})$ and for $\text{col}(\mathbf{A})^\perp = \ker(\mathbf{A}^T)$.

Def **Gram-Schmidt Process.** The Gram-Schmidt process is an algorithm for converting a set of linearly independent vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ into an orthogonal or orthonormal basis for the same subspace. The process works by sequentially projecting each vector onto the orthogonal complement of the subspace spanned by the previous orthogonalized vectors.

For a set of linearly independent vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$, the Gram-Schmidt process generates an orthogonal set $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$ as follows:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1 \tag{17}$$

$$\boldsymbol{u}_2 = \boldsymbol{v}_2 - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_2}{\boldsymbol{u}_1^T \boldsymbol{u}_1} \boldsymbol{u}_1 \tag{18}$$

$$\boldsymbol{u}_3 = \boldsymbol{v}_3 - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_3}{\boldsymbol{u}_1^T \boldsymbol{u}_1} \boldsymbol{u}_1 - \frac{\boldsymbol{u}_2^T \boldsymbol{v}_3}{\boldsymbol{u}_2^T \boldsymbol{u}_2} \boldsymbol{u}_2 \tag{19}$$

$$\vdots \tag{20}$$

$$\boldsymbol{u}_k = \boldsymbol{v}_k - \sum_{j=1}^{k-1} \frac{\boldsymbol{u}_j^T \boldsymbol{v}_k}{\boldsymbol{u}_j^T \boldsymbol{u}_j} \boldsymbol{u}_j \tag{21}$$

To obtain an orthonormal basis $\{\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n\}$, we normalize each orthogonal vector:

$$\boldsymbol{q}_i = \frac{\boldsymbol{u}_i}{\|\boldsymbol{u}_i\|_2} = \frac{\boldsymbol{u}_i}{\sqrt{\boldsymbol{u}_i^T \boldsymbol{u}_i}} \tag{22}$$

Note The Gram-Schmidt process can be interpreted geometrically:

- $\boldsymbol{u}_1$ is simply the first vector $\boldsymbol{v}_1$
- $\boldsymbol{u}_2$ is $\boldsymbol{v}_2$ minus its projection onto $\boldsymbol{u}_1$
- $\boldsymbol{u}_3$ is $\boldsymbol{v}_3$ minus its projections onto both $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$

Each $\boldsymbol{u}_k$ is the component of $\boldsymbol{v}_k$ that is orthogonal to the subspace spanned by $\{\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_{k-1}\}$.

Example Gram-Schmidt ProcessConsider the vectors $\boldsymbol{v}_1 = [1, 1, 1]^T$ and $\boldsymbol{v}_2 = [0, 1, 2]^T$ in $\mathbb{R}^3$. Let's apply the Gram-Schmidt process:

- Step 1: $\boldsymbol{u}_1 = \boldsymbol{v}_1 = [1, 1, 1]^T$

- Step 2: We compute $\boldsymbol{u}_2$ as follows:

$$\boldsymbol{u}_2 = \boldsymbol{v}_2 - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_2}{\boldsymbol{u}_1^T \boldsymbol{u}_1} \boldsymbol{u}_1 \tag{23}$$

$$= [0, 1, 2]^T - \frac{[1, 1, 1][0, 1, 2]^T}{[1, 1, 1][1, 1, 1]^T} [1, 1, 1]^T \tag{24}$$

$$= [0, 1, 2]^T - \frac{3}{3} [1, 1, 1]^T \tag{25}$$

$$= [0, 1, 2]^T - [1, 1, 1]^T \tag{26}$$

$$= [-1, 0, 1]^T \tag{27}$$

- To normalize:

$$\boldsymbol{q}_1 = \frac{\boldsymbol{u}_1}{\|\boldsymbol{u}_1\|_2} = \frac{[1, 1, 1]^T}{\sqrt{3}} = [\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}]^T \tag{28}$$

$$\boldsymbol{q}_2 = \frac{\boldsymbol{u}_2}{\|\boldsymbol{u}_2\|_2} = \frac{[-1, 0, 1]^T}{\sqrt{2}} = [-\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}]^T \tag{29}$$

  Thus, we have transformed the linearly independent vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ into the orthonormal vectors $\boldsymbol{q}_1$ and $\boldsymbol{q}_2$.

The Gram-Schmidt process provides a constructive proof of the existence of an orthonormal basis for any subspace of $\mathbb{R}^n$. In practice, however, the classical Gram-Schmidt algorithm can suffer from numerical instability due to accumulated rounding errors, especially for large sets of vectors. The modified Gram-Schmidt algorithm, which applies the orthogonalization against each previously computed orthogonal vector immediately, offers better numerical stability and is preferred in computational applications.

Def **Modified Gram-Schmidt.** The Modified Gram-Schmidt algorithm is a numerically more stable version of the classical Gram-Schmidt process. Instead of computing all projections simultaneously for each new vector, it orthogonalizes against each previous vector sequentially:

$$\boldsymbol{v}_1^{(1)} = \boldsymbol{v}_1 \tag{30}$$

$$\boldsymbol{u}_1 = \boldsymbol{v}_1^{(1)} \tag{31}$$

$$\boldsymbol{v}_2^{(1)} = \boldsymbol{v}_2 \tag{32}$$

$$\boldsymbol{v}_2^{(2)} = \boldsymbol{v}_2^{(1)} - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_2^{(1)}}{\boldsymbol{u}_1^T \boldsymbol{u}_1} \boldsymbol{u}_1 \tag{33}$$

$$\boldsymbol{u}_2 = \boldsymbol{v}_2^{(2)} \tag{34}$$

$$\vdots \tag{35}$$

$$\boldsymbol{v}_k^{(1)} = \boldsymbol{v}_k \tag{36}$$

$$\boldsymbol{v}_k^{(2)} = \boldsymbol{v}_k^{(1)} - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_k^{(1)}}{\boldsymbol{u}_1^T \boldsymbol{u}_1} \boldsymbol{u}_1 \tag{37}$$

$$\boldsymbol{v}_k^{(3)} = \boldsymbol{v}_k^{(2)} - \frac{\boldsymbol{u}_2^T \boldsymbol{v}_k^{(2)}}{\boldsymbol{u}_2^T \boldsymbol{u}_2} \boldsymbol{u}_2 \tag{38}$$

$$\vdots \tag{39}$$

$$\boldsymbol{v}_k^{(k)} = \boldsymbol{v}_k^{(k-1)} - \frac{\boldsymbol{u}_{k-1}^T \boldsymbol{v}_k^{(k-1)}}{\boldsymbol{u}_{k-1}^T \boldsymbol{u}_{k-1}} \boldsymbol{u}_{k-1} \tag{40}$$

$$\boldsymbol{u}_k = \boldsymbol{v}_k^{(k)} \tag{41}$$

As before, to obtain an orthonormal basis, normalize each vector:

$$\boldsymbol{q}_i = \frac{\boldsymbol{u}_i}{\|\boldsymbol{u}_i\|_2} \tag{42}$$

Thm 3.21 *Theorem 3.21* **(Classical vs. Modified Gram-Schmidt Numerical Stability).** Both CGS and MGS produce the QR factorization in exact arithmetic, but differ numerically. Let $\hat{\mathbf{Q}}$ denote the computed factor. Then:

- **Classical (CGS):** $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_2 = O(\varepsilon_{\text{mach}}\,\kappa(\mathbf{A})^2)$
- **Modified (MGS):** $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_2 = O(\varepsilon_{\text{mach}}\,\kappa(\mathbf{A}))$

Both algorithms cost $O(mn^2)$ flops for $\mathbf{A} \in \mathbb{R}^{m\times n}$.

Note The key difference: CGS computes all projections from the *original* vector $\boldsymbol{v}_k$, so rounding errors in early projections contaminate all subsequent ones. MGS re-uses the *partially orthogonalized* vector at each step, immediately removing components already seen, which limits error accumulation. For maximum stability (at the same $O(mn^2)$ cost), Householder QR achieves $O(\varepsilon_{\text{mach}})$ loss of orthogonality regardless of $\kappa(\mathbf{A})$; it is what `np.linalg.qr` actually uses.

Note

The Gram-Schmidt process is closely related to the QR decomposition. If we arrange the original vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ as columns of a matrix $\mathbf{A}$ and the orthonormal vectors $\{\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n\}$ as columns of a matrix $\mathbf{Q}$, then:

$$\mathbf{A} = \mathbf{QR}$$

where $\mathbf{R}$ is an upper triangular matrix with elements:

$$r_{ij} = \begin{cases} \boldsymbol{q}_i^T \boldsymbol{v}_j & \text{if } i \leq j \\ 0 & \text{if } i > j \end{cases}$$

The elements of $\mathbf{R}$ represent the projection coefficients computed during the Gram-Schmidt process.

Example

Gram-Schmidt and QR DecompositionConsider the matrix $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$ whose columns are the vectors $\boldsymbol{v}_1 = [1, 1, 1]^T$ and $\boldsymbol{v}_2 = [0, 1, 2]^T$ from our previous example.
From the Gram-Schmidt process, we obtained:

$$\boldsymbol{q}_1 = [\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}]^T \tag{43}$$

$$\boldsymbol{q}_2 = [-\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}]^T \tag{44}$$

The $\mathbf{R}$ matrix is given by:

$$r_{11} = \boldsymbol{q}_1^T \boldsymbol{v}_1 = \frac{1}{\sqrt{3}} \cdot 3 = \sqrt{3} \tag{45}$$

$$r_{12} = \boldsymbol{q}_1^T \boldsymbol{v}_2 = \frac{1}{\sqrt{3}} \cdot 3 = \sqrt{3} \tag{46}$$

$$r_{21} = 0 \quad \text{(by construction)} \tag{47}$$

$$r_{22} = \boldsymbol{q}_2^T \boldsymbol{v}_2^{(2)} = \boldsymbol{q}_2^T \boldsymbol{u}_2 = \frac{1}{\sqrt{2}} \cdot \sqrt{2} = \sqrt{2} \tag{48}$$

Therefore:

$$\mathbf{R} = \begin{bmatrix} \sqrt{3} & \sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix}$$

And we can verify that $\mathbf{A} = \mathbf{QR}$:

$$\mathbf{QR} = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{3}} & 0 \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{3} & \sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix} \tag{49}$$

$$= \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \mathbf{A} \tag{50}$$

Ex 3.9 Let $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 0 & 1 \end{pmatrix}$.

1. Apply classical Gram-Schmidt by hand to the columns of $\mathbf{A}$ to find an orthonormal basis $\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$ for $\mathrm{col}(\mathbf{A})$.
2. Write out $\mathbf{Q} \in \mathbb{R}^{3\times 2}$ and $\mathbf{R} \in \mathbb{R}^{2\times 2}$ and verify $\mathbf{A} = \mathbf{QR}$.
3. Apply modified Gram-Schmidt to the same columns. Do you get the same $\mathbf{Q}, \mathbf{R}$ (in exact arithmetic)?
4. In NumPy, compute `np.linalg.qr(A)` and compare. Verify `Q.T @ Q` is close to $\mathbf{I}$.

Ex 3.10 **(Orthogonal Projection)** Let $\mathbf{Q} \in \mathbb{R}^{m\times n}$ (with $m > n$) have orthonormal columns (thin QR). The orthogonal projector onto $\mathrm{col}(\mathbf{Q})$ is $\mathbf{P} = \mathbf{QQ}^T$.

1. Show that $\mathbf{P}^2 = \mathbf{P}$ (idempotent) and $\mathbf{P}^T = \mathbf{P}$ (symmetric).
2. Show that $\mathbf{P}\boldsymbol{x}$ is the closest vector in $\mathrm{col}(\mathbf{Q})$ to $\boldsymbol{x}$, i.e., $\|\boldsymbol{x}-\mathbf{P}\boldsymbol{x}\|_2 \le \|\boldsymbol{x}-\boldsymbol{y}\|_2$ for all $\boldsymbol{y} \in \mathrm{col}(\mathbf{Q})$.
3. If $\mathbf{A} = \mathbf{QR}$ (full QR with $\mathbf{Q} \in \mathbb{R}^{m\times m}$ orthogonal), what is the projection onto $\mathrm{col}(\mathbf{A})$ in terms of the thin $\mathbf{Q}_1$ (first n columns of $\mathbf{Q}$)?
4. Write NumPy code to project a vector `x = np.random.randn(5)` onto the column space of a random 5×3 matrix `A`. Verify the residual `x - P @ x` is orthogonal to each column of `A`.

Ex 3.11 **(Stability Experiment)** Generate a nearly linearly dependent set of vectors:

```
V = np.column_stack([np.ones(50), np.linspace(0,1,50)**k for k in
                                    range(1,6)])
```

This is a Vandermonde-like matrix with $\kappa(\mathbf{V}) \gg 1$.

1. Implement classical Gram-Schmidt and compute $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_F$.
2. Implement modified Gram-Schmidt and compute the same quantity. Compare.
3. Use `np.linalg.qr(V)` (Householder) and compute the same quantity. What do you observe?
4. Compute $\kappa(\mathbf{V})$ via `np.linalg.cond(V)`. Does the stability gap between CGS and MGS scale as the theory predicts?

Note The Gram-Schmidt process and QR factorization are foundational tools in numerical linear algebra, underlying algorithms for solving linear systems, least-squares problems, eigenvalue computation (the QR algorithm), and orthogonal basis construction in iterative solvers.

Ex 3.12 For each application below, identify which property of QR (orthogonality of $\mathbf{Q}$, triangularity of $\mathbf{R}$, or both) is the key ingredient and write one sentence explaining why.

1. Solving a square linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ via $\mathbf{A} = \mathbf{QR}$.
2. Computing the least-squares solution to an overdetermined system $\mathbf{A}\boldsymbol{x} \approx \boldsymbol{b}$.
3. One step of the QR eigenvalue algorithm: $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$, $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k$.
4. Constructing a coordinate system aligned with a given set of data directions.

4 Iterative Methods for Linear Systems

4.1 Iterative Methods for Linear Systems

So far we have examined direct methods for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$: computing the answer in a fixed, predictable number of operations. For large sparse systems, however, direct methods become impractical — even when $\mathbf{A}$ is sparse, its LU factors may be dense (*fill-in*), destroying the memory advantage. Iterative methods offer an alternative: refine an initial guess through successive approximations, exploiting sparsity at every step.

Def **Direct Inverse Method.** The analytical formula $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$ requires explicitly forming $\mathbf{A}^{-1}$, which costs $O(n^3)$ flops and can suffer from severe numerical instability when $\mathbf{A}$ is nearly singular.

Def **Matrix Factorization Methods.** Methods such as LU, Cholesky, and QR decompose $\mathbf{A}$ into simpler factors at a one-time cost of $O(n^3)$. Each subsequent solve with a new right-hand side then requires only $O(n^2)$ operations via triangular solves.

Note Factorization methods are effective for small to medium-sized problems, but become impractical for very large sparse systems: even when $\mathbf{A}$ is sparse, the factors $\mathbf{L}$ and $\mathbf{U}$ may be dense (fill-in), destroying the memory advantage. For example, the $n^2 \times n^2$ matrix arising from a 2D finite difference discretization on an $n \times n$ grid has $O(n^2)$ nonzeros, but its LU factors have $O(n^3)$ nonzeros.

Def

Stationary Iterative Scheme. A *stationary iterative method* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ takes the form

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}, \quad k = 0, 1, 2, \ldots$$

for a fixed *iteration matrix* $\mathbf{T} \in \mathbb{R}^{n\times n}$ and a fixed vector $\boldsymbol{c} \in \mathbb{R}^n$ derived from $\mathbf{A}$ and $\boldsymbol{b}$. Starting from an initial guess $\boldsymbol{x}^{(0)}$, the method generates a sequence $\{\boldsymbol{x}^{(k)}\}$.

Note The iteration is consistent with $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ if every fixed point $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$ satisfies $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. Rearranging $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ as $\boldsymbol{x} = (\mathbf{I} - \mathbf{M}^{-1}\mathbf{A})\boldsymbol{x} + \mathbf{M}^{-1}\boldsymbol{b}$ for some invertible *splitting matrix* $\mathbf{M}$ yields $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ and $\boldsymbol{c} = \mathbf{M}^{-1}\boldsymbol{b}$. The art lies in choosing $\mathbf{M}$ so that $\mathbf{T}$ has small spectral radius and $\mathbf{M}^{-1}$ is cheap to apply.

Def **Spectral Radius.** The *spectral radius* of a matrix $\mathbf{B} \in \mathbb{R}^{n\times n}$ is

$$\rho(\mathbf{B}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{B})\},$$

the largest absolute value among all eigenvalues of $\mathbf{B}$.

Thm 4.1 *Theorem 4.1* **(Convergence of Stationary Iteration).** The stationary iteration $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ converges to the fixed point $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$ for *every* initial vector $\boldsymbol{x}^{(0)}$ if and only if $\rho(\mathbf{T}) < 1$.

Proof *Proof.* Let $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$. Subtracting the fixed-point equation from the iteration gives $\boldsymbol{e}^{(k+1)} = \mathbf{T}\boldsymbol{e}^{(k)}$, so $\boldsymbol{e}^{(k)} = \mathbf{T}^k\boldsymbol{e}^{(0)}$.
($\Leftarrow$) If $\rho(\mathbf{T}) < 1$: Choose $\epsilon > 0$ small enough that $\rho(\mathbf{T}) + \epsilon < 1$. By Gelfand's formula, $\lim_{k\to\infty} \|\mathbf{T}^k\|^{1/k} = \rho(\mathbf{T})$, so $\|\mathbf{T}^k\| \le C(\rho(\mathbf{T}) + \epsilon)^k$ for some constant $C > 0$. Hence $\|\boldsymbol{e}^{(k)}\| \le C(\rho(\mathbf{T}) + \epsilon)^k\|\boldsymbol{e}^{(0)}\| \to 0$ for all $\boldsymbol{x}^{(0)}$.
($\Rightarrow$) If $\rho(\mathbf{T}) \ge 1$: There exists an eigenvalue λ with $|\lambda| \ge 1$ and eigenvector $\boldsymbol{v} \neq \mathbf{0}$. Take $\boldsymbol{x}^{(0)} = \boldsymbol{x}^* + \boldsymbol{v}$; then $\boldsymbol{e}^{(k)} = \lambda^k\boldsymbol{v}$, so $\|\boldsymbol{e}^{(k)}\| = |\lambda|^k\|\boldsymbol{v}\| \not\to 0$. □

Cor 4.2 *Corollary 4.2* **(Geometric Convergence Rate).** When $\rho(\mathbf{T}) < 1$, the error satisfies $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| = O(\rho(\mathbf{T})^k)$. To reduce the error by a factor of 10^{-p}, approximately $p/\log_{10}(1/\rho(\mathbf{T}))$ iterations are required.

Proof *Proof.* From Gelfand's formula, $\|\mathbf{T}^k\| \le C\rho(\mathbf{T})^k$ (up to a constant). Solving $C\rho(\mathbf{T})^k \le 10^{-p}$ for k and taking logarithms gives $k \ge (\log C + p)/\log_{10}(1/\rho(\mathbf{T}))$, which is $O(p/\log_{10}(1/\rho(\mathbf{T})))$. □

Ex 4.1 Let $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\boldsymbol{b} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$ (exact solution $\boldsymbol{x}^* = (1, 1)^T$).

1. Split $\mathbf{A} = \mathbf{D} + (\mathbf{A} - \mathbf{D})$ where $\mathbf{D} = \operatorname{diag}(3, 3)$. Write the stationary iteration in the form $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$, stating $\mathbf{T}$ and $\boldsymbol{c}$ explicitly.

2. Compute $\rho(\mathbf{T})$ and predict the number of iterations needed to reduce the initial error by 10^{-6}.

3. Starting from $\boldsymbol{x}^{(0)} = (0,0)^T$, carry out three iterations by hand and verify that the error decreases at the rate predicted by $\rho(\mathbf{T})$.

Ex 4.2 **(Fill-in phenomenon)** Let $\mathbf{A}$ be the $n \times n$ tridiagonal matrix with 2 on the main diagonal and -1 on the super- and sub-diagonals (arising from finite differences for $-u'' = f$ on $[0,1]$).

1. How many nonzeros does $\mathbf{A}$ have? Is it $O(n)$ or $O(n^2)$?
2. Use `scipy.linalg.lu` to compute the LU factorization for $n = 20$. Count the nonzeros in $\mathbf{L}$ and $\mathbf{U}$. Is there fill-in? Explain why or why not.
3. Use `scipy.sparse.linalg.spsolve` to solve $\mathbf{A}\boldsymbol{x} = \mathbf{1}$ in sparse format. Compare timing against dense LU for $n = 1000$.

4.2 Jacobi Iteration

4.2.1 Jacobi Iteration

Def **Splitting of A.** To derive a stationary iteration from $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, we decompose $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{D} + \mathbf{R}, \qquad \mathbf{D} = \operatorname{diag}(a_{11}, \ldots, a_{nn}), \quad \mathbf{R} = \mathbf{A} - \mathbf{D}.$$

Assuming all diagonal entries are nonzero ($a_{ii} \neq 0$ for all i), we can rearrange $(\mathbf{D} + \mathbf{R})\boldsymbol{x} = \boldsymbol{b}$ to obtain the fixed-point equation $\boldsymbol{x} = -\mathbf{D}^{-1}\mathbf{R}\boldsymbol{x} + \mathbf{D}^{-1}\boldsymbol{b}$.

Def **Jacobi Iteration.** The *Jacobi iteration* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is the stationary scheme

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}_J \boldsymbol{x}^{(k)} + \mathbf{D}^{-1}\boldsymbol{b}, \qquad \mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R},$$

or in component form,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}\Big), \quad i = 1, \ldots, n.$$

Each component $x_i^{(k+1)}$ depends only on values from the *previous* iterate $\boldsymbol{x}^{(k)}$, so all n components may be updated simultaneously (or in any order).

4.2.2 Convergence of Jacobi Iteration

Thm 4.3 *Theorem 4.3* **(Convergence of Jacobi Iteration).** The Jacobi iteration converges to the solution of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ for every initial vector $\boldsymbol{x}^{(0)}$ if and only if $\rho(\mathbf{T}_J) < 1$.

Proof *Proof.* This is a direct application of the general convergence theorem for stationary iterations: convergence for all $\boldsymbol{x}^{(0)}$ is equivalent to $\rho(\mathbf{T}) < 1$ for the iteration matrix. Here $\mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$, so the fixed-point equation $\boldsymbol{x}^* = \mathbf{T}_J\boldsymbol{x}^* + \mathbf{D}^{-1}\boldsymbol{b}$ simplifies to $\mathbf{D}^{-1}\mathbf{A}\boldsymbol{x}^* = \mathbf{D}^{-1}\boldsymbol{b}$, i.e., $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. □

Thm 4.4 *Theorem 4.4* **(Jacobi Convergence for Strictly Diagonally Dominant Matrices).** If $\mathbf{A}$ is strictly diagonally dominant, meaning

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \text{for all } i = 1, \ldots, n,$$

then $\rho(\mathbf{T}_J) < 1$ and the Jacobi iteration converges for every $\boldsymbol{x}^{(0)}$.

Proof *Proof.* Let λ be any eigenvalue of $\mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R}$ with eigenvector $\boldsymbol{v} \neq \mathbf{0}$. Let index i^* satisfy $|v_{i^*}| = \|\boldsymbol{v}\|_\infty > 0$. The eigenvalue equation $\mathbf{T}_J\boldsymbol{v} = \lambda\boldsymbol{v}$ gives, for row i^*:

$$\lambda v_{i^*} = -\frac{1}{a_{i^*i^*}} \sum_{j \neq i^*} a_{i^*j}\, v_j.$$

Taking absolute values and dividing by $|v_{i^*}|$:

$$|\lambda| \leq \frac{1}{|a_{i^*i^*}|} \sum_{j \neq i^*} |a_{i^*j}| \cdot \frac{|v_j|}{|v_{i^*}|} \leq \frac{\sum_{j \neq i^*} |a_{i^*j}|}{|a_{i^*i^*}|} < 1,$$

where the last inequality is strict diagonal dominance. Since λ was arbitrary, $\rho(\mathbf{T}_J) < 1$. □

Example **Effect of matrix structure on convergence.**

Case 1 — Strictly diagonally dominant: $\mathbf{A}_1 = \begin{pmatrix} 5 & 1 & 1 \\ 1 & 5 & 1 \\ 1 & 1 & 5 \end{pmatrix}$. Each row satisfies $|a_{ii}| = 5 > 2 = \sum_{j \neq i} |a_{ij}|$, so the theorem guarantees convergence. One can compute $\rho(\mathbf{T}_J) = 2/5 = 0.4$.

Case 2 — Diagonal dominance violated in one row: $\mathbf{A}_2 = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 3 & 1 \\ 2 & 2 & 2 \end{pmatrix}$. Row 3 violates strict diagonal dominance. The theorem gives no guarantee; $\rho(\mathbf{T}_J)$ must be checked numerically.

Case 3 — Divergent case: $\mathbf{A}_3 = \begin{pmatrix} 1 & 3 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$. Here $\rho(\mathbf{T}_J) > 1$ and the Jacobi iteration diverges for every $\boldsymbol{x}^{(0)} \neq \boldsymbol{x}^*$.

4.2.3 Stopping Criteria

Def **Residual-Based Stopping Criterion.** Stop when the norm of the residual falls below a tolerance:

$$\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\| < \epsilon_{\text{res}}.$$

This directly measures how well $\boldsymbol{x}^{(k)}$ satisfies the linear system.

Def **Increment-Based Stopping Criterion.** Stop when the change between successive iterates is small:

$$\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| < \epsilon_{\text{inc}}.$$

This is cheap (no matrix–vector product needed), but measures the rate of change of the iteration rather than the accuracy of the solution directly.

Def **Relative Residual Criterion.** Stop when the residual is small relative to the right-hand side:

$$\frac{\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\|}{\|\boldsymbol{b}\|} < \epsilon_{\text{rel}}.$$

This is scale-invariant: multiplying $\mathbf{A}$ and $\boldsymbol{b}$ by a constant does not alter the stopping condition.

Note A *maximum iteration count* is always imposed as a safeguard against slow convergence or divergence. Typical practice is to set $k_{\max}$ based on the problem size and report non-convergence if the tolerance is not met within the budget.

Thm 4.5 *Theorem 4.5* **(A Priori Error Bound for Jacobi Iteration).** Let $\rho = \rho(\mathbf{T}_J) < 1$. For all $k \geq 1$,

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| \leq \frac{\rho^k}{1-\rho}\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|.$$

Proof *Proof.* The basic contraction estimate gives $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| \leq \rho^k\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|$. We bound $\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|$ via the telescoping sum:

$$\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\| \leq \sum_{j=0}^{\infty}\|\boldsymbol{x}^{(j+1)} - \boldsymbol{x}^{(j)}\|.$$

Since $\|\boldsymbol{x}^{(j+1)} - \boldsymbol{x}^{(j)}\| = \|\mathbf{T}_J^j(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)})\| \leq \rho^j\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|$, the geometric series gives $\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\| \leq \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|/(1-\rho)$. Combining yields the stated bound. □

Ex 4.3 Implement the Jacobi method in Python.

1. Write `jacobi(A, b, x0, tol, max_iter)` that returns the iterate at convergence (relative residual below `tol`) and the number of iterations used.

2. Test on $\mathbf{A}_1$ from the example with $\boldsymbol{b} = (7,7,7)^T$. Compute the exact solution by hand. How many iterations are needed for $\epsilon_{\text{rel}} = 10^{-8}$?

3. Plot $\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\|$ versus k on a semi-log scale. What is the observed slope? Compare to $\log_{10} \rho(\mathbf{T}_J)$.

Ex 4.4 **(A priori vs. observed convergence)** For the system with $\mathbf{A}_1$ and $\boldsymbol{b} = (7,7,7)^T$:

1. Evaluate the a priori bound for $k = 20$ iterations. How many times larger is the bound than the actual error $\|\boldsymbol{x}^{(20)} - \boldsymbol{x}^*\|$?

2. Consider the 5×5 tridiagonal matrix with 4 on the diagonal and -1 on the off-diagonals. Verify that it is strictly diagonally dominant, compute $\rho(\mathbf{T}_J)$ numerically, and determine how many iterations are needed to achieve $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| < 10^{-10}$.

4.3 Conjugate Gradient Method

The Conjugate Gradient (CG) method is an efficient iterative solver for symmetric positive definite linear systems. By maintaining a set of $\mathbf{A}$-conjugate search directions built from the residuals, it converges to the exact solution in at most n steps in exact arithmetic, while requiring only $O(n)$ storage and one matrix–vector product per iteration. Its convergence rate depends on $\sqrt{\kappa(\mathbf{A})}$ rather than $\kappa(\mathbf{A})$, giving a dramatic speedup over stationary methods on ill-conditioned systems.

4.3.1 From Linear Systems to Optimization

Thm 4.6 *Theorem 4.6* **(Equivalence of Linear System and Quadratic Minimization).** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric positive definite and $\boldsymbol{b} \in \mathbb{R}^n$. Define

$$\phi(\boldsymbol{x}) = \tfrac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{x}^T\boldsymbol{b}.$$

Then $\boldsymbol{x}^*$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ if and only if $\boldsymbol{x}^*$ minimizes ϕ.

Proof *Proof.* Since $\mathbf{A}$ is symmetric, $\nabla\phi(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} - \boldsymbol{b}$. A point $\boldsymbol{x}^*$ is a critical point of ϕ if and only if $\nabla\phi(\boldsymbol{x}^*) = \mathbf{0}$, i.e., $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. Since $\mathbf{A}$ is positive definite, ϕ is strictly convex (its Hessian $\mathbf{A} \succ 0$), so the unique critical point is the global minimum. □

Note When $\mathbf{A}$ is symmetric positive definite, $\phi(\boldsymbol{x})$ is a paraboloid with elliptical level sets in $\mathbb{R}^n$. The condition number $\kappa_2(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$ controls the eccentricity: high $\kappa(\mathbf{A})$ produces narrow, elongated valleys. Gradient-based methods zigzag

inefficiently in these valleys, while CG exploits the geometry via $\mathbf{A}$-conjugate directions.

Def **Residual and Steepest Descent.** At iterate $\boldsymbol{x}^{(k)}$, the *residual* $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)} = -\nabla\phi(\boldsymbol{x}^{(k)})$ points in the direction of steepest descent. The *steepest descent* update

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{r}^{(k)}, \qquad \alpha_k = \frac{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}{(\boldsymbol{r}^{(k)})^T \mathbf{A}\boldsymbol{r}^{(k)}}$$

minimizes ϕ along the direction $\boldsymbol{r}^{(k)}$ but often exhibits zigzagging in ill-conditioned systems.

Def **Stationary Method Limitations.** A stationary iterative method $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ reduces the error by a constant factor $\rho(\mathbf{T})$ per step. Its convergence rate for the Jacobi splitting satisfies $\rho(\mathbf{T}_J) \leq (\kappa(\mathbf{A}) - 1)/(\kappa(\mathbf{A}) + 1)$, which approaches 1 as $\kappa(\mathbf{A}) \to \infty$.

Note For $\kappa(\mathbf{A}) = 100$, the Jacobi factor is ≈ 0.98, requiring $\sim$500 iterations to reduce the error by 10^{-4}. Steepest descent suffers the same limitation. Conjugate Gradient breaks this barrier by adapting its search directions to the problem geometry.

4.3.2 A-Conjugate Directions

Def **A-Conjugacy.** Two vectors $\boldsymbol{p}, \boldsymbol{q} \in \mathbb{R}^n$ are $\mathbf{A}$-*conjugate* (or $\mathbf{A}$-*orthogonal*) if $\boldsymbol{p}^T\mathbf{A}\boldsymbol{q} = 0$. A set $\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{m-1}\}$ of nonzero vectors is $\mathbf{A}$-conjugate if $\boldsymbol{p}_i^T\mathbf{A}\boldsymbol{p}_j = 0$ for all $i \neq j$.

Prop 4.7 *Proposition 4.7* **(A-Conjugate Vectors are Linearly Independent).** If $\mathbf{A}$ is positive definite, every set of $\mathbf{A}$-conjugate nonzero vectors is linearly independent.

Proof *Proof.* Suppose $\sum_{i=0}^{m-1} c_i\boldsymbol{p}_i = \mathbf{0}$. Multiply on the left by $\boldsymbol{p}_j^T\mathbf{A}$:

$$0 = \boldsymbol{p}_j^T\mathbf{A}\Big(\sum_i c_i\boldsymbol{p}_i\Big) = c_j\,\boldsymbol{p}_j^T\mathbf{A}\boldsymbol{p}_j.$$

Since $\mathbf{A} \succ 0$, $\boldsymbol{p}_j^T\mathbf{A}\boldsymbol{p}_j > 0$. Hence $c_j = 0$ for each j. □

Prop 4.8 *Proposition 4.8* **(Exact Minimization Along n Conjugate Directions).** Let $\mathbf{A}$ be symmetric positive definite and $\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{n-1}\}$ be $\mathbf{A}$-conjugate in $\mathbb{R}^n$. Starting from $\boldsymbol{x}^{(0)}$, define

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \sum_{i=0}^{k-1} \alpha_i\boldsymbol{p}_i, \qquad \alpha_i = \frac{\boldsymbol{p}_i^T\boldsymbol{r}^{(i)}}{\boldsymbol{p}_i^T\mathbf{A}\boldsymbol{p}_i},$$

where $\boldsymbol{r}^{(i)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(i)}$. Then $\boldsymbol{x}^{(n)} = \boldsymbol{x}^*$ exactly.

Proof *Proof.* Since the $\boldsymbol{p}_i$ are linearly independent in $\mathbb{R}^n$, they form a basis. Write $\boldsymbol{x}^* - \boldsymbol{x}^{(0)} = \sum_i \alpha_i^* \boldsymbol{p}_i$. Multiplying by $\boldsymbol{p}_j^T \mathbf{A}$:

$$\boldsymbol{p}_j^T \mathbf{A}(\boldsymbol{x}^* - \boldsymbol{x}^{(0)}) = \alpha_j^* \boldsymbol{p}_j^T \mathbf{A}\boldsymbol{p}_j.$$

Since $\mathbf{A}(\boldsymbol{x}^* - \boldsymbol{x}^{(0)}) = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)} = \boldsymbol{r}^{(0)}$, we get $\alpha_j^* = \boldsymbol{p}_j^T \boldsymbol{r}^{(0)} / (\boldsymbol{p}_j^T \mathbf{A}\boldsymbol{p}_j)$. One verifies by induction that $\boldsymbol{p}_j^T \boldsymbol{r}^{(j)} = \boldsymbol{p}_j^T \boldsymbol{r}^{(0)}$ (because $\boldsymbol{r}^{(j)}$ differs from $\boldsymbol{r}^{(0)}$ only by $\mathbf{A}$-multiples of previous $\boldsymbol{p}_i$, which are $\mathbf{A}$-conjugate to $\boldsymbol{p}_j$). Hence $\alpha_j^* = \alpha_j$ and $\boldsymbol{x}^{(n)} = \boldsymbol{x}^*$. □

Def **Iterative Generation of A-Conjugate Directions.** The CG method builds $\mathbf{A}$-conjugate directions iteratively via a three-term recurrence. Given residual $\boldsymbol{r}^{(k+1)}$, the next search direction is:

$$\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}, \qquad \beta_k = \frac{(\boldsymbol{r}^{(k+1)})^T \boldsymbol{r}^{(k+1)}}{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}.$$

The choice of β_k ensures $\boldsymbol{p}^{(k+1)}$ is $\mathbf{A}$-conjugate to *all* previous directions, not just $\boldsymbol{p}^{(k)}$.

4.3.3 The CG Algorithm and Convergence

Def **Conjugate Gradient Algorithm. Initialize:** $\boldsymbol{x}^{(0)}$ (arbitrary), $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$, $\boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)}$.
For $k = 0, 1, 2, \ldots$ **until** $\|\boldsymbol{r}^{(k)}\| < \epsilon$**:**

$$\begin{aligned}
\alpha_k &= \frac{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}{(\boldsymbol{p}^{(k)})^T \mathbf{A}\boldsymbol{p}^{(k)}}, \\
\boldsymbol{x}^{(k+1)} &= \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}, \\
\boldsymbol{r}^{(k+1)} &= \boldsymbol{r}^{(k)} - \alpha_k \mathbf{A}\boldsymbol{p}^{(k)}, \\
\beta_k &= \frac{(\boldsymbol{r}^{(k+1)})^T \boldsymbol{r}^{(k+1)}}{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}, \\
\boldsymbol{p}^{(k+1)} &= \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}.
\end{aligned}$$

Prop 4.9 *Proposition 4.9* **(CG Storage Requirement).** At each step, only $\boldsymbol{x}^{(k)}$, $\boldsymbol{r}^{(k)}$, $\boldsymbol{p}^{(k)}$, and the product $\mathbf{A}\boldsymbol{p}^{(k)}$ need to be stored simultaneously — a constant number of n-vectors, independent of the iteration count.

Prop 4.10 *Proposition 4.10* **(CG Cost Per Iteration).** Each CG iteration requires one matrix–vector product $\mathbf{A}\boldsymbol{p}^{(k)}$ (cost $O(n^2)$ for a dense matrix, $O(\mathrm{nnz}(\mathbf{A}))$ for a sparse matrix), two vector dot products, and three vector updates — all at $O(n)$.

Thm 4.11 *Theorem 4.11* **(Finite Termination of Conjugate Gradient).** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric positive definite. The CG iterates satisfy:

1. $(\boldsymbol{r}^{(i)})^T \boldsymbol{r}^{(j)} = 0$ for $i \neq j$ (residuals are mutually orthogonal).
2. $(\boldsymbol{p}^{(i)})^T \mathbf{A}\boldsymbol{p}^{(j)} = 0$ for $i \neq j$ (search directions are $\mathbf{A}$-conjugate).

Consequently, CG terminates with $\boldsymbol{x}^{(m)} = \boldsymbol{x}^*$ for some $m \leq n$.

Proof *Proof.* We prove both properties by induction. Both hold vacuously for $k = 0$. Assume (1) and (2) hold through step k. For the orthogonality of residuals: using $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k \mathbf{A}\boldsymbol{p}^{(k)}$,

$$(\boldsymbol{r}^{(k+1)})^T \boldsymbol{r}^{(j)} = (\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(j)} - \alpha_k (\boldsymbol{p}^{(k)})^T \mathbf{A}\boldsymbol{r}^{(j)}.$$

Since each residual $\boldsymbol{r}^{(j)}$ is a linear combination of $\boldsymbol{p}^{(0)}, \ldots, \boldsymbol{p}^{(j)}$, and the search directions satisfy $\mathbf{A}$-conjugacy, the term $(\boldsymbol{p}^{(k)})^T \mathbf{A}\boldsymbol{r}^{(j)}$ vanishes for $j < k$ and equals $(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}$ for $j = k$, which is cancelled by the choice of α_k. For the $\mathbf{A}$-conjugacy of directions: by construction of β_k,

$$(\boldsymbol{p}^{(k+1)})^T \mathbf{A}\boldsymbol{p}^{(k)} = (\boldsymbol{r}^{(k+1)})^T \mathbf{A}\boldsymbol{p}^{(k)} + \beta_k (\boldsymbol{p}^{(k)})^T \mathbf{A}\boldsymbol{p}^{(k)} = 0$$

by the choice of β_k; inner products with earlier directions vanish by the inductive hypothesis.
Since there can be at most n mutually orthogonal nonzero residuals in $\mathbb{R}^n$, the algorithm must produce $\boldsymbol{r}^{(m)} = \mathbf{0}$ for some $m \leq n$. □

Thm 4.12 *Theorem 4.12* **(CG Convergence Rate Bound).** Let $\mathbf{A}$ be symmetric positive definite with condition number $\kappa = \kappa_2(\mathbf{A})$. Then for all $k \geq 0$:

$$\frac{\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}}}{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}}} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

where $\|\boldsymbol{v}\|_{\mathbf{A}} = \sqrt{\boldsymbol{v}^T \mathbf{A}\boldsymbol{v}}$ is the $\mathbf{A}$-norm (energy norm).

Proof *Proof.* One can show that $\boldsymbol{x}^{(k)}$ minimizes $\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}$ over all $\boldsymbol{x}^{(0)} + \mathcal{K}_k$, where $\mathcal{K}_k = \operatorname{span}\{\boldsymbol{r}^{(0)}, \mathbf{A}\boldsymbol{r}^{(0)}, \ldots, \mathbf{A}^{k-1}\boldsymbol{r}^{(0)}\}$ is the k-th Krylov subspace. Hence

$$\frac{\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}}}{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}}} \leq \min_{\substack{p\in\mathcal{P}_k \\ p(0)=1}} \max_{\lambda\in\sigma(\mathbf{A})} |p(\lambda)|,$$

where the minimum is over polynomials of degree k satisfying $p(0) = 1$. The Chebyshev polynomial T_k minimizes the maximum absolute value over $[\lambda_{\min}, \lambda_{\max}]$ among all such polynomials, and satisfies

$$\min_{\substack{p\in\mathcal{P}_k \\ p(0)=1}} \max_{\lambda\in[\lambda_{\min},\lambda_{\max}]} |p(\lambda)| = \frac{1}{T_k((\lambda_{\max}+\lambda_{\min})/(\lambda_{\max}-\lambda_{\min}))} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

where the last bound follows from the identity $T_k(\cosh\theta) = \cosh(k\theta)$ with $\cosh\theta = (\kappa+1)/(\kappa-1)$. $\square$

Note The CG convergence factor $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ is far smaller than the stationary method factor $(\kappa-1)/(\kappa+1)$ for large κ. For $\kappa = 100$: CG factor ≈ 0.82 (needs ≈ 22 iterations for 10^{-4} reduction), versus Jacobi factor ≈ 0.98 (needs ≈ 500 iterations). Moreover, if the eigenvalues of $\mathbf{A}$ are clustered into m distinct groups, CG converges in roughly m steps — this is *superlinear convergence*, far better than the worst-case bound.

4.3.4 Preconditioning

Def **Preconditioner.** A *preconditioner* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is a symmetric positive definite matrix $\mathbf{M}$ such that $\mathbf{M} \approx \mathbf{A}$ and $\mathbf{M}^{-1}$ is cheap to apply. The preconditioned system $\mathbf{M}^{-1/2}\mathbf{A}\mathbf{M}^{-1/2}\boldsymbol{y} = \mathbf{M}^{-1/2}\boldsymbol{b}$ (where $\boldsymbol{y} = \mathbf{M}^{1/2}\boldsymbol{x}$) has the same solution but ideally satisfies $\kappa(\mathbf{M}^{-1/2}\mathbf{A}\mathbf{M}^{-1/2}) \ll \kappa(\mathbf{A})$.

Note In practice, the Preconditioned Conjugate Gradient (PCG) algorithm applies $\mathbf{M}^{-1}$ at each step without forming $\mathbf{M}^{1/2}$ explicitly. Common preconditioners include: **Jacobi** ($\mathbf{M} = \mathrm{diag}(\mathbf{A})$, cheap but limited benefit), **incomplete Cholesky** ($\mathbf{M} \approx \mathbf{L}\mathbf{L}^T$ with controlled fill-in, far more effective), and **multigrid** (near-optimal $O(n)$ convergence for PDE-derived systems).

Ex 4.5 Implement the Conjugate Gradient method in Python.

1. Write `conjugate_gradient(A, b, x0, tol)` following the CG algorithm, stopping when $\|\boldsymbol{r}^{(k)}\|/\|\boldsymbol{r}^{(0)}\| <$ `tol`.
2. Test on the 100×100 SPD tridiagonal matrix $\mathbf{A} = \mathrm{tridiag}(-1, 2, -1)$. Plot the $\mathbf{A}$-norm error vs. iteration count and compare to the theoretical bound.
3. Verify your implementation against `scipy.sparse.linalg.cg`.

Ex 4.6 **(Preconditioning effect)** Using the tridiagonal $\mathbf{A}$ for $n = 200$:

1. Compute $\kappa_2(\mathbf{A})$ with `np.linalg.cond`. How many CG iterations does the bound predict for 10^{-8} accuracy?
2. Apply the Jacobi preconditioner $\mathbf{M} = \mathrm{diag}(\mathbf{A})$. What is $\kappa(\mathbf{M}^{-1}\mathbf{A})$? Why does it barely help here?
3. Implement incomplete Cholesky preconditioning via `scipy.sparse.linalg.spilu`. Compare iteration counts for the original and preconditioned CG.

Ex 4.7 **(Eigenvalue clustering and superlinear convergence)** Build a 50×50 SPD matrix with eigenvalues: 40 eigenvalues equal to 1, 5 equal to 10, and 5 equal to 100.

1. What is $\kappa(\mathbf{A})$? What does the CG bound predict for the number of iterations needed to achieve $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}} / \|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}} < 10^{-10}$?

2. Construct the matrix using `Q @ np.diag(eigs) @ Q.T` with a random orthogonal $\mathbf{Q}$ from `scipy.stats.ortho_group.rvs(50)`. Run CG and plot the actual error versus the bound.

3. How many iterations does CG actually need? Explain why superlinear convergence occurs when eigenvalues are clustered.

4.4 Stability and Condition Number

In an idealized mathematical setting, the solution to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is simply $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$. In practice, two separate issues corrupt the computed answer: the *inherent sensitivity* of the problem to perturbations in the data (measured by the condition number) and the *algorithm's ability to avoid amplifying errors* beyond this inherent sensitivity (measured by numerical stability). Understanding both is essential for reliable scientific computing.

4.4.1 Why Not Invert Directly?

Def **Computational Cost of Explicit Inversion.** Computing $\mathbf{A}^{-1}$ requires solving n linear systems $\mathbf{A}\mathbf{x}_i = \mathbf{e}_i$. After the LU factorization (one-time cost $\frac{2}{3}n^3$), each solve costs $2n^2$, giving a total of $\frac{2}{3}n^3 + 2n^3 \approx \frac{8}{3}n^3$ flops. Forming and storing the $n \times n$ inverse requires $O(n^2)$ memory and provides no benefit when only the product $\mathbf{A}^{-1}\boldsymbol{b}$ is needed.

Def **Numerical Instability of Explicit Inversion.** When $\mathbf{A}$ is nearly singular, explicitly forming $\mathbf{A}^{-1}$ amplifies round-off errors severely. A single erroneous column of $\mathbf{A}^{-1}$ propagates into the entire solution $\mathbf{A}^{-1}\boldsymbol{b}$, whereas solving via a stable factorization confines errors to the individual triangular solve steps.

Def **Preferred Strategy: Factorize Once, Solve Many.** Given the LU factorization $\mathbf{A} = \mathbf{PLU}$ (computed once at cost $\frac{2}{3}n^3$), each new right-hand side $\boldsymbol{b}$ requires only two triangular solves at cost $2n^2$. For k right-hand sides, total cost is $\frac{2}{3}n^3 + 2kn^2$, far better than k separate $O(n^3)$ solves or storing $\mathbf{A}^{-1}$.

Note The practical rule: **never explicitly form $\mathbf{A}^{-1}$**. Use `np.linalg.solve(A, b)` for a single system, or `scipy.linalg.lu_factor` / `scipy.linalg.lu_solve` when solving multiple systems with the same $\mathbf{A}$.

4.4.2 Numerical Stability

Def

Backward Stability. An algorithm for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is *backward stable* if the computed solution $\hat{\boldsymbol{x}}$ satisfies

$$(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$$

for some *backward errors* with $\|\delta\mathbf{A}\|/\|\mathbf{A}\| = O(\epsilon_{\text{mach}})$ and $\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\| = O(\epsilon_{\text{mach}})$. In words: the computed answer is the exact answer to a slightly perturbed problem.

Note Backward stability is a property of the *algorithm*, not the problem. Gaussian elimination without pivoting can be unstable; with partial pivoting it is backward stable for virtually all practical matrices.

Prop 4.13 *Proposition 4.13* **(Gaussian Elimination with Partial Pivoting is Backward Stable).** Gaussian elimination with partial pivoting (GEPP) produces a computed solution $\hat{\boldsymbol{x}}$ satisfying $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}$ with

$$\|\delta\mathbf{A}\|_\infty \leq g_n \cdot \epsilon_{\text{mach}} \cdot \|\mathbf{A}\|_\infty,$$

where g_n is the *growth factor*. In the worst case $g_n \leq 2^{n-1}$, but for random matrices in practice $g_n = O(n^{2/3})$.

Proof *Proof.* At each elimination step, partial pivoting selects the largest entry in the current column as pivot, ensuring all multipliers satisfy $|m_{ij}| \leq 1$. This prevents exponential amplification of round-off at each stage. A detailed rounding error analysis over the $O(n^3)$ scalar operations shows that the accumulated backward error satisfies the stated bound. For a complete proof see Higham, *Accuracy and Stability of Numerical Algorithms*, Ch. 9. □

4.4.3 Condition Number

Def **Condition Number.** The *condition number* of an invertible matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ with respect to a matrix norm $\|\cdot\|$ is

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|\,\|\mathbf{A}^{-1}\|.$$

For the 2-norm, $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ (ratio of largest to smallest singular values); for a symmetric matrix, $\kappa_2(\mathbf{A}) = |\lambda_{\max}|/|\lambda_{\min}|$.

Thm 4.14 *Theorem 4.14* **(Perturbation Bound for Linear Systems).** Let $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A}$ invertible and $\boldsymbol{b} \neq \mathbf{0}$. Suppose $\hat{\boldsymbol{x}}$ satisfies $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$. If $\|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\| < 1$, then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \frac{\kappa(\mathbf{A})}{1 - \kappa(\mathbf{A})\,\|\delta\mathbf{A}\|/\|\mathbf{A}\|}\left(\frac{\|\delta\mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}\right).$$

Proof

Proof. Subtracting $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ from $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$ gives

$$(\mathbf{A} + \delta\mathbf{A})(\hat{\boldsymbol{x}} - \boldsymbol{x}) = \delta\boldsymbol{b} - \delta\mathbf{A}\,\boldsymbol{x}.$$

Since $\|\mathbf{A}^{-1}\delta\mathbf{A}\| \le \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\| < 1$, the matrix $\mathbf{I} + \mathbf{A}^{-1}\delta\mathbf{A}$ is invertible by the Neumann lemma, and

$$\|(\mathbf{A} + \delta\mathbf{A})^{-1}\| = \|\mathbf{A}^{-1}(\mathbf{I} + \mathbf{A}^{-1}\delta\mathbf{A})^{-1}\| \le \frac{\|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\|}.$$

Therefore:

$$\|\hat{\boldsymbol{x}} - \boldsymbol{x}\| \le \frac{\|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\|}\big(\|\delta\boldsymbol{b}\| + \|\delta\mathbf{A}\|\,\|\boldsymbol{x}\|\big).$$

Dividing by $\|\boldsymbol{x}\|$ and using $\|\boldsymbol{b}\| \le \|\mathbf{A}\|\,\|\boldsymbol{x}\|$ (so $1/\|\boldsymbol{x}\| \le \|\mathbf{A}\|/\|\boldsymbol{b}\|$) yields the stated bound. □

Cor 4.15 *Corollary 4.15* **(Effect of Right-Hand Side Perturbation).** If only $\boldsymbol{b}$ is perturbed ($\delta\mathbf{A} = \mathbf{0}$), then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \kappa(\mathbf{A})\,\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}.$$

Proof *Proof.* Set $\delta\mathbf{A} = \mathbf{0}$ in the perturbation bound. Then $\hat{\boldsymbol{x}} - \boldsymbol{x} = \mathbf{A}^{-1}\delta\boldsymbol{b}$, so $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\| \le \|\mathbf{A}^{-1}\|\,\|\delta\boldsymbol{b}\|$. Using $\|\boldsymbol{x}\| \ge \|\boldsymbol{b}\|/\|\mathbf{A}\|$ gives $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|/\|\boldsymbol{x}\| \le \|\mathbf{A}\|\,\|\mathbf{A}^{-1}\|\,\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\| = \kappa(\mathbf{A})\,\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\|$. □

Note The condition number tells us the worst-case amplification factor. A relative perturbation of size ϵ in the data can produce a relative error up to $\kappa(\mathbf{A}) \cdot \epsilon$ in the solution. In double precision $\epsilon_{\text{mach}} \approx 10^{-16}$, so we can expect at most $\lfloor 16 - \log_{10}\kappa(\mathbf{A}) \rfloor$ correct decimal digits in the computed solution.

Example Let $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 10^{-6} \end{pmatrix}$, $\boldsymbol{b} = (1, 1)^T$. The exact solution is $\boldsymbol{x}^* = (1, 10^6)^T$ and $\kappa_2(\mathbf{A}) = 10^6$. A perturbation $\delta\boldsymbol{b} = (10^{-4}, 0)^T$ has $\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\| \approx 10^{-4}$. The bound predicts a relative solution error up to $\kappa(\mathbf{A}) \cdot 10^{-4} = 100$. Indeed, the perturbed solution is $\hat{\boldsymbol{x}} = (1 + 10^{-4}, 10^6)^T$, giving $\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|/\|\boldsymbol{x}^*\| \approx 10^{-10}$ — much smaller than the bound, since the perturbation was in the well-conditioned direction.

4.4.4 Conditioning vs. Stability

Def **Conditioning (Problem Property).** *Conditioning* is an intrinsic property of the *mathematical problem*: it measures the sensitivity of the exact solution to perturbations in the data. A matrix is *ill-conditioned* if $\kappa(\mathbf{A}) \gg 1$ and *well-conditioned* if $\kappa(\mathbf{A})$ is small.

Def **Numerical Stability (Algorithm Property).** *Numerical stability* is a property of the *algorithm*: it measures how well the algorithm avoids introducing

errors beyond those inherent to the problem. A backward stable algorithm introduces no more error than is forced by the problem's conditioning.

Note A well-conditioned problem solved by an unstable algorithm may yield inaccurate results. An ill-conditioned problem solved by a stable algorithm is still inherently sensitive — no algorithm can overcome poor conditioning. The best strategy is: use a numerically stable algorithm, and if $\kappa(\mathbf{A})$ is large, reformulate or precondition the problem before solving.

4.4.5 Stability of Matrix Factorizations

Prop 4.16 *Proposition 4.16* **(Stability of LU with Partial Pivoting).** Gaussian elimination with partial pivoting is backward stable with growth factor bounded by 2^{n-1} in the worst case. This worst-case bound is rarely achieved; for random dense matrices the expected growth factor is $O(n^{2/3})$.

Proof *Proof.* Partial pivoting ensures all multipliers satisfy $|m_{ij}| \leq 1$. At each elimination step, entries can at most double, giving a growth factor bound of 2^{n-1}. Worst-case matrices achieving this bound are known but are extremely rare in practice. For most scientific problems, GEPP is effectively backward stable. □

Prop 4.17 *Proposition 4.17* **(Stability of Cholesky Factorization).** For symmetric positive definite $\mathbf{A}$, Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ is backward stable without any pivoting. All pivot elements $\ell_{kk} = \sqrt{a_{kk} - \sum_{j<k} \ell_{kj}^2}$ remain positive and bounded by $\sqrt{\|\mathbf{A}\|_2}$, so no element growth occurs.

Proof *Proof.* For an SPD matrix, all principal submatrices are SPD (their Schur complements are positive definite), so each diagonal entry encountered during Cholesky is positive. Since no cancellation or sign change can occur, the factorization is inherently stable. A rounding error analysis shows the backward error satisfies $\|\mathbf{A} + \delta\mathbf{A} - \hat{\mathbf{L}}\hat{\mathbf{L}}^T\|_F \leq c_n \epsilon_{\text{mach}} \|\mathbf{A}\|_F$ for a modest constant c_n. □

Prop 4.18 *Proposition 4.18* **(Stability of QR via Householder Reflections).** QR decomposition using Householder reflections is backward stable with no growth factor. Each Householder reflector $\mathbf{H} = \mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\|\boldsymbol{v}\|^2$ is orthogonal with $\kappa_2(\mathbf{H}) = 1$, so orthogonal transformations cannot amplify errors.

Proof *Proof.* Each Householder reflection is an orthogonal transformation, so $\|\mathbf{H}\|_2 = 1$ and $\|\mathbf{H}^{-1}\|_2 = 1$. Round-off errors introduced at each step are not amplified by subsequent orthogonal transformations. A detailed analysis shows the overall backward error satisfies $\|(\hat{\mathbf{Q}} + \delta\mathbf{Q})(\hat{\mathbf{R}} + \delta\mathbf{R}) - \mathbf{A}\|_F = O(\epsilon_{\text{mach}}\|\mathbf{A}\|_F)$. □

Note **Hierarchy of stability:** SVD $\geq$ QR (Householder) $>$ Cholesky (SPD only) $\approx$ LU with partial pivoting $\gg$ LU without pivoting. SVD is the most stable but

also the most expensive. For general systems, GEPP is the standard; for SPD systems, Cholesky is preferred; for least squares or ill-conditioned problems, QR is recommended.

Def **Iterative Refinement.** After computing $\hat{\boldsymbol{x}}$ via a factorization, *iterative refinement* improves accuracy by: (1) compute the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$; (2) solve $\mathbf{A}\boldsymbol{\delta} = \boldsymbol{r}$ using the *same* factorization; (3) update $\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + \boldsymbol{\delta}$. Each cycle recovers roughly $\lfloor \log_{10}(1/\kappa(\mathbf{A})) \rfloor$ additional correct digits, up to the precision limit.

Def **Condition Number and Convergence of Iterative Methods.** For iterative methods, the condition number governs convergence rate. For a stationary splitting with $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$, the spectral radius $\rho(\mathbf{T})$ is close to 1 when $\mathbf{M}^{-1}\mathbf{A}$ has eigenvalues far from 1, which happens when $\kappa(\mathbf{A})$ is large and $\mathbf{M}$ is a poor approximation to $\mathbf{A}$. Preconditioning reduces the effective condition number $\kappa(\mathbf{M}^{-1}\mathbf{A})$ to accelerate convergence.

Ex 4.8 **(Condition number and solution sensitivity)** Let $\mathbf{A}_\epsilon = \begin{pmatrix} 1 & 0 \\ 0 & \epsilon \end{pmatrix}$, $\boldsymbol{b} = (1, 1)^T$.

1. Compute $\kappa_2(\mathbf{A}_\epsilon)$ for $\epsilon = 10^{-4}, 10^{-8}, 10^{-12}$.
2. The exact solution is $\boldsymbol{x}^* = (1, 1/\epsilon)^T$. Perturb $\boldsymbol{b}$ to $\tilde{\boldsymbol{b}} = (1 + 10^{-4}, 1)^T$. Compute the relative error $\|\tilde{\boldsymbol{x}} - \boldsymbol{x}^*\|/\|\boldsymbol{x}^*\|$ and compare it to the bound $\kappa_2(\mathbf{A}_\epsilon) \cdot \|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\|$.
3. Using NumPy, solve both systems for each ϵ. At which ϵ does the computed solution become unreliable in double precision?

Ex 4.9 **(Comparing factorization stability)** Build the $n \times n$ Hilbert matrix $H_{ij} = 1/(i+j-1)$ using `scipy.linalg.hilbert(n)`, which is famously ill-conditioned.

1. For $n = 10$: compute $\kappa_2(H)$ and predict how many correct digits to expect. Solve $H\boldsymbol{x} = H \cdot \mathbf{1}$ (exact solution $\boldsymbol{x}^* = \mathbf{1}$) via `np.linalg.solve` (LU) and `np.linalg.lstsq` (QR-based). Compare the errors.
2. Use the `mpmath` library with `mp.dps = 50` to solve the same system in high precision. Does the error decrease significantly? What does this tell you about conditioning vs. precision?
3. Apply one step of iterative refinement to the LU solution. How much improvement do you observe?

Ex 4.10 **(Backward error analysis in practice)** For the system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A}$ and $\boldsymbol{b}$ of your choice (e.g., a random 10×10 matrix scaled so $\kappa(\mathbf{A}) \approx 10^4$):

1. Compute $\hat{\boldsymbol{x}}$ via `np.linalg.solve`. Compute the backward error $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|/\|\boldsymbol{b}\|$. Is it consistent with $O(\epsilon_{\text{mach}})$?

2. Compute the forward error $\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|/\|\boldsymbol{x}^*\|$ (using a solution computed in high precision as ground truth). Is the ratio (forward error)/(backward error) consistent with $\kappa(\mathbf{A})$?

3. Repeat with a worse-conditioned matrix ($\kappa \approx 10^{12}$) and interpret the results.

5 Advanced Matrix Decompositions

5.1 Singular Value Decomposition and the Four Fundamental Subspaces

The SVD is often regarded as the culmination of linear algebra. It reveals that any matrix — regardless of shape or rank — can be decomposed into two orthogonal transformations and a diagonal scaling. This structure simultaneously provides orthonormal bases for all four fundamental subspaces, the optimal low-rank approximation, and the numerically stable pseudoinverse.

Thm 5.1 *Theorem 5.1* **(Singular Value Decomposition (SVD)).** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$, there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ and a rectangular diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ with diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$ such that

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T.$$

The values σ_i are the *singular values* of $\mathbf{A}$. The columns of $\mathbf{U}$ and $\mathbf{V}$ are the *left* and *right singular vectors*.

Proof *Proof.* We sketch the constructive proof. The matrix $\mathbf{A}^T\mathbf{A} \in \mathbb{R}^{n\times n}$ is symmetric positive semidefinite (since $\boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x} = \|\mathbf{A}\boldsymbol{x}\|_2^2 \geq 0$). By the spectral theorem, there exists an orthonormal eigenbasis: $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i$ with $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. Set $\sigma_i = \sqrt{\lambda_i}$ and, for $\sigma_i > 0$, define $\boldsymbol{u}_i = \mathbf{A}\boldsymbol{v}_i/\sigma_i$. Then $\{\boldsymbol{u}_i\}$ are orthonormal (since $\boldsymbol{u}_i^T\boldsymbol{u}_j = \boldsymbol{v}_i^T\mathbf{A}^T\mathbf{A}\boldsymbol{v}_j/(\sigma_i\sigma_j) = \lambda_j\delta_{ij}/(\sigma_i\sigma_j) = \delta_{ij}$). Extending $\{\boldsymbol{u}_i\}$ to an orthonormal basis of $\mathbb{R}^m$ gives $\mathbf{U}$. One verifies $\mathbf{A}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ column-by-column, hence $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. □

Note Geometrically, $\mathbf{A}$ acts as: (1) rotate/reflect the input space via $\mathbf{V}^T$; (2) scale along the coordinate axes (and possibly embed into a higher or lower dimensional space) via $\mathbf{\Sigma}$; (3) rotate/reflect the output space via $\mathbf{U}$. The outer product expansion $\mathbf{A} = \sum_{i=1}^r \sigma_i\boldsymbol{u}_i\boldsymbol{v}_i^T$ expresses $\mathbf{A}$ as a sum of rank-1 matrices, each capturing one "mode" of the transformation.

5.1.1 The Four Fundamental Subspaces

Def **Column Space (Range).** The *column space* of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is $\mathcal{R}(\mathbf{A}) = \{\mathbf{A}\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$. It is spanned by the columns of $\mathbf{A}$ and has dimension equal to the rank $r = \operatorname{rank}(\mathbf{A})$.

Def **Null Space (Kernel).** The *null space* of $\mathbf{A}$ is $\mathcal{N}(\mathbf{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$. It consists of all inputs mapped to zero and has dimension $n - r$ (the *nullity* of $\mathbf{A}$).

Def **Row Space.** The *row space* of $\mathbf{A}$ is $\mathcal{R}(\mathbf{A}^T) = \{\mathbf{A}^T\boldsymbol{y} : \boldsymbol{y} \in \mathbb{R}^m\} \subseteq \mathbb{R}^n$, spanned by the rows of $\mathbf{A}$. It has dimension r and is the orthogonal complement of $\mathcal{N}(\mathbf{A})$ in $\mathbb{R}^n$.

Def **Left Null Space.** The *left null space* of $\mathbf{A}$ is $\mathcal{N}(\mathbf{A}^T) = \{\boldsymbol{y} \in \mathbb{R}^m : \mathbf{A}^T\boldsymbol{y} = \mathbf{0}\}$. It has dimension $m - r$ and is the orthogonal complement of $\mathcal{R}(\mathbf{A})$ in $\mathbb{R}^m$.

Thm 5.2 *Theorem 5.2* **(Rank-Nullity via SVD).** $\operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}) = n$ and $\operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}^T) = m$.

Proof *Proof.* From the SVD $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$, the number of positive singular values equals $r = \operatorname{rank}(\mathbf{A})$. The right singular vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r$ (corresponding to $\sigma_i > 0$) span $\mathcal{R}(\mathbf{A}^T)$, while $\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n$ span $\mathcal{N}(\mathbf{A})$ (since $\mathbf{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i = \mathbf{0}$ for $i > r$). Since $\{\boldsymbol{v}_i\}$ is an orthonormal basis of $\mathbb{R}^n$, we have $r + (n - r) = n$. The second identity follows symmetrically from the left singular vectors. □

Thm 5.3 *Theorem 5.3* **(SVD Provides Orthonormal Bases for All Four Subspaces).** In the SVD $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ with $r = \operatorname{rank}(\mathbf{A})$: the first r columns of $\mathbf{U}$ span $\mathcal{R}(\mathbf{A})$; the last $m - r$ columns of $\mathbf{U}$ span $\mathcal{N}(\mathbf{A}^T)$; the first r columns of $\mathbf{V}$ span $\mathcal{R}(\mathbf{A}^T)$; the last $n - r$ columns of $\mathbf{V}$ span $\mathcal{N}(\mathbf{A})$.

Proof *Proof.* For any $\boldsymbol{x}$, $\mathbf{A}\boldsymbol{x} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T\boldsymbol{x} = \sum_{i=1}^r \sigma_i(\boldsymbol{v}_i^T\boldsymbol{x})\boldsymbol{u}_i$, which is a linear combination of $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$. Conversely, $\mathbf{A}\boldsymbol{v}_i = \sigma_i\boldsymbol{u}_i$ for $i \le r$, so each $\boldsymbol{u}_i$ ($i \le r$) is in $\mathcal{R}(\mathbf{A})$. Hence $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$. Since $\mathcal{N}(\mathbf{A}^T)$ is the orthogonal complement of $\mathcal{R}(\mathbf{A})$ in $\mathbb{R}^m$ and $\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m$ are orthogonal to $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$, they span $\mathcal{N}(\mathbf{A}^T)$. The arguments for $\mathcal{R}(\mathbf{A}^T)$ and $\mathcal{N}(\mathbf{A})$ are symmetric. □

Prop 5.4 *Proposition 5.4* **(Existence and Uniqueness of the Least Squares Solution).** The system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has an exact solution if and only if $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$. When it does, the general solution is any particular solution $\hat{\boldsymbol{x}}$ plus an arbitrary vector from $\mathcal{N}(\mathbf{A})$. The *minimum-norm* least squares solution is $\hat{\boldsymbol{x}} = \mathbf{A}^+\boldsymbol{b} = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T\boldsymbol{b}$, where $\boldsymbol{\Sigma}^+$ has $1/\sigma_i$ for each positive singular value.

Proof *Proof.* $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is solvable iff $\boldsymbol{b} \perp \mathcal{N}(\mathbf{A}^T)$, i.e., $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$ (since the two are orthogonal complements). The general solution is $\boldsymbol{x} = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T\boldsymbol{b} + \mathbf{P}_{\mathcal{N}(\mathbf{A})}\boldsymbol{w}$ for any $\boldsymbol{w}$, where the second term lies in $\mathcal{N}(\mathbf{A})$ and contributes zero to $\mathbf{A}\boldsymbol{x}$ but increases $\|\boldsymbol{x}\|_2$. The minimum-norm solution has zero $\mathcal{N}(\mathbf{A})$ component, giving $\hat{\boldsymbol{x}} = \mathbf{A}^+\boldsymbol{b}$. □

5.1.2 Rank Truncation and the Eckart-Young-Mirsky Theorem

Def **Rank-r Approximation.** The *rank-r truncated SVD* of $\mathbf{A}$ is

$$\mathbf{A}_r = \sum_{i=1}^r \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T = \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^T,$$

where $\mathbf{U}_r, \mathbf{V}_r$ are the first r columns of $\mathbf{U}, \mathbf{V}$ and $\boldsymbol{\Sigma}_r \in \mathbb{R}^{r \times r}$ contains the r largest singular values.

Thm 5.5

Theorem 5.5 **(Eckart-Young-Mirsky Theorem).** Among all matrices of rank at most r, $\mathbf{A}_r$ is the closest to $\mathbf{A}$ in both the spectral norm and the Frobenius norm:

$$\|\mathbf{A} - \mathbf{A}_r\|_2 = \sigma_{r+1}, \qquad \|\mathbf{A} - \mathbf{A}_r\|_F = \sqrt{\sum_{i=r+1}^{p} \sigma_i^2},$$

where $p = \min(m, n)$. For any $\mathbf{B}$ with $\operatorname{rank}(\mathbf{B}) \leq r$: $\|\mathbf{A} - \mathbf{B}\|_2 \geq \sigma_{r+1}$ and $\|\mathbf{A} - \mathbf{B}\|_F \geq \|\mathbf{A} - \mathbf{A}_r\|_F$.

Proof *Proof.* The error formula follows immediately: $\mathbf{A} - \mathbf{A}_r = \sum_{i=r+1}^{p} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$ is itself an SVD, so $\|\mathbf{A} - \mathbf{A}_r\|_2 = \sigma_{r+1}$ and $\|\mathbf{A} - \mathbf{A}_r\|_F = \sqrt{\sum_{i>r} \sigma_i^2}$.
For the lower bound: suppose $\operatorname{rank}(\mathbf{B}) \leq r$, so $\dim \mathcal{N}(\mathbf{B}) \geq n - r$. The subspace $\mathcal{N}(\mathbf{B}) \cap \operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{r+1}\}$ has dimension at least $(n - r) + (r + 1) - n = 1$ by dimension counting, so there exists a unit vector $\boldsymbol{z}$ in this intersection. Then

$$\|\mathbf{A} - \mathbf{B}\|_2 \geq \|(\mathbf{A} - \mathbf{B})\boldsymbol{z}\|_2 = \|\mathbf{A}\boldsymbol{z}\|_2 = \left\| \sum_{i=1}^{r+1} \sigma_i (\boldsymbol{v}_i^T \boldsymbol{z}) \boldsymbol{v}_i \right\|_2 \geq \sigma_{r+1} \|\boldsymbol{z}\|_2 = \sigma_{r+1}.$$

The Frobenius norm lower bound follows similarly using the Frobenius norm version of the argument. □

Thm 5.6 *Theorem 5.6* **(SVD-Eigendecomposition Relationship).** The singular values and vectors of $\mathbf{A}$ satisfy: the right singular vectors $\boldsymbol{v}_i$ are eigenvectors of $\mathbf{A}^T\mathbf{A}$ with eigenvalues σ_i^2; the left singular vectors $\boldsymbol{u}_i$ are eigenvectors of $\mathbf{A}\mathbf{A}^T$ with eigenvalues σ_i^2.

Proof *Proof.* From $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$: $\mathbf{A}^T\mathbf{A} = \mathbf{V}\boldsymbol{\Sigma}^T\mathbf{U}^T\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T = \mathbf{V}\boldsymbol{\Sigma}^T\boldsymbol{\Sigma}\mathbf{V}^T$. Since $\boldsymbol{\Sigma}^T\boldsymbol{\Sigma} = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_n^2)$ and $\mathbf{V}$ is orthogonal, this is the eigendecomposition $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \sigma_i^2 \boldsymbol{v}_i$. Symmetrically, $\mathbf{A}\mathbf{A}^T = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^T\mathbf{U}^T$ gives $\mathbf{A}\mathbf{A}^T\boldsymbol{u}_i = \sigma_i^2 \boldsymbol{u}_i$. □

Def **Pseudoinverse via SVD.** The *Moore-Penrose pseudoinverse* of $\mathbf{A} \in \mathbb{R}^{m \times n}$ is

$$\mathbf{A}^+ = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T, \quad \text{where } (\boldsymbol{\Sigma}^+)_{ii} = \begin{cases} 1/\sigma_i & \text{if } \sigma_i > 0 \\ 0 & \text{otherwise.} \end{cases}$$

For full column rank, $\mathbf{A}^+ = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$. For full row rank, $\mathbf{A}^+ = \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1}$.

Def **SVD-Based Regularization.** For an ill-conditioned system, the truncated SVD solution

$$\hat{\boldsymbol{x}}_k = \sum_{i=1}^{k} \frac{\boldsymbol{u}_i^T \boldsymbol{b}}{\sigma_i} \boldsymbol{v}_i$$

ignores directions corresponding to small singular values, reducing sensitivity to perturbations. The effective condition number of the regularized system is $\sigma_1/\sigma_k \ll \sigma_1/\sigma_n$.

Ex 5.1 Let $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$.

1. Compute the SVD of $\mathbf{A}$ by hand: find the eigenvalues of $\mathbf{A}^T\mathbf{A}$, then the singular values, right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$), and left singular vectors ($\boldsymbol{u}_i = \mathbf{A}\boldsymbol{v}_i/\sigma_i$).
2. Verify that $\mathbf{U}$, $\boldsymbol{\Sigma}$, $\mathbf{V}$ recover $\mathbf{A}$ via $\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$.
3. What are the four fundamental subspaces of $\mathbf{A}$? (Since $\mathbf{A}$ is 2×2 and invertible, what are their dimensions?)

Ex 5.2 **(Image compression via truncated SVD)** Load a grayscale image (e.g., `from skimage import data; A = data.camera().astype(float)`) and compute its SVD using `np.linalg.svd(A, full_matrices=False)`.

1. Plot the singular values on a log scale. Where do they start to level off?
2. Compute the rank-r approximation $\mathbf{A}_r$ for $r = 5, 20, 50, 100$. Display the images.
3. For each r, compute the storage savings ($(m+n+1) \times r$ vs. mn entries) and the relative Frobenius error $\|\mathbf{A} - \mathbf{A}_r\|_F / \|\mathbf{A}\|_F$. Plot error vs. compression ratio.

Ex 5.3 **(Four fundamental subspaces)** Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$.

1. Use `np.linalg.svd` to find the singular values. How many are zero (up to numerical precision)? What is $\operatorname{rank}(\mathbf{A})$?
2. Extract bases for all four fundamental subspaces from the SVD. Verify that $\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T)$ and $\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A})$.
3. For $\boldsymbol{b} = (1, 1, 1)^T$, determine whether the system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution. If not, compute the least squares solution using the pseudoinverse `np.linalg.pinv(A) @ b`.

5.2 Matrix Norms

Matrix norms assign a scalar "size" to a matrix. They are fundamental in numerical analysis: they measure approximation errors in low-rank truncation, bound how much a matrix amplifies vectors, and define the condition number. Every norm inequality we use in convergence analysis ultimately rests on a handful of key theorems proved here.

Def **Matrix Norm.** A function $\|\cdot\| : \mathbb{R}^{m\times n} \to \mathbb{R}$ is a *matrix norm* if for all $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$ and all $c \in \mathbb{R}$: (i) $\|\mathbf{A}\| \geq 0$ and $\|\mathbf{A}\| = 0 \Leftrightarrow \mathbf{A} = \mathbf{0}$ (non-negativity and definiteness); (ii) $\|c\mathbf{A}\| = |c|\|\mathbf{A}\|$ (absolute homogeneity); (iii) $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$ (triangle inequality). A square-matrix norm is additionally *submultiplicative* if $\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$.

Def **Induced (Operator) Norm.** Given vector norms $\|\cdot\|_u$ on $\mathbb{R}^m$ and $\|\cdot\|_v$ on $\mathbb{R}^n$, the *induced* (or *operator*) norm of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is

$$\|\mathbf{A}\|_{u,v} = \max_{\boldsymbol{x}\neq\mathbf{0}} \frac{\|\mathbf{A}\boldsymbol{x}\|_u}{\|\boldsymbol{x}\|_v} = \max_{\|\boldsymbol{x}\|_v=1} \|\mathbf{A}\boldsymbol{x}\|_u.$$

It measures the maximum stretching factor applied by $\mathbf{A}$.

Thm 5.7 *Theorem 5.7* **(Induced Norms are Submultiplicative).** Every induced norm satisfies $\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$.

Proof *Proof.* For any $\boldsymbol{x} \neq \mathbf{0}$,

$$\|\mathbf{AB}\boldsymbol{x}\| \leq \|\mathbf{A}\|\,\|\mathbf{B}\boldsymbol{x}\| \leq \|\mathbf{A}\|\,\|\mathbf{B}\|\,\|\boldsymbol{x}\|.$$

Dividing by $\|\boldsymbol{x}\|$ and taking the supremum over all $\boldsymbol{x} \neq \mathbf{0}$ gives $\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$. □

Def L_1 **Norm (Maximum Absolute Column Sum).** The matrix L_1 norm, induced by the vector ℓ^1 norm, is

$$\|\mathbf{A}\|_1 = \max_{1\leq j\leq n} \sum_{i=1}^{m} |a_{ij}|.$$

It equals the maximum absolute column sum of $\mathbf{A}$.

Def L_∞**-Norm (Maximum Absolute Row Sum).** The matrix L_∞ norm, induced by the vector ℓ^∞ norm, is

$$\|\mathbf{A}\|_\infty = \max_{1\leq i\leq m} \sum_{j=1}^{n} |a_{ij}|.$$

It equals the maximum absolute row sum of $\mathbf{A}$.

Def **Spectral Norm.** The matrix L_2 norm (spectral norm), induced by the vector ℓ^2 norm, is

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}.$$

It equals the largest singular value and measures the maximum Euclidean stretching factor of $\mathbf{A}$.

Thm 5.8 *Theorem 5.8* **(Spectral Norm via Singular Values).** $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$.

Proof *Proof.* Write the SVD as $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$. For $\|\boldsymbol{x}\|_2 = 1$, let $\boldsymbol{y} = \mathbf{V}^T\boldsymbol{x}$ (which also satisfies $\|\boldsymbol{y}\|_2 = 1$, since $\mathbf{V}$ is orthogonal). Then

$$\|\mathbf{A}\boldsymbol{x}\|_2 = \|\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T\boldsymbol{x}\|_2 = \|\boldsymbol{\Sigma}\boldsymbol{y}\|_2 \leq \sigma_1\|\boldsymbol{y}\|_2 = \sigma_1,$$

with equality when $\boldsymbol{y} = \mathbf{e}_1$, i.e., $\boldsymbol{x} = \boldsymbol{v}_1$ (the first right singular vector). Hence $\|\mathbf{A}\|_2 = \sigma_1 = \sigma_{\max}(\mathbf{A})$. □

Def **Frobenius Norm.** The *Frobenius norm* treats $\mathbf{A}$ as a vector in $\mathbb{R}^{mn}$:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2}.$$

Thm 5.9 *Theorem 5.9* **(Frobenius Norm via Trace and Singular Values).** $\|\mathbf{A}\|_F = \sqrt{\mathrm{Tr}(\mathbf{A}^T\mathbf{A})} = \sqrt{\mathrm{Tr}(\mathbf{A}\mathbf{A}^T)} = \sqrt{\sum_k \sigma_k^2}$.

Proof *Proof.* By definition, $\|\mathbf{A}\|_F^2 = \sum_{i,j} a_{ij}^2 = \sum_j (\mathbf{A}^T\mathbf{A})_{jj} = \mathrm{Tr}(\mathbf{A}^T\mathbf{A})$. The matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive semidefinite with eigenvalues $\sigma_1^2 \geq \cdots \geq \sigma_n^2 \geq 0$ (the squared singular values of $\mathbf{A}$), so $\mathrm{Tr}(\mathbf{A}^T\mathbf{A}) = \sum_k \sigma_k^2$. The identity $\mathrm{Tr}(\mathbf{A}^T\mathbf{A}) = \mathrm{Tr}(\mathbf{A}\mathbf{A}^T)$ follows since both equal $\sum_{i,j} a_{ij}^2$. □

Note The Frobenius norm is submultiplicative and is consistent with the vector ℓ^2 norm: $\|\mathbf{A}\boldsymbol{x}\|_2 \leq \|\mathbf{A}\|_F\|\boldsymbol{x}\|_2$. It is *not* induced by any vector norm, but satisfies $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{r}\,\|\mathbf{A}\|_2$ where $r = \mathrm{rank}(\mathbf{A})$.

Def **Max Norm.** The *max norm* is $\|\mathbf{A}\|_{\max} = \max_{i,j} |a_{ij}|$. It is a valid matrix norm but is *not* submultiplicative in general ($\|\mathbf{AB}\|_{\max}$ can exceed $\|\mathbf{A}\|_{\max}\|\mathbf{B}\|_{\max}$) and is not an induced norm.

Thm 5.10 *Theorem 5.10* **(Norm Equivalence in Finite Dimensions).** All matrix norms on $\mathbb{R}^{m\times n}$ are equivalent: for any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$, there exist constants $c_1, c_2 > 0$ such that $c_1\|\mathbf{A}\|_\beta \leq \|\mathbf{A}\|_\alpha \leq c_2\|\mathbf{A}\|_\beta$ for all $\mathbf{A}$.

Proof *Proof.* Since $\mathbb{R}^{m\times n}$ is a finite-dimensional vector space (of dimension mn), any norm on it is equivalent to the standard Euclidean norm on $\mathbb{R}^{mn}$ (which equals the Frobenius norm). Specifically, the unit ball of any norm is compact, so the continuous function $\mathbf{A} \mapsto \|\mathbf{A}\|_\alpha/\|\mathbf{A}\|_\beta$ attains its supremum and infimum on the unit sphere $\{\mathbf{A} : \|\mathbf{A}\|_\beta = 1\}$, yielding the constants c_1 and c_2. □

Prop 5.11 *Proposition 5.11* **(Consistency with Vector Norms).** An induced norm $\|\mathbf{A}\|$ satisfies $\|\mathbf{A}\boldsymbol{x}\|_v \leq \|\mathbf{A}\|\|\boldsymbol{x}\|_v$ for all $\boldsymbol{x}$ by definition. The Frobenius norm also satisfies $\|\mathbf{A}\boldsymbol{x}\|_2 \leq \|\mathbf{A}\|_F\|\boldsymbol{x}\|_2$.

Proof *Proof.* For the Frobenius norm, use Cauchy-Schwarz: $(\mathbf{A}\boldsymbol{x})_i = \sum_j a_{ij}x_j$, so $|(\mathbf{A}\boldsymbol{x})_i|^2 \leq \sum_j a_{ij}^2 \cdot \|\boldsymbol{x}\|_2^2$. Summing over i gives $\|\mathbf{A}\boldsymbol{x}\|_2^2 \leq \|\mathbf{A}\|_F^2\|\boldsymbol{x}\|_2^2$. □

Note Matrix norms appear in the condition number $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$. For the 2-norm this gives $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$. For the Frobenius norm, $\|\mathbf{A}\|_F$ prominently features in the Eckart-Young-Mirsky theorem for optimal low-rank approximation.

Example Let $\mathbf{A} = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$.

- $\|\mathbf{A}\|_1 = \max(1+3,\, 2+4) = 6$ (maximum column sum).
- $\|\mathbf{A}\|_\infty = \max(1+2,\, 3+4) = 7$ (maximum row sum).
- $\|\mathbf{A}\|_F = \sqrt{1+4+9+16} = \sqrt{30} \approx 5.48$.
- $\|\mathbf{A}\|_2 = \sigma_{\max}$: eigenvalues of $\mathbf{A}^T\mathbf{A}$ satisfy $\lambda^2 - 30\lambda + 4 = 0$, giving $\lambda_{\max} = 15 + \sqrt{221}$, so $\|\mathbf{A}\|_2 \approx 5.46$.

Note $\|\mathbf{A}\|_2 \le \|\mathbf{A}\|_F$ as guaranteed by the theorem.

Ex 5.4

1. Prove that $\|\mathbf{A}\|_2 \le \|\mathbf{A}\|_F \le \sqrt{r}\,\|\mathbf{A}\|_2$ where $r = \operatorname{rank}(\mathbf{A})$. *Hint: use the singular value representation of both norms.*
2. For $n \times n$ matrices, prove $\|\mathbf{A}\|_2 \le \sqrt{n}\|\mathbf{A}\|_\infty$ and $\|\mathbf{A}\|_\infty \le \sqrt{n}\|\mathbf{A}\|_2$.
3. Using NumPy, verify these inequalities for 1000 random 5×5 matrices drawn from `np.random.randn`. Plot histograms of the ratios $\|\mathbf{A}\|_F/\|\mathbf{A}\|_2$ and $\|\mathbf{A}\|_\infty/\|\mathbf{A}\|_2$.

Ex 5.5 **(Max norm is not submultiplicative)** Find a pair of 2×2 matrices $\mathbf{A}, \mathbf{B}$ such that $\|\mathbf{AB}\|_{\max} > \|\mathbf{A}\|_{\max}\|\mathbf{B}\|_{\max}$. Conclude that the max norm cannot be an induced norm.

5.3 Overdetermined Systems and Least Squares

So far we have focused on square invertible systems $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. In practice, data fitting and regression produce overdetermined systems ($m > n$ equations, n unknowns) that typically have no exact solution. The method of least squares provides the principled approach: find the $\boldsymbol{x}$ minimizing $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2$. The geometry, optimality conditions, and numerically stable algorithms for this problem form the core of this section.

5.3.1 Overdetermined Systems

Def **Overdetermined Linear System.** A system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A} \in \mathbb{R}^{m\times n}$, $m > n$, is *overdetermined*. Since the range of $\mathbf{A}$ has dimension at most $n < m$, a generic $\boldsymbol{b} \in \mathbb{R}^m$ lies outside $\mathcal{R}(\mathbf{A})$ and no exact solution exists.

Def **Least Squares Problem.** Given $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$, the *least squares problem* is

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathbb{R}^n} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2.$$

The minimizer $\hat{\boldsymbol{x}}$ makes $\mathbf{A}\hat{\boldsymbol{x}}$ the closest point in $\mathcal{R}(\mathbf{A})$ to $\boldsymbol{b}$ in the Euclidean norm.

Note Overdetermined systems arise throughout science: fitting a model with n parameters to $m \gg n$ measurements, regression in statistics, signal denoising, and computer vision. The key insight is that the least squares solution corresponds to the orthogonal projection of $\boldsymbol{b}$ onto the column space of $\mathbf{A}$.

5.3.2 Normal Equations

Thm 5.12 *Theorem 5.12* **(Normal Equations).** The vector $\hat{\boldsymbol{x}}$ minimizes $\|\mathbf{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2$ if and only if it satisfies the *normal equations*

$$\mathbf{A}^T\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}.$$

If $\mathbf{A}$ has full column rank, the unique least squares solution is $\hat{\boldsymbol{x}} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\boldsymbol{b}$.

Proof *Proof.* Let $f(\boldsymbol{x}) = \|\mathbf{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 = \boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x}-2\boldsymbol{b}^T\mathbf{A}\boldsymbol{x}+\boldsymbol{b}^T\boldsymbol{b}$. Since $\mathbf{A}^T\mathbf{A}$ is symmetric, the gradient is $\nabla f = 2\mathbf{A}^T\mathbf{A}\boldsymbol{x} - 2\mathbf{A}^T\boldsymbol{b}$. Setting $\nabla f = \mathbf{0}$ gives $\mathbf{A}^T\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}$. The Hessian is $2\mathbf{A}^T\mathbf{A} \succeq 0$, so this critical point is a global minimum; it is unique when $\mathbf{A}^T\mathbf{A} \succ 0$, i.e. when $\mathbf{A}$ has full column rank. *Geometric view:* $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ must satisfy $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ (residual is orthogonal to each column of $\mathbf{A}$), which is exactly the normal equations. □

Note The name "normal equations" comes from the fact that the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ is *normal* (perpendicular) to the column space of $\mathbf{A}$. This orthogonality principle $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ is fundamental to least squares theory.

Thm 5.13 *Theorem 5.13* **(Condition Number Squaring).** If $\mathbf{A}$ has full column rank, then $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$.

Proof *Proof.* The singular values of $\mathbf{A}$ are $\sigma_1 \geq \cdots \geq \sigma_n > 0$. The matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite with eigenvalues equal to the squared singular values

$\sigma_1^2 \geq \cdots \geq \sigma_n^2 > 0$ (since $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \sigma_i^2\boldsymbol{v}_i$ where $\boldsymbol{v}_i$ are right singular vectors). Therefore

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = \frac{\sigma_1^2}{\sigma_n^2} = \left(\frac{\sigma_1}{\sigma_n}\right)^2 = \kappa_2(\mathbf{A})^2.$$

□

Note Condition number squaring is a serious practical concern. If $\kappa_2(\mathbf{A}) = 10^6$ (a moderately ill-conditioned design matrix), then $\kappa_2(\mathbf{A}^T\mathbf{A}) = 10^{12}$, and we lose 12 decimal digits of accuracy in the normal equations solution. This motivates QR-based methods that avoid forming $\mathbf{A}^T\mathbf{A}$ entirely.

Example Fitting a line $y = mx+b$ to four data points $(1,2),(2,3),(3,5),(4,4)$. The design matrix is

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 3 \\ 5 \\ 4 \end{pmatrix}, \quad \mathbf{A}^T\mathbf{A} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}, \quad \mathbf{A}^T\boldsymbol{b} = \begin{pmatrix} 14 \\ 41 \end{pmatrix}.$$

Solving $(\mathbf{A}^T\mathbf{A})\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}$ gives $\hat{b} = 1.3$ and $\hat{m} = 0.9$, so the best-fit line is $y = 0.9x + 1.3$.

5.3.3 QR-Based Least Squares

Thm 5.14 *Theorem 5.14* **(Least Squares via QR Decomposition).** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ ($m \geq n$) have full column rank with thin QR decomposition $\mathbf{A} = \mathbf{QR}$ ($\mathbf{Q} \in \mathbb{R}^{m\times n}$ with orthonormal columns, $\mathbf{R} \in \mathbb{R}^{n\times n}$ upper triangular and invertible). Then the unique least squares solution is found by solving the triangular system

$$\mathbf{R}\hat{\boldsymbol{x}} = \mathbf{Q}^T\boldsymbol{b}.$$

Proof *Proof.* Since $\mathbf{Q}$ has orthonormal columns, $\|\mathbf{Q}\boldsymbol{v}\|_2 = \|\boldsymbol{v}\|_2$ for any $\boldsymbol{v}$. Using $\mathbf{A} = \mathbf{QR}$, write $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2 = \|\mathbf{QR}\boldsymbol{x} - \boldsymbol{b}\|_2$. Split $\boldsymbol{b} = \mathbf{QQ}^T\boldsymbol{b} + (\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}$ into its projection onto $\mathcal{R}(\mathbf{A})$ and orthogonal complement. Expanding and using orthogonality of the two components:

$$\|\mathbf{QR}\boldsymbol{x} - \boldsymbol{b}\|_2^2 = \|\mathbf{R}\boldsymbol{x} - \mathbf{Q}^T\boldsymbol{b}\|_2^2 + \|(\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}\|_2^2.$$

The second term is independent of $\boldsymbol{x}$, so minimizing over $\boldsymbol{x}$ is equivalent to solving $\mathbf{R}\hat{\boldsymbol{x}} = \mathbf{Q}^T\boldsymbol{b}$, which has unique solution $\hat{\boldsymbol{x}} = \mathbf{R}^{-1}\mathbf{Q}^T\boldsymbol{b}$ since $\mathbf{R}$ is invertible. □

Note The QR approach avoids forming $\mathbf{A}^T\mathbf{A}$ and thus does not suffer from condition number squaring. Its cost is $O(mn^2)$ for the QR factorization plus $O(n^2)$ for the triangular solve—comparable to the normal equations but numerically far

superior. In NumPy, `np.linalg.lstsq(A, b)` uses this approach internally (QR or SVD depending on shape and rank).

Def **Weighted Least Squares.** If measurements have unequal reliability, one assigns a weight $w_i > 0$ to the i-th equation. The *weighted least squares* problem minimizes $\sum_i w_i((\mathbf{A}\boldsymbol{x})_i - b_i)^2 = \|\mathbf{W}^{1/2}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})\|_2^2$ where $\mathbf{W} = \operatorname{diag}(w_1, \ldots, w_m)$. Its normal equations are $(\mathbf{A}^T\mathbf{W}\mathbf{A})\hat{\boldsymbol{x}} = \mathbf{A}^T\mathbf{W}\boldsymbol{b}$.

Def **Regularized Least Squares (Ridge Regression).** When $\mathbf{A}$ is rank-deficient or nearly so, the least squares solution can have enormous norm. *Tikhonov regularization* (ridge regression) adds a penalty: $\hat{\boldsymbol{x}}_\lambda = \arg\min_{\boldsymbol{x}}(\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda\|\boldsymbol{x}\|_2^2) = (\mathbf{A}^T\mathbf{A} + \lambda\mathbf{I})^{-1}\mathbf{A}^T\boldsymbol{b}$. The parameter $\lambda > 0$ reduces the effective condition number to $(\sigma_1^2 + \lambda)/(\sigma_n^2 + \lambda)$.

Note The least squares framework extends to many important problems: nonlinear least squares fits nonlinear models by iteratively linearizing (Gauss-Newton/Levenberg-Marquardt); sparse regression (LASSO) replaces the ℓ^2 penalty by an ℓ^1 penalty to promote sparsity; total least squares accounts for errors in both $\mathbf{A}$ and $\boldsymbol{b}$.

Ex 5.6 **(Polynomial regression)** Fit a degree-2 polynomial $p(t) = c_0 + c_1 t + c_2 t^2$ to the data (t_i, y_i) for $i = 1, \ldots, 6$: $t = (0, 0.5, 1, 1.5, 2, 2.5)$, $y = (1.0, 1.8, 2.3, 2.5, 2.3, 2.0)$.

1. Write down the Vandermonde design matrix $\mathbf{A} \in \mathbb{R}^{6\times 3}$ explicitly.
2. Solve the normal equations by hand (or with NumPy). State the fitted polynomial.
3. Compute the residual $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|_2$ and the coefficient of determination $R^2 = 1 - \|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|_2^2 / \|\boldsymbol{b} - \bar{b}\mathbf{1}\|_2^2$.
4. Now solve via QR using `np.linalg.lstsq` and verify you get the same coefficients. Compare $\kappa_2(\mathbf{A})$ with $\kappa_2(\mathbf{A}^T\mathbf{A})$ to illustrate condition number squaring.

Ex 5.7 **(Effect of regularization)** Use the Vandermonde design matrix for fitting a degree-8 polynomial to 10 equally spaced points on $[0, 1]$ with $y_i = \sin(2\pi t_i) + 0.1\varepsilon_i$ (small noise).

1. Compute the least squares fit without regularization. Plot the fitted polynomial. Does it overfit?
2. Compute $\hat{\boldsymbol{x}}_\lambda$ for $\lambda = 10^{-4}, 10^{-2}, 1$. How does the fit change?
3. Plot $\|\hat{\boldsymbol{x}}_\lambda\|_2$ and the residual $\|\mathbf{A}\hat{\boldsymbol{x}}_\lambda - \boldsymbol{b}\|_2$ as functions of λ (the bias-variance tradeoff curve). Which λ gives the best fit?

Ex 5.16 **(Numerical stability of normal equations vs. QR)** Construct the $m \times n$ matrix $\mathbf{A}$ from `np.random.randn(m, n)` scaled so $\kappa_2(\mathbf{A}) \approx 10^6$ (use SVD to rescale singular values). Let $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}_{\text{true}} + \boldsymbol{\varepsilon}$ with $\boldsymbol{x}_{\text{true}} = \mathbf{1}$ and small noise.

1. Solve via normal equations: `np.linalg.solve(A.T @ A, A.T @ b)`.
2. Solve via QR: `np.linalg.lstsq(A, b)`.
3. Compare the errors $\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{\text{true}}\|_2$. Which method is more accurate? Explain in terms of condition numbers.

6 Applications of Numerical Linear Algebra

6.1 Principal Component Analysis via SVD

Principal Component Analysis (PCA) is a fundamental technique for dimensionality reduction. It seeks an orthogonal change of basis that aligns the coordinate axes with the directions of maximum variance in the data. Via the SVD, PCA is both theoretically elegant — it is the optimal linear dimensionality reduction — and computationally efficient. Understanding PCA requires understanding orthogonal projections, eigendecompositions, and the Eckart-Young theorem.

Def **Data Centering.** For PCA, the data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n samples, p features) must be *centered*: subtract the column mean $\bar{x}_j = \frac{1}{n}\sum_i X_{ij}$ from each entry in column j. Throughout this section, $\mathbf{X}$ denotes a centered data matrix.

Def **Sample Covariance Matrix.** For a centered data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, the *sample covariance matrix* is

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{p \times p}.$$

$\mathbf{S}$ is symmetric and positive semidefinite. Its diagonal entry S_{jj} is the sample variance of feature j, and S_{ij} is the sample covariance between features i and j.

6.1.1 Principal Components as Variance-Maximizing Directions

Thm 6.1 *Theorem 6.1* **(First Principal Component Maximizes Variance).** Among all unit vectors $\boldsymbol{w} \in \mathbb{R}^p$, the direction that maximizes the variance of the projected data $\mathbf{X}\boldsymbol{w}$ is the eigenvector $\boldsymbol{v}_1$ of $\mathbf{S}$ corresponding to the largest eigenvalue λ_1.

Proof *Proof.* The sample variance of $\mathbf{X}\boldsymbol{w}$ is

$$\text{Var}(\mathbf{X}\boldsymbol{w}) = \frac{1}{n-1}\|\mathbf{X}\boldsymbol{w}\|_2^2 = \boldsymbol{w}^T\mathbf{S}\boldsymbol{w}.$$

We maximize $\boldsymbol{w}^T\mathbf{S}\boldsymbol{w}$ subject to $\|\boldsymbol{w}\|_2 = 1$. By the Rayleigh quotient theorem (a consequence of the spectral theorem for symmetric matrices), this maximum is λ_1, attained at $\boldsymbol{w} = \boldsymbol{v}_1$ (the eigenvector of $\mathbf{S}$ for λ_1). To verify: $\boldsymbol{v}_1^T\mathbf{S}\boldsymbol{v}_1 = \lambda_1\boldsymbol{v}_1^T\boldsymbol{v}_1 = \lambda_1$, and for any unit $\boldsymbol{w} = \sum_i c_i\boldsymbol{v}_i$, $\boldsymbol{w}^T\mathbf{S}\boldsymbol{w} = \sum_i \lambda_i c_i^2 \leq \lambda_1 \sum_i c_i^2 = \lambda_1$. □

Def **Principal Directions (Loading Vectors).** The *principal directions* $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_p$ are the eigenvectors of $\mathbf{S}$ ordered by decreasing eigenvalue $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$. The k-th principal direction $\boldsymbol{v}_k$ maximizes variance among all unit vectors orthogonal to $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}$.

Def **Principal Component Scores.** The *principal component scores* are the projections of the data onto the principal directions:

$$\mathbf{Z} = \mathbf{X}\mathbf{V}, \qquad \mathbf{V} = [\boldsymbol{v}_1 \mid \cdots \mid \boldsymbol{v}_p].$$

The k-th column $\boldsymbol{z}_k = \mathbf{X}\boldsymbol{v}_k$ is the k-th principal component. Its sample variance is λ_k.

Thm 6.2 *Theorem 6.2* **(Principal Components are Uncorrelated).** The principal component scores are pairwise uncorrelated: $\mathrm{Cov}(\boldsymbol{z}_j, \boldsymbol{z}_k) = 0$ for $j \neq k$.

Proof *Proof.* The sample covariance matrix of the scores $\mathbf{Z} = \mathbf{X}\mathbf{V}$ is

$$\frac{1}{n-1}\mathbf{Z}^T\mathbf{Z} = \frac{1}{n-1}\mathbf{V}^T\mathbf{X}^T\mathbf{X}\mathbf{V} = \mathbf{V}^T\mathbf{S}\mathbf{V} = \boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_p),$$

where the last equality uses $\mathbf{S}\mathbf{V} = \mathbf{V}\boldsymbol{\Lambda}$ (eigendecomposition) and $\mathbf{V}^T\mathbf{V} = \mathbf{I}$ (orthogonality). The off-diagonal entries of $\boldsymbol{\Lambda}$ are zero, confirming the principal components are uncorrelated. □

6.1.2 The SVD-PCA Connection

Thm 6.3 *Theorem 6.3* **(PCA via SVD).** Let $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ be the SVD of the centered data matrix. Then: the columns of $\mathbf{V}$ are the principal directions (eigenvectors of $\mathbf{S}$); the eigenvalues of $\mathbf{S}$ satisfy $\lambda_i = \sigma_i^2/(n-1)$; the principal component scores satisfy $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\boldsymbol{\Sigma}$.

Proof *Proof.* Substitute the SVD into the covariance matrix:

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} = \frac{1}{n-1}(\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T)^T(\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T) = \frac{1}{n-1}\mathbf{V}\boldsymbol{\Sigma}^T(\mathbf{U}^T\mathbf{U})\boldsymbol{\Sigma}\mathbf{V}^T = \frac{1}{n-1}\mathbf{V}\boldsymbol{\Sigma}^2\mathbf{V}^T,$$

using $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ (orthogonality of $\mathbf{U}$) and $\boldsymbol{\Sigma}^T\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^2 = \mathrm{diag}(\sigma_1^2, \ldots)$. Comparing with $\mathbf{S} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T$ identifies the columns of $\mathbf{V}$ as eigenvectors and $\lambda_i = \sigma_i^2/(n-1)$. For the scores: $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T\mathbf{V} = \mathbf{U}\boldsymbol{\Sigma}$. □

Note This theorem shows that PCA can be performed directly from the SVD of $\mathbf{X}$, avoiding the explicit formation of $\mathbf{S}$. Since $\mathbf{S} = \mathbf{X}^T\mathbf{X}/(n-1)$ squares the condition number, the SVD route is numerically more stable. In NumPy: `U, s, Vt = np.linalg.svd(X, full_matrices=False); Z = U * s`.

6.1.3 Dimensionality Reduction

Def **Rank-k Approximation via PCA.** The best rank-k approximation of $\mathbf{X}$ using PCA retains only the first k principal components:

$$\mathbf{X}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T = \mathbf{Z}_k\mathbf{V}_k^T = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^T,$$

where $\mathbf{U}_k, \mathbf{V}_k$ are the first k columns of $\mathbf{U}, \mathbf{V}$ and $\mathbf{\Sigma}_k = \mathrm{diag}(\sigma_1, \ldots, \sigma_k)$.

Thm 6.4 *Theorem 6.4* **(PCA is the Optimal Linear Dimensionality Reduction).** Among all rank-k approximations, $\mathbf{X}_k$ minimizes the reconstruction error:

$$\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2 = (n-1) \sum_{i=k+1}^{r} \lambda_i,$$

and $\mathbf{X}_k$ is the best rank-k matrix approximation to $\mathbf{X}$ in the Frobenius norm.

Proof *Proof.* This is a direct consequence of the Eckart-Young-Mirsky theorem applied to $\mathbf{X}$: $\mathbf{X}_k$ is the best rank-k approximation and the error is $\|\mathbf{X}-\mathbf{X}_k\|_F^2 = \sum_{i>k} \sigma_i^2$. Converting via $\lambda_i = \sigma_i^2/(n-1)$ gives the formula in terms of eigenvalues. □

Def **Proportion of Variance Explained.** The *proportion of variance explained* (PVE) by the first k principal components is

$$\mathrm{PVE}_k = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{p} \lambda_i} = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{p} \sigma_i^2}.$$

A common criterion is to choose k such that $\mathrm{PVE}_k \geq 0.90$ or 0.95.

Def **Scree Plot.** A *scree plot* displays the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p$ (or singular values σ_i) in decreasing order. The "elbow" — the point where the values level off — suggests a natural cutoff for k. Components beyond the elbow likely represent noise rather than signal.

Note The choice of k should also be guided by downstream tasks: visualization typically uses $k = 2$ or $k = 3$; clustering may need more components; the goal is to retain enough variance for the application while discarding noisy dimensions.

6.1.4 Statistical Interpretation

Def

Loadings. The entries of the principal directions $\boldsymbol{v}_i$ are called *loadings*. The loading v_{ji} quantifies the contribution of the j-th original feature to the i-th principal component. When data are standardized (unit-variance features), v_{ji} equals the correlation between feature j and PC score $\boldsymbol{z}_i$.

High-Dimensional PCA ($p \gg n$). When $p \gg n$, forming the $p \times p$ covariance matrix $\mathbf{S}$ is prohibitively expensive. Instead, one works with the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. If $\boldsymbol{u}_i$ is an eigenvector of $\mathbf{K}$ with eigenvalue σ_i^2, then $\boldsymbol{v}_i = \mathbf{X}^T\boldsymbol{u}_i/\sigma_i$ is the corresponding principal direction. This kernel trick has complexity $O(n^2p + n^3)$ vs. $O(np^2 + p^3)$ for the direct approach.

Note **Limitations of PCA.** PCA assumes linear structure: it finds directions of high variance in the *original* feature space. If the data lies on a nonlinear manifold (e.g., a sphere), PCA may fail to capture its intrinsic geometry (Kernel PCA addresses this).
PCA is sensitive to outliers, since they strongly influence the variance and hence the principal directions.
PCA is unsupervised: it maximizes variance regardless of class labels. In classification tasks, directions that best separate classes may have low variance and be discarded by PCA (Linear Discriminant Analysis is supervised and addresses this).
The effectiveness of PCA depends on whether high-variance directions are truly informative. Rapid decay of singular values is a reliable indicator that PCA will achieve effective compression.

Example Consider a centered data matrix $\mathbf{X} \in \mathbb{R}^{100\times 3}$ with singular values $\sigma_1 = 10\sqrt{99}$, $\sigma_2 = 5\sqrt{99}$, $\sigma_3 = 0.5\sqrt{99}$. The covariance eigenvalues are $\lambda_1 = 100$, $\lambda_2 = 25$, $\lambda_3 = 0.25$.

- PVE for $k = 1$: $100/125.25 \approx 79.8\%$.
- PVE for $k = 2$: $125/125.25 \approx 99.8\%$.

The data is nearly coplanar: 99.8% of variance lies in a 2D subspace. A rank-2 approximation $\mathbf{X}_2$ has reconstruction error $\|\mathbf{X} - \mathbf{X}_2\|_F = |\sigma_3| = 0.5\sqrt{99} \approx 4.97$.

Note The SVD $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ gives a geometric picture: $\mathbf{V}$ rotates the feature space to align the axes with principal directions; $\boldsymbol{\Sigma}$ reveals the variance captured by each direction; $\mathbf{U}$ contains the normalized principal component scores. The transformation $\mathbf{X} \mapsto \mathbf{X}\mathbf{V} = \mathbf{U}\boldsymbol{\Sigma}$ is simply a rigid rotation in feature space followed by reading off coordinates in the new basis.

Ex 6.1 **(PCA by hand)** Let $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ (already centered: verify $\sum_i X_{i1} = \sum_i X_{i2} = 0$).

1. Compute $\mathbf{S} = \mathbf{X}^T\mathbf{X}/(n-1)$ with $n = 3$.

2. Find the eigenvalues and eigenvectors of $\mathbf{S}$. What are λ_1, λ_2 and the principal directions $\boldsymbol{v}_1, \boldsymbol{v}_2$?

3. Compute the principal component scores $\mathbf{Z} = \mathbf{X}\mathbf{V}$. Verify that $\mathbf{Z}^T\mathbf{Z}/(n-1)$ is diagonal.

4. Compute the SVD of $\mathbf{X}$ and verify that the right singular vectors match the principal directions from step 2.

Ex 6.2 **(PCA for dimensionality reduction)** Load the Iris dataset from `sklearn.datasets.load_ir` The data matrix $\mathbf{X}$ has $n = 150$ samples and $p = 4$ features.

1. Center the data. Compute the SVD and plot the first two principal components as a scatter plot colored by species. Do the classes separate?

2. Plot the scree plot (singular values vs. index). How many components capture 95% of the variance?

3. Compute the loadings $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$. Which original features contribute most to each principal component? What is the biological interpretation?

4. Reconstruct the data from rank-k approximation for $k = 1, 2, 3, 4$. Plot the relative reconstruction error $\|\mathbf{X} - \mathbf{X}_k\|_F / \|\mathbf{X}\|_F$ vs. k.

Ex 6.3 **(High-dimensional PCA)** Generate a data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$ with $n = 50, p = 500$ from `np.random.randn`.

1. Compute the $p \times p$ covariance matrix $\mathbf{S} = \mathbf{X}^T\mathbf{X}/49$ and time its eigendecomposition. How long does it take?

2. Compute the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T/49$ and time its eigendecomposition. Which is faster?

3. Recover the principal directions $\boldsymbol{v}_i = \mathbf{X}^T\boldsymbol{u}_i/\sigma_i$ from the Gram matrix eigenvectors. Verify these match the top eigenvectors of $\mathbf{S}$ (up to sign).

4. For $n \ll p$, how many nonzero eigenvalues does $\mathbf{S}$ have? Explain using the rank-nullity theorem.

6.2 Spectral Graph Theory

Spectral graph theory studies graph properties through the eigenvalues and eigenvectors of their associated matrices. This approach bridges discrete graph structures with continuous spectral analysis, enabling powerful insights into connectivity, partitioning, diffusion, and community detection.

Def **Graph Representation.** A graph $G = (V, E)$ consists of a vertex set V with $n = |V|$ vertices and an edge set E with $m = |E|$ edges. For an undirected graph, each edge is an unordered pair (i, j) with $i, j \in V$. A weighted graph assigns a positive weight $w_{ij} > 0$ to each edge $(i, j) \in E$. The degree d_i of vertex i is defined as $d_i = \sum_{j:(i,j)\in E} w_{ij}$, which reduces to the number of adjacent vertices in the unweighted case.

Def **Matrix Representations.** A graph can be represented by several matrices:

- **Adjacency Matrix (A)**: For unweighted graphs, $A_{ij} = 1$ if $(i, j) \in E$ and 0 otherwise. For weighted graphs, $A_{ij} = w_{ij}$ if $(i, j) \in E$ and 0 otherwise.
- **Degree Matrix (D)**: A diagonal matrix with $D_{ii} = d_i$.
- **Incidence Matrix (M)**: For an undirected graph with arbitrary orientation, $M_{ie} = 1$ if vertex i is the head of edge e, $M_{ie} = -1$ if it is the tail, and $M_{ie} = 0$ otherwise.
- **Laplacian Matrix (L)**: $\mathbf{L} = \mathbf{D} - \mathbf{A}$, or equivalently, $\mathbf{L} = \mathbf{M}\mathbf{M}^T$.

Def **Laplacian as a Differential Operator.** The graph Laplacian can be interpreted as a discrete analog of the continuous Laplace operator ∇^2. For a function $f : V \to \mathbb{R}$ defined on vertices, the Laplacian applied to f at vertex i is:

$$(\mathbf{L}f)_i = \sum_{j:(i,j)\in E} w_{ij}(f_i - f_j)$$

This measures how f at vertex i differs from its neighbors, similar to how the continuous Laplacian measures the deviation of a function from its average value in an infinitesimal neighborhood.

Def **Quadratic Form.** The quadratic form of the Laplacian provides a measure of function smoothness across the graph:

$$
\begin{aligned}
f^T \mathbf{L} f &= \sum_{i=1}^{n} \sum_{j=1}^{n} L_{ij} f_i f_j && (51)\\
&= \sum_{i=1}^{n} d_i f_i^2 - \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ij} f_i f_j && (52)\\
&= \sum_{i=1}^{n} \sum_{j:(i,j)\in E} w_{ij} f_i^2 - \sum_{i=1}^{n} \sum_{j:(i,j)\in E} w_{ij} f_i f_j && (53)\\
&= \frac{1}{2} \sum_{i=1}^{n} \sum_{j:(i,j)\in E} w_{ij} (f_i^2 + f_j^2 - 2 f_i f_j) && (54)\\
&= \frac{1}{2} \sum_{i=1}^{n} \sum_{j:(i,j)\in E} w_{ij} (f_i - f_j)^2 && (55)
\end{aligned}
$$

The last expression is a weighted sum of squared differences across edges, clearly non-negative, proving $\mathbf{L}$ is positive semidefinite.

Def **Matrix Form of Laplacian.** If we identify the elements w_{ij} as defining an adjacency matrix $\mathbf{W}$, then the discrete Laplacian in quadratic form for a vector input (x) can be written as

$$
\begin{aligned}
\mathbf{x}^T \mathbf{L} \mathbf{x} &= \sum_{i\sim j} w_{ij} (\mathbf{x}_i - \mathbf{x}_j)^2 && (56)\\
&= \sum_{i\sim j} w_{ij} \mathbf{x}_i^2 + \sum_{i\sim j} w_{ij} \mathbf{x}_j^2 - \sum_{i\sim j} 2 w_{ij} \mathbf{x}_i \mathbf{x}_j && (57)\\
&= \sum_{i\sim j} \mathbf{d}_j \mathbf{x}_i^2 + \sum_{i\sim j} \mathbf{d}_i \mathbf{x}_j^2 - 2 \sum_{i\sim j} w_{ij} \mathbf{x}_i \mathbf{x}_j && (58)\\
&= \frac{1}{2} (\mathbf{x}^T \mathbf{D} \mathbf{x} + \mathbf{x}^T \mathbf{D} \mathbf{x} - 2 \mathbf{x}^T \mathbf{W} \mathbf{x}) && (59)\\
&= \mathbf{x}^T (\mathbf{D} - \mathbf{W}) \mathbf{x}. && (60)
\end{aligned}
$$

Where $\mathbf{D}$ is the degree diagonal matrix and $\mathbf{W}$ is a matrix encoding w_{ij} values. The expression is divided by half in the third step as $\sum_{i\sim j} = \frac{1}{2} \sum_i \sum_j$, so we do not count repeated maps in the permutation sum ($\therefore \mathbf{D}_{ij} = \mathbf{D}_{ji}$). The previous derivation implies the following well-known relationship for the *Laplacian matrix*

Def **Spectral Properties of the Laplacian.** The spectrum of the Laplacian matrix $\mathbf{L}$ consists of eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$ with corresponding eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$.
Key spectral properties include:

- **Zero Eigenvalue**: $\lambda_1 = 0$ always, with eigenvector $\mathbf{v}_1 = \mathbf{1}/\sqrt{n}$ (the normalized constant vector).

- **Component Count**: The multiplicity of $\lambda_1 = 0$ equals the number of connected components in G.
- **Algebraic Connectivity**: $\lambda_2 > 0$ if and only if G is connected. This value, known as the Fiedler value, quantifies how well-connected the graph is.
- **Diameter Bound**: For a connected graph, $\text{diam}(G) \geq \frac{2}{\lambda_2}$, where $\text{diam}(G)$ is the maximum shortest path length between any two vertices.
- **Cheeger Inequality**: $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2\lambda_2}$, where $h(G)$ is the Cheeger constant (isoperimetric number) measuring the graph's "bottleneckedness."

Example Complete and Path GraphsConsider two extremes: the complete graph K_n and the path graph P_n.
For K_n, every vertex connects to all others, so $d_i = n-1$ for all i. The Laplacian has eigenvalues:

- $\lambda_1 = 0$ (multiplicity 1)
- $\lambda_2 = \lambda_3 = \ldots = \lambda_n = n$ (multiplicity $n-1$)

For P_3 (path with 3 vertices), the Laplacian is:

$$\mathbf{L} = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}$$

Its eigenvalues are $\lambda_1 = 0$, $\lambda_2 = 2 - \sqrt{2} \approx 0.59$, $\lambda_3 = 2 + \sqrt{2} \approx 3.41$, with corresponding eigenvectors:

$$\mathbf{v}_1 = \frac{1}{\sqrt{3}}(1, 1, 1)^T \tag{61}$$

$$\mathbf{v}_2 = \frac{1}{\sqrt{2 + (2 - \sqrt{2})^2}}(1, \sqrt{2} - 1, -1)^T \tag{62}$$

$$\mathbf{v}_3 = \frac{1}{\sqrt{2 + (2 + \sqrt{2})^2}}(1, -(\sqrt{2} + 1), 1)^T \tag{63}$$

The low λ_2 for P_3 compared to K_3 reflects its weaker connectivity.

Def **Variational Characterization.** The Courant-Fischer theorem provides a variational characterization of Laplacian eigenvalues:

$$\lambda_k = \min_{\substack{S \subset \mathbb{R}^n, \dim(S) = k \\ S \perp \text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_{k-1}\}}} \max_{\mathbf{x} \in S, \mathbf{x} \neq \mathbf{0}} \frac{\mathbf{x}^T \mathbf{L} \mathbf{x}}{\mathbf{x}^T \mathbf{x}}$$

For the second eigenvalue (Fiedler value):

$$\lambda_2 = \min_{\mathbf{x}\perp\mathbf{1},\mathbf{x}\neq\mathbf{0}} \frac{\mathbf{x}^T\mathbf{L}\mathbf{x}}{\mathbf{x}^T\mathbf{x}} = \min_{\mathbf{x}\perp\mathbf{1},\mathbf{x}\neq\mathbf{0}} \frac{\sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2}{\sum_{i=1}^n x_i^2}$$

This connects the Fiedler value to the optimal solution of a continuous relaxation of the graph partitioning problem.

Note The Fiedler vector $\mathbf{v}_2$ carries significant structural information about the graph. Its components can be interpreted as an embedding of vertices on a line that minimizes the sum of squared distances between connected vertices, subject to orthogonality to the constant vector and unit norm constraint. This provides a natural ordering of vertices that tends to place connected vertices close together.

Figure 1: Graph with vertices embedded according to Fiedler vector values. The signs of the components suggest a natural bipartition.

Def **Normalized Laplacians.** Two important normalized versions of the Laplacian address the bias towards high-degree vertices in the standard Laplacian:

- **Symmetric Normalized Laplacian**: $\mathbf{L}_{sym} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$
- **Random Walk Normalized Laplacian**: $\mathbf{L}_{rw} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$

These matrices have eigenvalues in $[0, 2]$. While $\mathbf{L}_{sym}$ is symmetric, $\mathbf{L}_{rw}$ is not generally symmetric but relates directly to random walks on the graph. The matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ represents transition probabilities of a random walk, where P_{ij} is the probability of moving from vertex i to vertex j.
The two normalized Laplacians share the same eigenvalues. If $\mathbf{v}$ is an eigenvector of $\mathbf{L}_{sym}$ with eigenvalue λ, then $\mathbf{D}^{-1/2}\mathbf{v}$ is an eigenvector of $\mathbf{L}_{rw}$ with the same eigenvalue.

Spectral Clustering Algorithm. Spectral clustering leverages the Laplacian's eigenvectors to partition graphs:

Algorithm 1 Spectral Clustering

```
Construct the graph Laplacian L (or normalized version)
Compute eigenvectors v_1, v_2, ..., v_k for the k smallest eigenvalues
Form matrix X = [v_2, ..., v_k] ∈ R^{n×(k−1)} (skip v_1)
Let row i of X represent vertex i in R^{k−1}
Apply k-means clustering to these rows
Assign vertex i to cluster j if row i is assigned to cluster j
```

This algorithm maps the graph to a low-dimensional space where standard Euclidean clustering methods can be applied. The eigenvectors provide coordinates that capture the graph's community structure.

Def **Graph Cuts and Spectral Partitioning.** For a partition of vertices V into disjoint sets A and $B = V \setminus A$, the cut between them is defined as:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

Several normalized cut objectives have been proposed:

- **RatioCut**: $\text{RatioCut}(A, B) = \frac{\text{cut}(A,B)}{|A|} + \frac{\text{cut}(A,B)}{|B|}$
- **NCut (Normalized Cut)**: $\text{NCut}(A, B) = \frac{\text{cut}(A,B)}{\text{vol}(A)} + \frac{\text{cut}(A,B)}{\text{vol}(B)}$

where $\text{vol}(A) = \sum_{i \in A} d_i$ is the volume of set A.
Minimizing RatioCut is NP-hard, but a relaxation leads to:

$$\min_{\mathbf{f} \perp \mathbf{1}, \|\mathbf{f}\| = \sqrt{n}} \mathbf{f}^T \mathbf{L} \mathbf{f}$$

The solution is the Fiedler vector $\mathbf{v}_2$. Similarly, minimizing NCut relates to the second eigenvector of $\mathbf{L}_{rw}$.

Note The connection between spectral graph theory and random walks provides further insight. The transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ describes a Markov process on the graph. Its eigenvalues relate to mixing times and convergence rates. For a connected, non-bipartite graph, a random walk converges to a stationary distribution π with $\pi_i = \frac{d_i}{\sum_j d_j}$.
The convergence rate is governed by the spectral gap $1 - |\eta_2|$, where η_2 is the second largest eigenvalue of $\mathbf{P}$ in absolute value. This relates to the Laplacian through $\eta_i = 1 - \lambda_i$ for the eigenvalues of $\mathbf{L}_{rw}$.
This spectral perspective on random walks has applications in PageRank algorithms, diffusion maps, and semi-supervised learning on graphs.

Def **Heat Kernel and Diffusion.** The graph Laplacian governs diffusion processes on graphs through the heat equation:

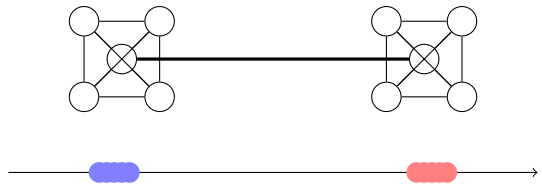
$$\frac{\partial f}{\partial t} = -\mathbf{L} f$$

The solution is $f(t) = e^{-t\mathbf{L}} f(0)$, where $e^{-t\mathbf{L}}$ is the heat kernel. Using the spectral decomposition $\mathbf{L} = \sum_{i=1}^{n} \lambda_i \mathbf{v}_i \mathbf{v}_i^T$, the heat kernel is:

$$e^{-t\mathbf{L}} = \sum_{i=1}^{n} e^{-t\lambda_i} \mathbf{v}_i \mathbf{v}_i^T$$

This exponentially dampens high-frequency components associated with large eigenvalues while preserving low-frequency components associated with small eigenvalues, functioning as a low-pass filter.
The heat kernel trace $\text{Tr}(e^{-t\mathbf{L}}) = \sum_{i=1}^{n} e^{-t\lambda_i}$ captures the graph's global properties and relates to the concept of "shape" in spectral geometry.

Example Visualizing the Fiedler VectorConsider the barbell graph consisting of two complete graphs K_5 connected by a single edge.



Fiedler vector embedding

The Fiedler vector for this graph has approximately constant values within each clique, with opposite signs, reflecting the natural partition. The algebraic connectivity λ_2 is very small, indicating the graph can be easily disconnected.

Def **Matrix Tree Theorem.** The Matrix Tree Theorem provides a remarkable connection between the Laplacian spectrum and spanning trees. For a connected graph G, the number of spanning trees $\tau(G)$ is:

$$\tau(G) = \frac{1}{n} \prod_{i=2}^{n} \lambda_i$$

where λ_i are the non-zero eigenvalues of the Laplacian. Equivalently, for any i:

$$\tau(G) = \det(\mathbf{L}_{i,i})$$

where $\mathbf{L}_{i,i}$ is the (i, i)-minor of $\mathbf{L}$, obtained by removing the i-th row and column. This theorem connects the purely combinatorial concept of spanning tree enumeration with the analytical properties of the Laplacian spectrum.

6.3 The PageRank Algorithm

PageRank is a celebrated algorithm, famously developed and used by Google, to measure the importance or "rank" of web pages. The core idea is that the importance of a page is determined not just by its content, but also by the number and quality of other pages that link to it. A link from an important page is considered more valuable than a link from a less important one. PageRank models this concept using a random surfer navigating the web graph, with a page's rank being proportional to the likelihood of the surfer landing on that page.

Def **The Web Graph and Random Surfer.** The World Wide Web can be modeled as a directed graph $G = (V, E)$, where V is the set of web pages (vertices) and a directed edge $(p_j, p_i) \in E$ exists if page p_j contains a hyperlink to page p_i. Let $N = |V|$ be the total number of pages.
The PageRank algorithm is often explained via the "random surfer" model:

- A user starts on a random web page.

- At each step, the user randomly clicks on one of the outbound links on the current page to navigate to a new page.
- If the current page has no outbound links (a "dangling node"), the user jumps to another page chosen randomly from all N pages.

The PageRank of a page p_i, denoted $PR(p_i)$, is the long-term probability that the random surfer will be on page p_i. Pages visited more frequently by the random surfer are considered more important.

Def **Basic PageRank Formulation.** Let $PR(p_i)$ be the PageRank score of page p_i. Let $B(p_i)$ be the set of pages that link to page p_i. For any page $p_j \in B(p_i)$, let $L(p_j)$ be the number of outbound links from page p_j. The basic idea is that page p_j "distributes" its PageRank score equally among all the pages it links to. Thus, the PageRank of p_i is the sum of the PageRank contributions from all pages linking to it:

$$PR(p_i) = \sum_{p_j \in B(p_i)} \frac{PR(p_j)}{L(p_j)}$$

This is a recursive definition: the rank of a page depends on the rank of other pages.

Matrix Formulation and Power IterationLet $\mathbf{r}$ be a column vector of PageRank scores, where $r_i = PR(p_i)$, and $\sum_i r_i = 1$. We can define a hyperlink transition matrix $\mathbf{H} \in \mathbb{R}^{N \times N}$ (sometimes denoted $\mathbf{M}^T$ or $\mathbf{P}^T$ if $\mathbf{P}$ is row-stochastic) where:

$$H_{ij} = \begin{cases} 1/L(p_j) & \text{if page } p_j \text{ links to page } p_i \text{ and } L(p_j) > 0 \\ 0 & \text{otherwise} \end{cases}$$

The PageRank equation can then be written as an eigenvector problem: $\mathbf{r} = \mathbf{H}\mathbf{r}$. The PageRank vector $\mathbf{r}$ is the principal eigenvector of $\mathbf{H}$ corresponding to the eigenvalue 1. This can be found using the power iteration method: 1. Initialize $\mathbf{r}^{(0)}$ (e.g., $r_i^{(0)} = 1/N$ for all i). 2. Iterate: $\mathbf{r}^{(k+1)} = \mathbf{H}\mathbf{r}^{(k)}$. 3. Repeat until convergence.
However, this basic formulation has issues:

- **Dangling Nodes:** Pages with no outbound links ($L(p_j) = 0$). Columns in $\mathbf{H}$ corresponding to dangling nodes would be all zero, causing rank to "leak out" of the system during power iteration.
- **Sink Regions (Rank Traps):** If the graph has groups of pages that link to each other but not to outside pages (or if the graph is not strongly connected), the random surfer can get trapped, and power iteration may not converge to a unique, meaningful PageRank distribution.

Def

The Damped PageRank Algorithm (Google Matrix). To address the issues of dangling nodes and sink regions, and to ensure convergence to a unique positive PageRank vector, the damping factor d (typically $d \approx 0.85$) is introduced. The random surfer model is modified:

- With probability d, the surfer clicks a random link on the current page.
- With probability $1-d$, the surfer jumps to a random page chosen uniformly from all N pages in the web graph.

The PageRank formula becomes:

$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in B(p_i)} \frac{PR(p_j)}{L(p_j)}$$

This leads to the Google Matrix $\mathbf{G}$. Let $\mathbf{S} \in \mathbb{R}^{N \times N}$ be the column-stochastic hyperlink matrix:

$$S_{ij} = \begin{cases} 1/L(p_j) & \text{if page } p_j \text{ links to page } p_i \text{ and } L(p_j) > 0 \\ 1/N & \text{if page } p_j \text{ is a dangling node (i.e., } L(p_j) = 0) \\ 0 & \text{otherwise (no link and not dangling contribution)} \end{cases}$$

Note: The $1/N$ for dangling nodes ensures $\mathbf{S}$ is column-stochastic (columns sum to 1). The Google Matrix $\mathbf{G}$ is then defined as:

$$\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J}$$

where $\mathbf{J}$ is an $N \times N$ matrix of all ones. The matrix $\mathbf{G}$ is column-stochastic, irreducible, and primitive. By the Perron-Frobenius theorem, $\mathbf{G}$ has a unique largest eigenvalue $\lambda_{\max} = 1$, and its corresponding eigenvector $\mathbf{r}$ (the PageRank vector) has all positive entries and is unique up to scaling. We scale $\mathbf{r}$ such that $\|\mathbf{r}\|_1 = \sum_i r_i = 1$. The PageRank vector $\mathbf{r}$ is the solution to $\mathbf{r} = \mathbf{G}\mathbf{r}$.

Solving for PageRank with the Google MatrixThe PageRank vector $\mathbf{r}$ is typically computed using the power iteration method: 1. Initialize $\mathbf{r}^{(0)}$ with $r_i^{(0)} = 1/N$ for all $i = 1, \ldots, N$. Ensure $\|\mathbf{r}^{(0)}\|_1 = 1$. 2. Iterate for $k = 0, 1, 2, \ldots$:

$$\mathbf{r}^{(k+1)} = \mathbf{G}\mathbf{r}^{(k)} = d\mathbf{S}\mathbf{r}^{(k)} + \frac{1-d}{N}\mathbf{J}\mathbf{r}^{(k)}$$

Since $\mathbf{J}\mathbf{r}^{(k)} = \mathbf{1}(\mathbf{1}^T\mathbf{r}^{(k)}) = \mathbf{1}(1) = \mathbf{1}$ (as $\|\mathbf{r}^{(k)}\|_1 = 1$), the iteration simplifies to:

$$\mathbf{r}^{(k+1)} = d\mathbf{S}\mathbf{r}^{(k)} + \frac{1-d}{N}\mathbf{1}$$

3. Continue until convergence, i.e., $\|\mathbf{r}^{(k+1)} - \mathbf{r}^{(k)}\|_1 < \epsilon$ for some small tolerance ϵ.

This iterative process is efficient because the matrix $\mathbf{S}$ (or the original hyperlink graph) is typically very sparse, even though $\mathbf{G}$ itself is dense. The term $d\mathbf{S}\mathbf{r}^{(k)}$ can be computed by exploiting the sparse structure of web links. The $(1-d)/N \cdot \mathbf{1}$ term represents the teleportation probability uniformly distributed.

Example Illustrative PageRank CalculationConsider a simple web graph with three pages: P_1, P_2, P_3. Links: $P_1 \to P_2$, $P_2 \to P_1$, $P_2 \to P_3$, $P_3 \to P_1$. Out-degrees: $L(P_1) = 1$, $L(P_2) = 2$, $L(P_3) = 1$. No dangling nodes. Let $d = 0.85$. $N = 3$. $(1-d)/N = 0.15/3 = 0.05$.
The matrix $\mathbf{S}$:

- Column 1 (P_1 links to P_2): $S_{21} = 1/1 = 1$. Others 0.
- Column 2 (P_2 links to P_1, P_3): $S_{12} = 1/2 = 0.5$, $S_{32} = 1/2 = 0.5$. Others 0.
- Column 3 (P_3 links to P_1): $S_{13} = 1/1 = 1$. Others 0.

$$\mathbf{S} = \begin{pmatrix} 0 & 0.5 & 1 \\ 1 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix}$$

Initialize $\mathbf{r}^{(0)} = (1/3, 1/3, 1/3)^T \approx (0.333, 0.333, 0.333)^T$.
Iteration 1: $\mathbf{r}^{(1)} = d\mathbf{S}\mathbf{r}^{(0)} + \frac{1-d}{N}\mathbf{1}\ d\mathbf{S}\mathbf{r}^{(0)} = 0.85 \begin{pmatrix} 0 & 0.5 & 1 \\ 1 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix} \begin{pmatrix} 1/3 \\ 1/3 \\ 1/3 \end{pmatrix} = 0.85 \begin{pmatrix} 0.5 \cdot 1/3 + 1 \cdot 1/3 \\ 1 \cdot 1/3 \\ 0.5 \cdot 1/3 \end{pmatrix}$
$0.85 \begin{pmatrix} 1.5/3 \\ 1/3 \\ 0.5/3 \end{pmatrix} = \begin{pmatrix} 0.425 \\ 0.283 \\ 0.142 \end{pmatrix}$ (approx) $\frac{1-d}{N}\mathbf{1} = \begin{pmatrix} 0.05 \\ 0.05 \\ 0.05 \end{pmatrix}$ $\mathbf{r}^{(1)} = \begin{pmatrix} 0.425 \\ 0.283 \\ 0.142 \end{pmatrix} + \begin{pmatrix} 0.05 \\ 0.05 \\ 0.05 \end{pmatrix} =$
$\begin{pmatrix} 0.475 \\ 0.333 \\ 0.192 \end{pmatrix}$ (Sum = 1.0)

Iteration 2: $\mathbf{r}^{(2)} = d\mathbf{S}\mathbf{r}^{(1)} + \frac{1-d}{N}\mathbf{1}\ d\mathbf{S}\mathbf{r}^{(1)} = 0.85 \begin{pmatrix} 0 & 0.5 & 1 \\ 1 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix} \begin{pmatrix} 0.475 \\ 0.333 \\ 0.192 \end{pmatrix} = 0.85 \begin{pmatrix} 0.5(0.333) + 1(0.19 \\ 1(0.475) \\ 0.5(0.333) \end{pmatrix}$
$0.85 \begin{pmatrix} 0.1665 + 0.192 \\ 0.475 \\ 0.1665 \end{pmatrix} = 0.85 \begin{pmatrix} 0.3585 \\ 0.475 \\ 0.1665 \end{pmatrix} \approx \begin{pmatrix} 0.3047 \\ 0.4038 \\ 0.1415 \end{pmatrix}$ $\mathbf{r}^{(2)} \approx \begin{pmatrix} 0.3047 \\ 0.4038 \\ 0.1415 \end{pmatrix} +$
$\begin{pmatrix} 0.05 \\ 0.05 \\ 0.05 \end{pmatrix} = \begin{pmatrix} 0.3547 \\ 0.4538 \\ 0.1915 \end{pmatrix}$ (Sum $\approx$ 1.0)
Iterations would continue until convergence. P_1 appears to receive the most rank due to two direct incoming sources (one of which is P_2 that splits its rank).

Note

- **Probability Distribution:** The PageRank vector $\mathbf{r}$ represents a stationary probability distribution of the random surfer model with teleportation. r_i is the long-term probability of finding the surfer on page p_i.

- **Damping Factor (d):** The choice of d balances between the link structure of the web and the uniform "teleportation" probability. A higher d gives more weight to the link structure, while a lower d makes ranks more uniform. It also ensures convergence and helps mitigate rank sinks.

- **Scalability:** PageRank is designed to scale to billions of pages due to the sparse nature of the web graph and the efficiency of the power iteration method (when implemented carefully).

- **Personalized/Topic-Specific PageRank:** The teleportation term ($\frac{1-d}{N}\mathbf{1}$) can be modified to jump to a specific subset of pages or pages related to a topic, leading to personalized or topic-sensitive rankings. Instead of jumping to any page with uniform probability $1/N$, the jump can be biased towards a personalized vector $\mathbf{v}$, leading to $\mathbf{r}^{(k+1)} = d\mathbf{S}\mathbf{r}^{(k)} + (1-d)\mathbf{v}$.

- **Robustness:** While PageRank provides a robust measure of importance, it is not entirely immune to manipulation (e.g., link farms), leading to ongoing refinements in search engine algorithms.

PageRank revolutionized web search by providing a query-independent measure of page quality, which, when combined with content relevance, significantly improved search results.